

SYSTEM DESIGN GUIDE

A Complete Backend & Distributed Systems Reference

13 Chapters • 130+ Interview Q&As • Production Patterns

Topics Covered

- › HTTP Status Codes & Error Handling
 - › Monolith vs Microservices
 - › Load Balancing
- › CDN & Advanced System Design
- › Database Design (SQL, NoSQL, ACID)
 - › Caching Strategies & Redis
 - › Content Delivery Networks
 - › Horizontal vs Vertical Scaling
- › Async Processing & Message Queues
 - › Authentication & Authorization
- › CAP Theorem & Distributed Systems
 - › Indexing & Query Optimization
- › GraphQL — Modern API Design
 - › Fault Tolerance & Reliability
- › How to Approach System Design

2024 Edition

TABLE OF CONTENTS

System Design Guide — Complete Reference

Chapter 1 — HTTP Status Codes & Error Handling

- › What Are HTTP Status Codes
- › 1xx — Informational Responses
- › 2xx — Success Responses (200, 201, 202, 204)
- › 3xx — Redirection Responses (301, 302, 304, 307, 308)
- › 4xx — Client Error Responses (400, 401, 403, 404, 409, 422, 429)
- › 5xx — Server Error Responses (500, 502, 503, 504)
- › Choosing the Right Status Code
- › Error Response Body Design
- › Status Codes in REST vs GraphQL
- › Interview Questions & Answers

Chapter 2 — Monolith vs Microservices

- › What a Monolith Really Looks Like
- › Where Monoliths Start Breaking
- › What Microservices Actually Change
- › Hidden Complexity in Microservices
- › When to Move to Microservices
- › Migration Strategy: The Strangler Pattern
- › Real-World Examples (Amazon, Netflix)
- › Trade-off Summary
- › Interview Questions & Answers

Chapter 3 — Load Balancing

- › What Problem Does Load Balancing Solve
- › The Core Idea
- › How It Actually Works
- › Load Balancing Algorithms (Round Robin, Least Connections, IP Hash)
- › Why Load Balancing Is Powerful
- › Health Checks, Sticky Sessions, Layer 4 vs Layer 7
- › Common Tools (NGINX, HAProxy, AWS ELB)
- › Interview Questions & Answers

Chapter 4 — Advanced Load Balancing & System Design

- › Load Balancing in Microservices
- › Reverse Proxy vs Load Balancer
- › System Design: Serving 1 Million Users
- › CDN and Load Balancer Working Together
- › Full System Component Summary
- › Interview Questions & Answers

Chapter 5 — Database Design Handbook

- › What Is a Transaction
- › ACID Properties

- › SQL Databases (PostgreSQL, MySQL)
- › NoSQL Databases
- › Apache Cassandra
- › MongoDB
- › Cassandra vs MongoDB Comparison
- › SQL vs NoSQL Decision Guide
- › Polyglot Persistence
- › Interview Questions & Answers

Chapter 6 — Caching in Web Systems

- › Why Do We Need Caching
- › Browser Cache, CDN Cache, Server Cache
- › Cache Hit vs Cache Miss
- › Caching Strategies (Cache-Aside, Write-Through, Write-Back, Read-Through)
- › TTL — Time To Live
- › Cache Invalidation
- › Cache Stampede, Penetration, Stale Data
- › Real-World Example: User Profile API
- › Interview Questions & Answers

Chapter 7 — Content Delivery Networks (CDN)

- › What Is a CDN and How It Works
- › CDN as Consumer vs CDN as Infrastructure
- › Impact of Not Using a Dedicated CDN
- › What to Serve via CDN
- › CDN Provider Comparison
- › Best Practices (Cache-Control, Versioning, Compression)
- › Before vs After Comparison
- › Interview Questions & Answers

Chapter 8 — Horizontal vs Vertical Scaling

- › What Is Scaling
- › Vertical Scaling (Scale Up)
- › Horizontal Scaling (Scale Out)
- › Key Differences Comparison Table
- › Why Modern Systems Prefer Horizontal Scaling
- › Challenges (State, Consistency, DB Bottleneck, Observability)
- › The Hybrid Approach
- › Auto-Scaling in Cloud Environments
- › Interview Questions & Answers

Chapter 9 — Asynchronous Processing & Message Queues

- › The Problem with Synchronous Processing
- › Solution: Asynchronous Processing
- › Key Components (Producer, Queue, Worker)
- › BullMQ — Redis-Backed Job Queue
- › Amazon SQS — Managed Queue Service
- › Apache Kafka — Event Streaming Platform
- › BullMQ vs SQS vs Kafka Comparison
- › Idempotency, Retry, DLQ, Visibility Timeout
- › Real-World Multi-Queue Architecture

- › Interview Questions & Answers

Chapter 10 — Advanced Authentication & Authorization

- › Authentication vs Authorization
- › JWT Structure and Two-Token System
- › Login API, Middleware, Token Refresh (Node.js)
- › Role-Based Access Control (RBAC)
- › Authentication in Microservices
- › Service-to-Service Auth and mTLS
- › OAuth 2.0 and OpenID Connect
- › Interview Questions & Answers

Chapter 11 — CAP Theorem

- › What Is the CAP Theorem
- › Consistency, Availability, Partition Tolerance Explained
- › CP Systems vs AP Systems vs CA Systems
- › What Happens During a Network Partition
- › Eventual Consistency and Conflict Resolution
- › PACELC Theorem
- › Database Classification (PostgreSQL, Cassandra, MongoDB, DynamoDB)
- › Decision Framework: CP vs AP
- › Interview Questions & Answers

Chapter 12 — Indexing & Query Optimization

- › What Is an Index and Why It Matters
- › Full Table Scan vs Index Lookup
- › How B-Tree Indexes Work Internally
- › Types of Indexes (Single, Composite, Unique, Full-Text, Hash)
- › The Left-Prefix Rule for Composite Indexes
- › When Indexes Are NOT Used (Pitfalls)
- › Query Optimization Techniques
- › EXPLAIN Plans — Reading Query Execution
- › Indexing in NoSQL (MongoDB, Cassandra)
- › Common Mistakes and When to Avoid Indexes
- › Interview Questions & Answers

Chapter 13 — GraphQL — Modern API Design

- › What Is GraphQL and Why It Was Created
- › GraphQL vs REST — Key Differences
- › Core Concepts: Schema, Types, Queries, Mutations, Subscriptions
- › Resolvers — How GraphQL Fetches Data
- › The N+1 Problem in GraphQL and DataLoader
- › GraphQL in Microservices — Schema Stitching & Federation
- › Authentication & Authorization in GraphQL
- › Performance Considerations and Persisted Queries
- › When to Use GraphQL vs REST
- › Interview Questions & Answers

Chapter 14 — Fault Tolerance & Reliability

- › What Is Fault Tolerance and Why It Matters
- › Failure Modes in Distributed Systems

- › Retry Mechanisms — Strategies and Backoff Patterns
- › Circuit Breaker Pattern — States and Implementation
- › Timeout Patterns
- › Bulkhead Pattern — Isolating Failures
- › Redundancy — Active-Active vs Active-Passive
- › Health Checks and Self-Healing Systems
- › Graceful Degradation
- › Chaos Engineering
- › Interview Questions & Answers

Final Chapter — How to Approach System Design

- › The 8-Step Framework for Any System Design Problem
- › Step 1 — Clarify Requirements (Functional & Non-Functional)
- › Step 2 — Estimate Scale (Users, Data, Throughput)
- › Step 3 — Define APIs
- › Step 4 — Choose Architecture (Monolith vs Microservices)
- › Step 5 — Design the Database Layer
- › Step 6 — Add Caching
- › Step 7 — Plan Scaling
- › Step 8 — Handle Failures
- › Putting It All Together — A Worked Example
- › System Design Interview Cheat Sheet

HTTP Status Codes & Error Handling

What every status code means, when to use it, and why it matters

1. What Are HTTP Status Codes?

HTTP status codes are three-digit numbers returned by a server in response to every HTTP request. They tell the client — whether that is a browser, a mobile app, or another service — whether the request succeeded, failed, or requires some further action. Status codes are the primary communication channel between a server and its clients about the outcome of an operation.

Status codes are grouped into five classes based on their leading digit. Each class represents a distinct category of response, and understanding these categories deeply is essential for building APIs that communicate clearly, for debugging production issues quickly, and for designing systems that handle errors gracefully.

Core rule: Status codes are not optional decoration — they are a contract. A client must be able to rely on them to decide what to do next: retry, redirect, display an error, or process the response.

1xx — Informational	(request received, processing continues)
2xx — Success	(request was received, understood, and accepted)
3xx — Redirection	(further action needed to complete the request)
4xx — Client Error	(the request contains bad syntax or cannot be fulfilled)
5xx — Server Error	(the server failed to fulfil a valid request)

2. 1xx — Informational Responses

Informational responses indicate that the server has received the request and is continuing to process it. These are rarely encountered in everyday API development but appear in specific HTTP/1.1 and HTTP/2 scenarios.

100 Continue

The server has received the request headers and is asking the client to proceed with sending the request body. Used when a client wants to check whether the server will accept a large request (e.g. a file upload) before sending the full payload.

```
Client sends: Expect: 100-continue header
Server responds: 100 Continue → client proceeds to send body
                  417 Expectation Failed → client abandons
```

101 Switching Protocols

The server is switching to a different protocol as requested by the client. Most commonly seen when upgrading an HTTP connection to a WebSocket connection.

```
Client: GET /chat HTTP/1.1
        Upgrade: websocket
        Connection: Upgrade

Server: HTTP/1.1 101 Switching Protocols
        Upgrade: websocket
        Connection: Upgrade
        → WebSocket connection is now established
```

103 Early Hints

Allows the server to send preliminary response headers before the final response is ready. Used to hint at resources the browser should start preloading (CSS, fonts, scripts) while the server is still generating the main response, improving perceived page load time.

3. 2xx — Success Responses

The 2xx class indicates that the client's request was successfully received, understood, and accepted. Choosing the specific 2xx code communicates important nuance about what happened — not every success is the same.

200 OK

The most general success response. The request succeeded and the response body contains the requested data. Used for successful GET, PUT, PATCH, and POST requests when there is a response body to return.

```
GET /users/42      → 200 OK + user object in body
PUT /users/42      → 200 OK + updated user object
POST /auth/login   → 200 OK + { accessToken, refreshToken }
```

Common mistake:

Returning 200 for everything — including errors — with an error flag in the body. This breaks every HTTP client, monitoring system, and API consumer that relies on status codes to detect failures.

```
// WRONG — never do this:
{ status: 200, error: true, message: 'User not found' }

// CORRECT:
HTTP 404 Not Found
{ error: 'USER_NOT_FOUND', message: 'No user with id 42 exists' }
```

201 Created

A new resource was successfully created as a result of the request. Should be returned for POST requests that create a new resource. The response should include a Location header pointing to the new resource and typically returns the created resource in the body.

```
POST /users
Body: { name: 'John', email: 'john@example.com' }

→ 201 Created
Location: /users/42
Body: { id: 42, name: 'John', email: 'john@example.com', createdAt: '...' }
```

202 Accepted

The request has been accepted for processing but the processing has not been completed yet — and may never be. Used for asynchronous operations where the work has been queued and the result will be available later.

```
POST /reports/generate
→ 202 Accepted
Body: { jobId: 'report-abc123', status: 'queued',
      statusUrl: '/reports/report-abc123/status' }

// Client polls the statusUrl to check when the report is ready
```

202 communicates clearly: 'We got your request and we are working on it, but do not wait here — go check back later.' This is the correct code for any async processing scenario: video encoding, bulk data imports, PDF generation, or any operation that exceeds a reasonable HTTP timeout.

204 No Content

The request succeeded but there is no response body to return. The correct code for DELETE operations that do not return the deleted resource, and for PUT/PATCH operations when the updated resource is not returned.

```
DELETE /users/42      → 204 No Content   (no body)
PUT /users/42/status  → 204 No Content   (acknowledged, no body returned)
```

Common mistake:

Returning 200 with an empty body instead of 204. While often harmless, 204 is semantically correct and signals to clients that they should not attempt to parse a response body.

206 Partial Content

The server is returning only part of the resource, as requested by the client via a Range header. Used for resumable downloads, video streaming, and large file transfers where the client wants to fetch a specific byte range.

```
GET /videos/lecture.mp4
Range: bytes=0-1048575

→ 206 Partial Content
Content-Range: bytes 0-1048575/52428800
Content-Length: 1048576
[first 1MB of the video]
```

4. 3xx — Redirection Responses

The 3xx class indicates that the client must take additional action to complete the request — typically following a redirect to a different URL. Choosing the correct redirect code has significant implications for SEO, browser behaviour, and HTTP method preservation.

301 Moved Permanently

The resource has been permanently moved to a new URL. All future requests should use the new URL. Browsers and HTTP clients permanently update their cached URL. Search engines transfer link equity (PageRank) to the new URL.

```
GET http://example.com/old-page
→ 301 Moved Permanently
Location: https://example.com/new-page
```

Use cases:

- HTTP → HTTPS redirect

- Domain migration (old-domain.com → new-domain.com)
- Permanent URL restructuring

Important: Browsers may change POST requests to GET after following a 301. If you need to preserve the HTTP method, use 308 instead.

302 Found (Temporary Redirect)

The resource is temporarily at a different URL. The client should continue using the original URL for future requests. Search engines do not transfer link equity for 302s. Historically browsers changed POST to GET on 302 — use 307 to preserve the method.

Common uses:

- Redirect after login to the originally requested page
- Temporary maintenance redirect
- A/B testing redirects

304 Not Modified

The resource has not changed since the client last fetched it (determined by ETag or Last-Modified headers). The server sends no body — the client should use its cached version. This is how HTTP conditional caching works, dramatically reducing bandwidth for repeated requests.

Client first request:

```
GET /api/config  
→ 200 OK ETag: 'abc123' Body: { ... config ... }
```

Client subsequent request:

```
GET /api/config  
If-None-Match: 'abc123'  
→ 304 Not Modified (no body — client uses its cached version)
```

307 Temporary Redirect

Like 302, but explicitly guarantees that the HTTP method and body are preserved when following the redirect. A POST to a URL returning 307 will be followed with another POST to the Location URL.

308 Permanent Redirect

Like 301, but explicitly guarantees that the HTTP method and body are preserved. The modern, method-safe replacement for 301 in APIs. Use when you permanently move a POST endpoint and clients must continue sending POST requests to the new URL.

```
301 / 302 → browser may convert POST to GET
307 / 308 → method is always preserved (POST stays POST)
```

5. 4xx — Client Error Responses

The 4xx class indicates that the request contains an error on the client's side — bad syntax, missing authorisation, a non-existent resource, or a conflict with server state. These errors are the client's responsibility to fix. Retrying the same request without changing it will not help.

400 Bad Request

The server cannot process the request because of a client-side error — malformed JSON, missing required fields, invalid parameter values, or a request body that fails validation. This is the most general client error.

```
POST /users
Body: { name: '', email: 'not-an-email' }

→ 400 Bad Request
{
  error: 'VALIDATION_ERROR',
  message: 'Request validation failed',
  details: [
    { field: 'name', message: 'Name is required' },
    { field: 'email', message: 'Must be a valid email address' }
  ]
}
```

Common causes:

- Malformed JSON in the request body
- Missing required fields
- Field values that fail validation rules (too short, wrong format, out of range)
- Invalid query parameter types (passing a string where an integer is expected)

401 Unauthorized

Authentication is required and has either not been provided or has failed. Despite the name, this is an authentication error — the client has not proved who it is. The response must include a WWW-Authenticate header specifying how to authenticate.

```
GET /api/profile
(no Authorization header)

→ 401 Unauthorized
```

```
WWW-Authenticate: Bearer realm='api'
{ error: 'AUTHENTICATION_REQUIRED',
  message: 'A valid Bearer token is required' }
```

Common causes:

- No Authorization header provided
- JWT token has expired
- JWT signature is invalid
- API key is missing or malformed

401 vs 403: 401 means 'I do not know who you are — please authenticate.' 403 means 'I know who you are, but you are not allowed to do this.'

403 Forbidden

The server understood the request and the client is authenticated, but the client does not have permission to access the requested resource. Unlike 401, re-authenticating will not help — the server knows who you are, and the answer is still no.

```
DELETE /users/42
Authorization: Bearer <valid-token-for-user-99>

→ 403 Forbidden
{ error: 'FORBIDDEN',
  message: 'You do not have permission to delete this user' }
```

Common causes:

- Authenticated user has insufficient role or permissions (RBAC violation)
- User is trying to access or modify another user's private resource
- IP address is blocked or not in the allowlist
- Resource requires a higher subscription tier than the user has

404 Not Found

The server cannot find the requested resource. Either the resource never existed, it has been deleted, or the URL is wrong. This is the most universally recognised HTTP status code. It should also be returned when a server wants to hide the existence of a resource for security reasons (for example, not revealing whether a user ID exists to an unauthenticated caller).

```
GET /users/9999
→ 404 Not Found
{ error: 'USER_NOT_FOUND',
  message: 'No user with id 9999 exists' }
```

Common causes:

- ID or slug does not exist in the database
- Resource was deleted
- Typo in the URL path
- Route not registered on the server (missing endpoint)

405 Method Not Allowed

The HTTP method used is not supported for the requested resource. The response must include an Allow header listing the supported methods.

```
DELETE /auth/login
→ 405 Method Not Allowed
Allow: GET, POST
{ error: 'METHOD_NOT_ALLOWED',
  message: 'DELETE is not supported on this endpoint' }
```

408 Request Timeout

The server timed out waiting for the request. The client did not send a complete request within the server's timeout window. The client may repeat the request without modifications.

```
Common causes:
• Client started sending a large request but took too long
• Network interruption during request transmission
• Client connection dropped mid-request
```

409 Conflict

The request conflicts with the current state of the server. Used when a request cannot be completed because of a conflict with an existing resource — a duplicate entry, an optimistic locking violation, or a state machine transition that is not allowed.

```
POST /users
Body: { email: 'john@example.com' } // already registered

→ 409 Conflict
{ error: 'EMAIL_ALREADY_EXISTS',
  message: 'An account with this email address already exists' }

PUT /orders/99
If-Match: 'version-3' // server is now at version-5

→ 409 Conflict
{ error: 'OPTIMISTIC_LOCK_CONFLICT',
  message: 'Resource was modified by another request - re-fetch and retry' }
```

Common causes:

- Duplicate entry — unique constraint violation (email, username, order reference)
- Optimistic concurrency conflict — someone else updated the resource first
- State machine violation — trying to cancel an already-delivered order
- Concurrent modification of the same resource by two simultaneous requests

410 Gone

The resource existed but has been permanently deleted and will not be available again. Unlike 404, which is ambiguous (maybe it never existed, maybe it was deleted), 410 is definitive: it existed and it is gone for good. Search engines will remove 410 URLs from their index faster than 404s.

```
GET /articles/deprecated-post-2019
→ 410 Gone
{ error: 'RESOURCE_DELETED',
  message: 'This article has been permanently removed' }
```

422 Unprocessable Entity

The request is well-formed (valid JSON, correct Content-Type) but the semantic content is invalid — the server understands the request but cannot process it because the data fails business logic validation. This is more specific than 400, which covers syntactic errors.

```
POST /orders
Body: { productId: 42, quantity: -5 }

→ 422 Unprocessable Entity
{ error: 'INVALID_QUANTITY',
  message: 'Quantity must be a positive integer' }

POST /transfers
Body: { amount: 10000, fromAccountId: 1 } // account has £50

→ 422 Unprocessable Entity
{ error: 'INSUFFICIENT_FUNDS',
  message: 'Account balance is insufficient for this transfer' }
```

400 vs 422:

- 400 Bad Request — the request itself is malformed (invalid JSON, wrong data type, missing required field)
- 422 Unprocessable Entity — the request is syntactically valid but semantically fails business rules

429 Too Many Requests

The client has sent too many requests in a given time window and is being rate limited. The response should include a `Retry-After` header indicating how long the client should wait before retrying.

```
GET /api/search?q=test
(100th request in 60 seconds - limit is 60/min)

→ 429 Too Many Requests
Retry-After: 47
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1719000060
{ error: 'RATE_LIMIT_EXCEEDED',
  message: 'Too many requests. Please retry after 47 seconds.' }
```

Implementation considerations:

- Always include `Retry-After` so clients know when to retry
- Rate limit by IP for unauthenticated endpoints, by user ID for authenticated ones
- Use sliding window or token bucket algorithms rather than fixed window to prevent burst exploitation

6. 5xx — Server Error Responses

The 5xx class indicates that the server encountered an error that prevented it from fulfilling a valid request. These are server-side failures — the client's request was correct, but something went wrong on the server. Unlike 4xx errors, 5xx errors may be worth retrying (with appropriate backoff).

500 Internal Server Error

A generic server-side error occurred. Something unexpected went wrong that the server does not have a more specific code for. This is the catch-all for unhandled exceptions and bugs that escape to the HTTP layer.

```
GET /users/42

→ 500 Internal Server Error
{ error: 'INTERNAL_SERVER_ERROR',
  message: 'An unexpected error occurred',
  requestId: 'req-abc123' // for log correlation
}
```

Common causes:

- Unhandled exception in application code — null pointer dereference, unexpected data format
- Database query error — syntax error in generated SQL, unexpected result shape
- Out of memory — application running out of heap space
- Dependency throwing an unexpected error that is not caught

Security: Never expose stack traces, internal error messages, or implementation details in 500 responses. Log the full error server-side and return only a generic message with a requestId for correlation.

501 Not Implemented

The server does not support the functionality required to fulfil the request. The method is valid but not implemented. Indicates a planned feature that is not yet available, rather than a permanent limitation.

```
PATCH /users/42
→ 501 Not Implemented
{ error: 'NOT_IMPLEMENTED',
  message: 'PATCH is not yet supported - use PUT to update the full resource'
}
```

502 Bad Gateway

The server, acting as a gateway or proxy, received an invalid response from the upstream server it was trying to reach. This indicates a problem between your load balancer, API gateway, or reverse proxy and the backend application server.

```
Request path:
  Client → Load Balancer → App Server

502 means: App Server returned an invalid/unparseable response to the Load
Balancer
```

Common causes:

- App server crashed while processing a request — load balancer received a broken response
- App server returned a response the proxy could not parse
- Deployment in progress — new version briefly returning malformed responses during startup
- App server running out of memory and returning garbage

503 Service Unavailable

The server is currently unable to handle the request — either because it is overloaded or down for maintenance. Unlike 500, which indicates an unexpected error, 503 is typically planned or predictable. The response should include a Retry-After header.

```
All app servers are at capacity or undergoing maintenance:
→ 503 Service Unavailable
  Retry-After: 120
  { error: 'SERVICE_UNAVAILABLE',
    message: 'Service is temporarily unavailable. Please retry in 2 minutes.' }
```

Common causes:

- Planned maintenance window — servers deliberately taken offline
- Server overloaded — connection pool exhausted, CPU at 100%, out of threads
- Circuit breaker open — the service is shedding load intentionally
- Database unreachable — the readiness check fails, traffic is stopped

504 Gateway Timeout

The server, acting as a gateway or proxy, did not receive a timely response from the upstream server. The upstream server is reachable but is responding too slowly — it took longer than the gateway's timeout allows.

```
Request path:
  Client → Load Balancer (30s timeout) → App Server (taking 45s)

→ 504 Gateway Timeout
  { error: 'GATEWAY_TIMEOUT',
    message: 'The upstream service did not respond within the expected time' }
```

502 vs 503 vs 504 — the key distinction:

- 502 Bad Gateway — upstream server responded but with an invalid/unexpected response
- 503 Service Unavailable — the server itself is down, overloaded, or in maintenance
- 504 Gateway Timeout — upstream server is reachable but responding too slowly

507 Insufficient Storage

The server cannot store the representation needed to complete the request. Used for file upload endpoints when the server or a user's storage quota has been exceeded.

```
POST /files/upload (5GB file, user quota is 2GB)
→ 507 Insufficient Storage
  { error: 'STORAGE_QUOTA_EXCEEDED',
    message: 'Upload exceeds your 2GB storage limit' }
```

7. Choosing the Right Status Code

Situation	Correct Code	Wrong Code
Resource created successfully	201 Created	200 OK
Request accepted, processing async	202 Accepted	200 OK
Delete succeeded, no body	204 No Content	200 OK
No token provided	401 Unauthorized	403 Forbidden
Token valid but insufficient permission	403 Forbidden	401 Unauthorized
Resource does not exist	404 Not Found	400 Bad Request
Duplicate email on registration	409 Conflict	400 Bad Request
Business rule violation (negative qty)	422 Unprocessable Entity	400 Bad Request
Rate limit exceeded	429 Too Many Requests	403 Forbidden
Unhandled server exception	500 Internal Server Error	200 OK
Upstream service unresponsive	503 Service Unavailable	500 Internal Server Error
Gateway timeout from upstream	504 Gateway Timeout	500 Internal Server Error

8. Error Response Body Design

A well-designed error response body is as important as the correct status code. The status code tells the client what category of problem occurred; the body tells it exactly what went wrong and how to fix it.

8.1 What a Good Error Body Contains

```
{
  error:      string,    // machine-readable error code in SCREAMING_SNAKE_CASE
  message:    string,    // human-readable description of what went wrong
  requestId:  string,    // unique ID for log correlation (essential for
                        debugging)
  details?:  array,      // field-level errors for validation failures
  docs?:     string,     // URL to relevant documentation
}
```

```
}

// Good validation error:
{
  error:      'VALIDATION_ERROR',
  message:    'Request validation failed',
  requestId:  'req-7f3a9b2c',
  details: [
    { field: 'email',    message: 'Must be a valid email address' },
    { field: 'password', message: 'Must be at least 8 characters' }
  ]
}
```

8.2 What Never to Include

- Stack traces — they expose your tech stack, file paths, and internal logic to attackers
- Database error messages — ORA-00942, relation 'users' does not exist — reveals schema details
- Internal service names or IP addresses — exposes your infrastructure topology
- Sensitive data from the failed operation — passwords, tokens, PII

8.3 The requestId Pattern

Every error response should include a `requestId` — a unique identifier for that specific request that is also written to your server logs. When a user reports an error, they can provide the `requestId` and you can instantly find the full context in your logs — the exact request, the full stack trace, and the chain of events that led to the failure.

```
// Middleware that generates and attaches a requestId:
app.use((req, res, next) => {
  req.requestId = crypto.randomUUID();
  res.setHeader('X-Request-Id', req.requestId);
  next();
});

// Global error handler:
app.use((err, req, res, next) => {
  logger.error({ requestId: req.requestId, error: err });
  res.status(500).json({
    error: 'INTERNAL_SERVER_ERROR',
    message: 'An unexpected error occurred',
    requestId: req.requestId
  });
});
```

9. Status Codes in REST vs GraphQL

One of the most common points of confusion for engineers moving between REST and GraphQL is how each treats HTTP status codes.

9.1 REST — Status Codes Carry Full Meaning

In REST, the HTTP status code is the primary signal. Every outcome has a specific code. Clients inspect the status code first to decide how to handle the response.

```
GET /users/999    → 404 Not Found    (resource missing)
POST /users       → 201 Created      (resource created)
DELETE /users/1   → 403 Forbidden    (permission denied)
```

9.2 GraphQL — Almost Always 200

GraphQL returns 200 OK for almost every response — even when the query fails or returns errors. This is by design: a single GraphQL request can partially succeed, with some fields resolved successfully and others failing. The errors array in the response body carries the actual error information.

```
POST /graphql
Body: { query: '{ user(id: 999) { name } }' }

→ 200 OK
{
  data: { user: null },
  errors: [{ message: 'User not found', locations: [...],
    extensions: { code: 'USER_NOT_FOUND' } }]
}
```

GraphQL only uses non-200 codes for request-level failures — not operation-level failures. A 400 is returned if the GraphQL query itself is syntactically invalid (cannot be parsed). A 401 or 403 may be returned if the entire request is blocked at the authentication layer before GraphQL even processes it.

Interview Questions & Answers

Q1: What is the difference between 401 and 403?

- 401 Unauthorized means the request lacks valid authentication credentials — the client has not proved who it is

- 403 Forbidden means the client is authenticated but does not have permission to perform the requested action
- 401 says 'I do not know who you are'; 403 says 'I know who you are, but you cannot do this'
- Retrying with a valid token may fix 401; it will not fix 403 unless permissions change

Q2: When should you use 400 vs 422?

- 400 Bad Request: the request is syntactically malformed — invalid JSON, missing required fields, wrong data type
- 422 Unprocessable Entity: the request is syntactically valid but fails business logic validation — negative quantity, insufficient funds, invalid state transition
- 400 is about the structure of the request; 422 is about the content violating domain rules

Q3: What is the difference between 502, 503, and 504?

- 502 Bad Gateway: the upstream server returned an invalid or unexpected response to the gateway
- 503 Service Unavailable: the server itself is down, overloaded, or in planned maintenance
- 504 Gateway Timeout: the upstream server is reachable but responded too slowly — the gateway timed out waiting

Q4: Why should you never return 200 with an error in the body?

- Monitoring systems alert on 5xx/4xx rates — a 200 with an error body is invisible to these systems
- HTTP clients, API gateways, and load balancers make routing decisions based on status codes
- Circuit breakers track failure rates using status codes — false 200s mask failures from the breaker
- Client libraries automatically retry 5xx responses; they never retry 200 responses

Q5: When would you use 202 Accepted instead of 200 OK?

- 202 means the request was accepted and queued for asynchronous processing — the work is not done yet
- Use it when the operation will take longer than a reasonable HTTP timeout: report generation, video encoding, bulk imports
- The response body should include a job ID and a URL where the client can poll for status
- 200 means the operation is complete and the result is in the response body

Q6: What is the difference between 301 and 308?

- Both are permanent redirects — the resource has moved and all future requests should use the new URL
- 301 may cause browsers to change POST requests to GET when following the redirect
- 308 guarantees the HTTP method is preserved — a POST request is followed with another POST
- Prefer 308 for API endpoint redirects to avoid unexpected method changes

Q7: What information should an error response body always contain?

- error: a machine-readable error code in SCREAMING_SNAKE_CASE for programmatic handling
- message: a human-readable description of what went wrong
- requestId: a unique identifier for the request so engineers can correlate errors with server logs
- details: for validation errors, field-level breakdown of each specific validation failure
- Never include: stack traces, internal error messages, database errors, or sensitive data

Q8: What status code does GraphQL usually return, and why?

- GraphQL almost always returns 200 OK, even for operation errors
- Because a single request can partially succeed — some fields resolve, others fail
- Errors are reported in the errors array in the response body alongside any successful data
- Non-200 codes are used only for request-level failures: 400 for invalid query syntax, 401/403 for auth failures before GraphQL processes the request

Q9: When should you return 204 No Content instead of 200?

- 204 is correct when the request succeeded but there is no meaningful body to return
- DELETE operations that do not return the deleted resource
- PUT or PATCH operations that acknowledge the update without returning the updated resource
- Sending 204 explicitly tells the client not to attempt to parse a response body — which is cleaner than 200 with an empty body

Q10: How should a rate-limited 429 response be handled by the client?

- Read the Retry-After header — it specifies how many seconds to wait before retrying
- Do not retry immediately — this will just result in more 429s and potentially a longer ban
- Implement exponential backoff with jitter if no Retry-After header is present
- Surface a user-friendly message — 'You are sending too many requests. Please wait a moment.'
- In automated systems, implement a token bucket or sliding window rate limiter on the client side to avoid hitting the limit in the first place

Final Takeaways

Mental Model: Status codes are a conversation between client and server. 2xx says 'success'. 3xx says 'look elsewhere'. 4xx says 'you did something wrong'. 5xx says 'we did something wrong'. Use the right code for every outcome — it is the first thing every engineer, monitoring system, and API consumer will look at.

- Use 201 for created resources, 202 for async acceptance, 204 for no-body success — 200 is not the right code for every success
- 301/302 may change POST to GET — use 307/308 when the HTTP method must be preserved
- 401 is an authentication failure; 403 is an authorisation failure — they are not interchangeable
- 400 is for malformed requests; 422 is for semantically invalid requests that pass syntax checks
- 502 is a bad upstream response; 503 is server unavailability; 504 is upstream timeout — each points to a different failure location
- Always include a requestId in error responses — it is the bridge between what users report and what logs contain
- Never expose stack traces, database errors, or internal details in error responses — log them server-side only

1. What a Monolith Really Looks Like

In reality, a monolith isn't just 'one file'—it's usually a well-structured application with multiple internal modules that share a single deployment unit.

Typical Monolith Structure:

```
App
├── Auth Module
├── Payment Module
└── Order Module
```



Key Insight: Even if the code is modular, the deployment is not. Everything runs in one process and one deployment unit.

2. Where Monoliths Start Breaking

Monoliths work very well in the early stages. Problems only appear when scale increases. Here are the four primary breaking points:

2.1 Scaling Problem

Imagine a scenario where the Payments module is heavily used while Auth is lightly used. In a monolith, you must scale the entire application — not just the overloaded service.

- Payments module receives 10x the traffic of Auth
- Monolith forces you to scale everything equally
- Result: Wasted infrastructure resources and increased costs

2.2 Deployment Bottleneck

Every small change requires a full redeployment of the entire application, raising the risk of introducing unrelated bugs.

- One small change → redeploy the whole app
- Risk of breaking unrelated features
- Teams become afraid to touch shared code

2.3 Team Coordination Chaos

As teams grow, working on a shared monolithic codebase becomes increasingly painful:

- 10+ developers working on the same codebase simultaneously
- Frequent merge conflicts slow down velocity
- Slower release cycles

2.4 Technology Lock-in

Monoliths make it difficult to use the best tool for each job. Want to use Python for ML and Node.js for APIs? Not easily possible in a standard monolith.

3. What Microservices Actually Change

Instead of one big application, microservices decompose functionality into independent, deployable services — each with its own database.

Microservices Architecture:

```
Auth Service      → own DB
Payment Service   → own DB
Order Service     → own DB
```

Each service has three key properties:

- Runs independently
- Deploys independently
- Scales independently

3.1 Real Difference in Request Flow

Monolith Request Flow:

```
Client → One Server → One Database
```

Microservices Request Flow:

```
Client → API Gateway → Multiple Services → Multiple DBs
```

This provides flexibility, but also introduces significant complexity.

4. Hidden Complexity in Microservices

People often underestimate the operational burden that comes with microservices. Here are the four major complexity areas:

4.1 Network Calls Instead of Function Calls

In a monolith, service communication is a simple function call. In microservices, it becomes a network call that is slower and can fail.

Monolith:

```
payment.process()
```

Microservices:

```
HTTP call → payment-service
```

4.2 Data Consistency Problems

A single database in a monolith makes transactions straightforward. With multiple databases, you must handle distributed transactions — one of the hardest problems in distributed systems.

- Partial failures — some services succeed, others fail
- Distributed transactions are complex to implement correctly
- Eventual consistency models require careful design

4.3 Debugging Challenges

When an error occurs, tracking it down across multiple services requires dedicated tooling:

- Centralized logging systems (e.g., ELK Stack)
- Distributed tracing tools (e.g., Jaeger, Zipkin)
- Error could originate from Auth, Payment, Network, or elsewhere

4.4 DevOps Overhead

Microservices significantly increase operational complexity:

- Multiple independent deployments to manage
- Container technology required (Docker)
- Orchestration platform needed (Kubernetes)
- Service discovery, load balancing, and health checks per service

5. When Should You Move to Microservices?

Rule of Thumb: Delay microservices until pain forces you. Microservices solve scaling problems, but introduce operational complexity.

Stay with Monolith If:

- Small team (1–10 developers)
- Early-stage startup — product is still evolving
- No significant performance bottlenecks yet
- Simple deployment requirements

Move to Microservices If:

- Hitting clear scaling bottlenecks

- Multiple teams stepping on each other's work
- Need for independent deployments per service
- Different services have dramatically different load patterns
- Technology diversity is genuinely needed

6. Migration Strategy: The Strangler Pattern

You don't rewrite everything at once. The Strangler Pattern is the industry-standard approach for incrementally migrating from a monolith to microservices.

Step-by-Step Migration:

1. Start with your existing monolith — don't rush to break it apart
2. Identify one high-value, high-load module (e.g., Payments)
3. Extract it into a standalone service with its own database
4. Route requests to the new service via an API Gateway
5. Monitor, stabilize, then repeat for the next module

7. Real-World Examples

Amazon

Amazon started as a monolithic application in the early days of e-commerce. As the platform grew massively in traffic and team size, they hit severe scaling and coordination limits. They famously broke their architecture into microservices, which eventually evolved into AWS — the cloud infrastructure that now powers much of the internet.

Netflix

Netflix migrated from a monolithic DVD rental system to a global microservices-based streaming platform. The migration was driven by the need for massive scalability, fault tolerance, and independent deployment of hundreds of features. Netflix has also open-sourced many of the tools they built during this journey (Hystrix, Eureka, Zuul).

8. Trade-off Summary

Feature	Monolith	Microservices	Notes
Simplicity	✓ Easy	✗ Complex	Monolith wins early on
Deployment	✗ Full app redeploy	✓ Independent	Microservices win at scale
Scaling	✗ Limited	✓ Flexible	Microservices allow per-service scaling
Debugging	✓ Easier	✗ Hard	Monolith has one codebase to trace
Speed (early stage)	✓ Fast	✗ Slow setup	Monolith is faster to build initially

9. The Real Rule

Instead of just "Start monolith → move to microservices", think: "Delay microservices until pain forces you."

The architecture decision is not about what's trendy — it's about what solves your current, real pain. Many successful companies run monoliths at significant scale. Microservices are a solution to specific organizational and technical problems, not a default best practice.

Interview Questions & Answers

Q1: What is the main difference between a monolith and microservices?

- A monolith is a single deployable unit where all modules share one process and one database
- Microservices split functionality into independent services, each with its own database and deployment lifecycle
- The key difference is deployment independence — in a monolith you redeploy everything; in microservices you deploy only the changed service

Q2: When should you NOT use microservices?

- Small teams of 1–10 developers — the operational overhead outweighs the benefit
- Early-stage products where requirements are still evolving rapidly
- When the team lacks experience with distributed systems, containers, and orchestration
- Rule: if you cannot clearly articulate the pain that microservices solve, stay with the monolith

Q3: What is the Strangler Pattern?

- A migration strategy for incrementally replacing a monolith with microservices
- Instead of a big-bang rewrite, you extract one module at a time into a standalone service
- Requests are progressively routed to the new service while the monolith still handles the rest
- The monolith shrinks gradually until it can be decommissioned entirely

Q4: What are the main challenges of microservices?

- Network failures — service calls can fail; monolith function calls cannot
- Distributed transactions — maintaining consistency across multiple databases is hard
- Operational complexity — each service needs its own CI/CD, monitoring, and logging
- Service discovery — services must find each other dynamically in a distributed environment

Q5: How do microservices communicate with each other?

- Synchronously via REST (HTTP/JSON) or gRPC — the caller waits for a response
- Asynchronously via message queues (Kafka, SQS, RabbitMQ) — the caller fires and forgets
- Synchronous communication is simpler but creates tight coupling; async is more resilient

Q6: What is service discovery in microservices?

- The mechanism by which services locate each other dynamically at runtime
- Services register their address with a service registry (e.g. Consul, Eureka) on startup
- Clients query the registry to find available instances of a target service
- Kubernetes provides built-in service discovery via DNS and environment variables

Q7: How do you handle data consistency across microservices?

- The Saga pattern — a sequence of local transactions, each publishing events to trigger the next
- Eventual consistency — accept that data may be momentarily inconsistent but will converge
- Avoid distributed transactions (two-phase commit) — they are slow and brittle in practice
- Design operations to be idempotent so retries are safe

Q8: What is an API Gateway?

- A single entry point that routes requests to the appropriate microservice
- Handles cross-cutting concerns: authentication, rate limiting, SSL termination, logging
- Decouples clients from the internal service topology — clients never call services directly
- Examples: Kong, AWS API Gateway, NGINX, Traefik

Q9: What is a distributed monolith and why is it worse than a regular monolith?

- A system that has been split into multiple services but remains tightly coupled at runtime
- Services share a database, synchronously depend on each other, and cannot be deployed independently
- It carries all the complexity of microservices with none of the benefits
- Symptom: you must deploy all services together even when only one changes

Q10: How do you monitor microservices?

- Centralised logging — all services ship logs to a single aggregation system (ELK, Loki)
- Distributed tracing — a trace ID follows each request across services (Jaeger, Zipkin)
- Metrics dashboards — aggregate health, latency, and error rates per service (Prometheus, Grafana)
- Alerting — automated alerts when error rates or latencies exceed thresholds

CHAPTER 3

Load Balancing

Making systems real-world scalable

1. What Problem Does Load Balancing Solve?

Imagine a single server receiving 10,000 simultaneous user requests. Without any distribution mechanism, the result is predictable and catastrophic:

- Drastically slow responses as the server becomes overwhelmed
- Application crashes under memory and CPU pressure
- Complete downtime — all users are affected at once

2. The Core Idea

Load balancing solves this by introducing a traffic manager between users and servers. Instead of routing all requests to one machine, a load balancer distributes them intelligently across a pool of servers.

Without Load Balancing:

```
Users → ONE Server
```

With Load Balancing:

```
Users → Load Balancer → Server A
                        → Server B
                        → Server C
```

Key Insight: The user never knows multiple servers exist. From their perspective, they are talking to one system.

3. How It Actually Works (Step-by-Step)

6. User sends a request (e.g., opens a webpage or calls an API)
7. Request hits the load balancer — the single entry point
8. Load balancer picks a backend server using its algorithm
9. Selected server processes the request
10. Response travels back to the user transparently

4. Load Balancing Algorithms

The algorithm determines which server receives each incoming request. Choosing the right one depends on your infrastructure and traffic patterns.

4.1 Round Robin

Requests are distributed one-by-one to each server in sequence. After the last server, it cycles back to the first.

```
User 1 → Server A
User 2 → Server B
User 3 → Server C
User 4 → Server A (cycle restarts)
```

- Good: Simple to implement and understand
- Good: Works well when all servers have identical capacity
- Problem: Ignores actual server load — a busy server gets the same traffic as an idle one

4.2 Least Connections

Each new request is sent to the server currently handling the fewest active connections. This is smarter than Round Robin because it accounts for real-time load.

Example: If Server A has 100 active users and Server B has 20, the next request goes to Server B.

- Good: More intelligent distribution — adapts to actual load
- Good: Better for workloads where request processing times vary
- Problem: Slightly more complex to implement than Round Robin

4.3 IP Hash (Sticky Sessions)

The client's IP address is hashed to consistently map to the same backend server. The same user always reaches the same server.

- Useful for: Sessions stored in server memory (not a shared store)
- Useful for: Ensuring user experience continuity in stateful applications
- Problem: Can cause uneven load if users share IP patterns (e.g., behind a corporate proxy)
- Problem: Scaling down servers breaks session affinity

5. Why Load Balancing Is Powerful

5.1 Horizontal Scaling

Instead of upgrading one expensive server (vertical scaling), you simply add more servers to the pool. The load balancer automatically distributes traffic to the new capacity.

+ 1 Server → more capacity immediately available

5.2 High Availability

If one server crashes or becomes unresponsive, the load balancer detects this via health checks and automatically stops routing traffic to it. Users are seamlessly redirected to healthy servers with zero downtime.

5.3 Better Performance

- Traffic is spread evenly — no single server becomes a bottleneck
- Faster response times across the board
- Consistent user experience regardless of total system load

6. Real-World Example

When millions of users simultaneously browse Amazon or stream on Netflix, their requests are distributed across hundreds — sometimes thousands — of servers. Load balancers sit at every layer of the infrastructure, routing traffic intelligently based on geography, server health, and current load. This is what makes global-scale availability possible.

7. Advanced Concepts

7.1 Health Checks

The load balancer continuously probes each backend server to verify it is alive and responding correctly. If a server fails a health check, it is removed from the rotation until it recovers.

- Prevents traffic from being sent to crashed or degraded servers
- Enables automatic recovery — the server rejoins rotation when healthy again

7.2 Sticky Sessions vs Stateless Architecture

Sticky sessions tie a user to one specific server, which is necessary when session state is stored locally. However, modern systems prefer stateless architectures where any server can handle any request, because they scale more gracefully and tolerate failures better.

Best Practice: Design stateless services wherever possible. Store session data in a shared store (Redis, a database) so any server can serve any user.

7.3 Layer 4 vs Layer 7 Load Balancing

Load balancers operate at different levels of the networking stack, each with different trade-offs:

Layer 4 — Transport Level:

- Routes based on IP address and TCP/UDP port
- Faster — does not inspect packet contents
- Good for raw throughput

Layer 7 — Application Level:

- Routes based on HTTP URL, headers, cookies, or request body
- Smarter routing — can direct `/api/payments` to one cluster and `/api/auth` to another
- Enables A/B testing, canary deployments, and content-based routing

8. Common Load Balancing Tools

Tool	Type	Best For
NGINX	Software (L4 + L7)	Web servers, reverse proxy, high performance
HAProxy	Software (L4 + L7)	TCP/HTTP load balancing, very high reliability
AWS ELB / ALB	Cloud-managed	AWS-native apps, auto-scaling integration
Google Cloud LB	Cloud-managed	Global load balancing across GCP regions

9. Trade-offs

Advantages:

- Scalability — add servers to grow capacity instantly

- Reliability — no single point of failure for your application servers
- Performance — even traffic distribution prevents bottlenecks

Disadvantages:

- Extra infrastructure — the load balancer itself must be managed and made highly available
- Slight latency — each request passes through an additional network hop
- Configuration complexity — algorithms, health checks, and SSL termination need tuning

10. Mental Model

Think of a load balancer like a restaurant host. Customers arrive at the door and the host assigns them to available tables (servers). Without the host, everyone rushes to the same table — chaos. With the host, flow is smooth, wait times are short, and no table is overwhelmed.

This mental model scales well: as the restaurant gets busier, you add more tables (servers). The host (load balancer) handles the rest automatically.

Interview Questions & Answers

Q1: How does load balancing differ in a microservices architecture?

- In microservices, load balancing happens at multiple hops — not just once at the edge
- Edge load balancer distributes traffic to the API Gateway; API Gateway routes to individual services; each service may have multiple instances with their own balancing
- Service meshes like Istio handle service-to-service load balancing transparently

Q2: What is the difference between a reverse proxy and a load balancer?

- A reverse proxy sits in front of servers, forwarding requests — it handles SSL, caching, compression, and security
- A load balancer is a specialised reverse proxy that also distributes traffic across multiple servers
- All load balancers are reverse proxies, but not all reverse proxies are load balancers

Q3: How would you design a system for 1 million concurrent users?

- CDN for static assets — offload all images, JS, CSS from origin servers
- Multiple horizontally scaled stateless app servers behind a load balancer
- Redis caching layer to absorb read traffic before it hits the database
- Database with primary for writes and multiple read replicas for reads
- Auto-scaling groups to handle traffic spikes automatically

Q4: What is an API Gateway and how does it differ from a load balancer?

- A load balancer distributes traffic across identical server instances
- An API Gateway routes requests to different services based on URL path or headers
- The API Gateway also handles auth, rate limiting, SSL termination, and request transformation
- In microservices, you typically have both: a load balancer in front of the gateway, and the gateway routing to services

Q5: What is client-side load balancing?

- Instead of a centralised load balancer, the client decides which server instance to call
- The client queries a service registry for available instances and applies a load balancing algorithm locally
- Used in microservice-to-microservice communication — Netflix Ribbon and gRPC implement this
- Eliminates the load balancer as a network hop but increases client complexity

Q6: What is a service mesh?

- A dedicated infrastructure layer that handles service-to-service communication in microservices
- Implemented as sidecar proxies deployed alongside each service instance
- Provides load balancing, service discovery, circuit breaking, mTLS, and tracing transparently
- Istio is the most widely deployed service mesh

Q7: What is the circuit breaker pattern?

- Prevents a failing downstream service from causing cascading failures across the system

- When error rates exceed a threshold, the circuit opens and requests fail fast without calling the service
- After a cooldown, a few test requests are allowed through — if they succeed the circuit closes
- Resilience4j is a popular Java implementation; similar libraries exist for Node.js and Go

Q8: How do you handle session persistence in a load-balanced environment?

- Externalise session state to a shared store (Redis) accessible by all servers — true stateless design
- Any server in the pool can then handle any user request without sticky sessions
- Alternatively use sticky sessions (IP hash) — simpler but less resilient to server failures
- Stateless design is strongly preferred because it scales better and handles failures gracefully

Q9: What is geo-based load balancing?

- Routes user requests to the server cluster geographically closest to them
- Reduces latency by minimising the physical distance data must travel
- Implemented via GeoDNS — different DNS responses based on the requester's IP geolocation
- AWS Route 53 latency-based routing and Cloudflare load balancing both support this

Q10: How would you achieve zero-downtime deployments in a load-balanced system?

- Rolling deployment — update one server at a time while others continue serving traffic
- Blue-green deployment — run two identical environments and switch the load balancer atomically
- Canary deployment — route a small percentage of traffic to the new version and gradually increase
- Health checks ensure traffic is only routed to servers that have completed startup successfully

CHAPTER 4

Advanced Load Balancing & System Design

1. Load Balancing in Microservices

In microservices, load balancing does not happen just once at the edge. Because the system is made up of many independent services each potentially running multiple instances, balancing happens at multiple hops throughout the request lifecycle.

Key Insight: In microservices, load balancing happens at multiple hops — not just once at the entry point.

1.1 The Full Request Flow

User → Load Balancer → API Gateway → Services → Databases

1.2 Layers of Load Balancing

Layer 1 — Edge Load Balancer:

- The first entry point into your system
- Distributes incoming traffic across backend clusters or API gateways

Layer 2 — API Gateway:

- Routes each request to the correct microservice based on the URL path
- /login → Auth Service
- /pay → Payment Service

Layer 3 — Service-to-Service Load Balancing:

- Inside the system, each service may have multiple instances running in parallel
- Auth Service: 5 instances; Payment Service: 10 instances
- Requests are distributed across all available instances automatically

1.3 Real Tools

- Kubernetes — built-in load balancing across pods
- Istio — advanced service mesh with fine-grained routing
- NGINX — flexible reverse proxy and load balancer

2. Reverse Proxy vs Load Balancer

A reverse proxy sits in front of servers and forwards requests. A load balancer is a specialized reverse proxy that also distributes traffic across multiple servers.

2.1 Reverse Proxy Responsibilities

- SSL termination — handles HTTPS so backend servers do not have to
- Caching — stores responses to reduce repeated processing
- Security — shields backend servers from direct exposure
- Routing — directs traffic based on URL, headers, or other rules

2.2 The Relationship

- All load balancers are reverse proxies
- Not all reverse proxies are load balancers

Simple Analogy: A reverse proxy is a receptionist. A load balancer is a receptionist who also manages a team of agents and routes every call to whoever is free.

2.3 Example with NGINX

As a reverse proxy only:

```
Client → NGINX → Single Backend Server
```

As a load balancer:

```
Client → NGINX → Server A
                  → Server B
                  → Server C
```

3. System Design: Serving 1 Million Users

Let us walk through designing a web application that must serve 1 million concurrent users reliably with fast response times.

3.1 Requirements

- Users can log in and view data
- Response time must be under 200ms
- System must remain available even if individual components fail

3.2 High-Level Architecture

```
Users
  ↓
CDN (static assets: images, JS, CSS)
  ↓
Load Balancer
  ↓
App Servers (horizontally scaled, stateless)
  ↓
Cache (Redis)
  ↓
Database (Primary + Read Replicas)
```

3.3 Components Explained

CDN:

- Serves static content from edge nodes close to the user
- Dramatically reduces load on origin servers

Load Balancer:

- Distributes all dynamic traffic across the app server pool
- Performs health checks and removes failed servers automatically

App Servers:

- Stateless — handle business logic only, no session data stored locally
- Scaled horizontally: add more servers as traffic grows

Cache (Redis):

- Stores frequently accessed data in memory
- Eliminates redundant database queries for hot data

Database:

- Primary node handles all write operations
- Read replicas handle all read queries, distributing the load

3.4 Scaling Strategy & Bottlenecks

- Add more app servers behind the load balancer as traffic increases
- Add database read replicas to handle growing read volume
- Use caching aggressively — aim to serve 80%+ of reads from cache
- Database overload is usually the first bottleneck; solved with replicas and caching
- Cache misses require careful cache warming and expiry strategy

4. CDN and Load Balancer Working Together

Modern production systems combine CDNs and load balancers at different layers. They divide responsibilities cleanly and work best in combination.

4.1 The Flow

User → CDN → Load Balancer → App Servers → Database

4.2 Division of Responsibilities

CDN:

- Stores and serves static content from edge nodes close to users
- Reduces latency by serving content without hitting origin servers

Load Balancer:

- Handles all dynamic requests that cannot be cached
- Distributes API calls and user interactions across the server pool

4.3 Practical Example

When a user loads a page on a social media platform:

- Profile image → served directly by CDN from the nearest edge node
- API call to fetch posts → goes to load balancer → routed to an app server
- Post data → app server checks Redis cache first, then database if needed

Key Insight: CDN handles static content and edge caching. The load balancer handles dynamic traffic distribution. They solve different problems and work best together.

5. Full System Component Summary

Component	Primary Role	Solves
CDN	Global static content delivery	Latency, origin server load
Load Balancer	Traffic distribution	Scalability, availability
API Gateway	Request routing in microservices	Service discovery, routing

Component	Primary Role	Solves
App Servers	Business logic execution	Processing capacity
Cache (Redis)	Fast in-memory data access	Database load, response speed
Database	Persistent data storage	Durability, consistency

Interview Questions & Answers

Q1: How does load balancing differ in a microservices architecture?

- In microservices, load balancing happens at multiple hops — not just once at the edge
- Edge load balancer distributes traffic to the API Gateway; API Gateway routes to individual services; each service may have multiple instances with their own balancing
- Service meshes like Istio handle service-to-service load balancing transparently without application code changes

Q2: What is the difference between a reverse proxy and a load balancer?

- A reverse proxy sits in front of servers, forwarding requests — it handles SSL, caching, compression, and security
- A load balancer is a specialised reverse proxy that also distributes traffic across multiple servers
- All load balancers are reverse proxies, but not all reverse proxies are load balancers
- NGINX can act as either depending on configuration

Q3: How would you design a system to handle 1 million concurrent users?

- CDN for static assets — offload all images, JS, CSS from origin servers
- Multiple horizontally scaled, stateless app servers behind a load balancer
- Redis caching layer to absorb read traffic before it hits the database
- Database with primary for writes and multiple read replicas for read traffic
- Auto-scaling groups to handle traffic spikes automatically

Q4: What is an API Gateway and how does it differ from a load balancer?

- A load balancer distributes traffic across identical server instances
- An API Gateway routes requests to different services based on the URL path or headers
- The API Gateway also handles auth, rate limiting, SSL termination, and request transformation
- In microservices, you typically have both: a load balancer in front of the gateway, and the gateway routing to services

Q5: What is client-side load balancing?

- Instead of a centralised load balancer, the client itself decides which server instance to call
- The client queries a service registry for available instances and applies a load balancing algorithm locally
- Used in microservice-to-microservice communication — Netflix Ribbon and gRPC implement this pattern
- Eliminates the load balancer as a network hop but increases client complexity

Q6: What is a service mesh?

- A dedicated infrastructure layer that handles service-to-service communication in microservices
- Implemented as sidecar proxies (typically Envoy) deployed alongside each service instance
- Provides load balancing, service discovery, circuit breaking, mTLS, and distributed tracing transparently
- Istio is the most widely deployed service mesh; Linkerd is a lighter-weight alternative

Q7: What is a circuit breaker pattern?

- Prevents a failing downstream service from causing cascading failures across the system
- When error rates for a service exceed a threshold, the circuit 'opens' and requests fail fast without attempting to call the service
- After a cooldown period, a few requests are allowed through to test recovery — if they succeed, the circuit closes
- Implemented by Netflix Hystrix (now deprecated) and Resilience4j

Q8: How do you handle session persistence in a load-balanced environment?

- Externalise session state to a shared store (Redis) accessible by all servers
- Any server in the pool can then handle any user request — true stateless design
- Alternatively, use sticky sessions (IP hash or session cookie) — simpler but less resilient
- Stateless design is strongly preferred because it scales better and handles server failures gracefully

Q9: What is geo-based load balancing?

- Routes user requests to the server cluster geographically closest to them
- Reduces latency by minimising the physical distance data must travel
- Implemented via GeoDNS — different DNS responses based on the requester's IP geolocation
- AWS Route 53 latency-based routing and Cloudflare load balancing both support this pattern

Q10: How would you design for zero-downtime deployments in a load-balanced system?

- Rolling deployment — update one server at a time while others continue serving traffic
- Blue-green deployment — run two identical environments; switch the load balancer from blue to green atomically
- Canary deployment — route a small percentage of traffic to the new version and gradually increase it
- The load balancer's health checks ensure traffic is only sent to servers that have completed startup

CHAPTER 5

Database Design Handbook

SQL vs NoSQL, Transactions, ACID, Cassandra, and MongoDB

1. Fundamental Principle

There is no best database. Choose based on use case, scale, and consistency needs.

2. What Is a Transaction?

A transaction is a group of operations executed as a single unit of work. The core rule is simple: either all operations succeed (COMMIT), or everything rolls back (ROLLBACK). There is no in-between.

2.1 Example: Bank Transfer

Consider transferring money between two accounts. There are two steps: deduct from Account A, then add to Account B.

Without a transaction:

- Step 1 succeeds — money is deducted from Account A
- Step 2 fails — money is never added to Account B
- Result: Data inconsistency — money is lost

With a transaction:

```
BEGIN TRANSACTION
  Deduct from Account A
  Add to Account B
COMMIT -- both succeed, or ROLLBACK if either fails
```

If any step fails, the entire transaction rolls back and the database remains in its original, consistent state.

3. ACID Properties

ACID defines the guarantees that transactional database systems provide. Together they ensure data correctness and reliability.

3.1 Atomicity

- All operations in a transaction succeed, or none of them do
- No partial updates are ever written to the database

3.2 Consistency

- The database always transitions from one valid state to another
- All constraints, rules, and data integrity checks are always honoured

3.3 Isolation

- Concurrent transactions do not interfere with each other
- Each transaction executes as if it were the only one running

3.4 Durability

- Once a transaction is committed, data is permanently stored
- Committed data survives crashes, power failures, and system restarts

4. When Are Transactions Required?

Transactions are essential for:

- Banking and financial systems
- Payment processing
- Order and inventory management
- Any workflow where partial failure causes data corruption

Transactions are not required for:

- Logging and audit trails
- Analytics and reporting
- Social media interactions (likes, views)
- Chat and messaging systems

5. SQL Databases (Relational)

SQL databases organise data into structured tables with rows and columns. Relationships between tables are enforced through foreign keys, and the system guarantees full ACID compliance.

5.1 Advantages

- Strong consistency and full ACID transaction support
- Complex queries using JOINS across multiple tables
- Strict data integrity through constraints and schema enforcement

5.2 Disadvantages

- Harder to scale horizontally — designed for vertical scaling
- Schema changes are rigid and often require migrations
- Less flexible when data structure evolves frequently

5.3 Popular SQL Databases

PostgreSQL:

- Advanced features including strong JSON support
- Highly extensible with custom types, functions, and extensions
- Recommended for modern backend systems requiring reliability

MySQL:

- Simpler to operate and widely adopted
- Fast for read-heavy workloads
- Strong ecosystem and hosting support

5.4 When to Use SQL

- Data is relational and structured
- Transactions and ACID guarantees are required
- Data accuracy is critical to the business

6. NoSQL Databases

NoSQL databases trade strict consistency for flexibility and horizontal scalability. They are designed for high-throughput workloads where data structures are semi-structured or change frequently.

6.1 Types of NoSQL

- Document — MongoDB (JSON-like documents)
- Key-Value — Redis (fast in-memory lookups)
- Wide-Column — Cassandra (query-optimised columns)
- Graph — Neo4j (relationship-heavy data)

6.2 Advantages

- Scales horizontally with ease — add nodes to grow capacity
- Flexible schema — fields can vary between records
- High write and read throughput for large-scale systems

6.3 Disadvantages

- Weak or eventual consistency — not suitable for financial data
- Limited or no multi-document transaction support
- Data duplication is often required for performance

6.4 When to Use NoSQL

- Massive scale is required
- Schema changes frequently during development
- High throughput is the primary requirement
- Data is semi-structured or unstructured

7. Apache Cassandra (Wide-Column NoSQL)

Cassandra is a distributed, wide-column NoSQL database built for extremely high write throughput and availability at scale. Its fundamental design principle is to model data around query patterns rather than relationships.

7.1 Key Features

- Distributed architecture — no single point of failure
- Extremely high write throughput across large clusters
- Linear horizontal scalability — add nodes without downtime
- Eventual consistency — prioritises availability over immediate accuracy
- No JOINS — data must be denormalised

7.2 Data Modelling Approach

Cassandra requires you to design tables around the queries you will run, not the relationships in your data. This means creating multiple tables for different access patterns:

```
orders_by_user    -- query: fetch orders for a user
orders_by_date    -- query: fetch orders in a date range
orders_by_status  -- query: fetch orders by status
```

7.3 Best Use Cases

- Time-series data — metrics, sensor readings, telemetry
- Logging and audit systems at high volume
- IoT data ingestion pipelines
- Messaging and activity feed systems

7.4 Not Suitable For

- Systems requiring ACID transactions
- Complex relational queries with JOINS

- Use cases requiring strong, immediate consistency

8. MongoDB (Document NoSQL)

MongoDB stores data as flexible JSON-like documents, grouping related data together in a single record. This model maps naturally to application objects and eliminates the need for complex JOINS.

8.1 Example Document

```
{
  "user": "John",
  "orders": [
    { "item": "Phone", "price": 500 }
  ]
}
```

8.2 Key Features

- Flexible schema — fields can be added or removed without migrations
- Rich query support including filters, sorting, and nested queries
- Indexing support for fast lookups on any field
- Aggregation framework for complex data transformations
- Limited multi-document transaction support (added in v4.0)

8.3 Design Approach

- Embed related data in a single document when read together
- Reference separate documents only when data is large or reused
- Create indexes on all frequently queried fields

8.4 Best Use Cases

- Web applications and REST APIs
- Content management systems
- User profiles and personalisation
- Product catalogues with varying attributes

8.5 Limitations

- Very large documents can degrade read and write performance
- Not ideal for workloads requiring heavy relational queries

9. Cassandra vs MongoDB

Feature	Cassandra	MongoDB
Data Model	Wide-column	Document (JSON)
Query Flexibility	Low — query-driven schema	High — rich query language
Schema	Query-driven, rigid	Flexible
Transactions	Limited	Moderate (v4.0+)
Scalability	Excellent	Good
Best For	High write, predictable load	Flexible, dev-friendly apps

Key Difference: Cassandra is for predictable, high-scale write systems. MongoDB is for flexible, developer-friendly applications.

10. SQL vs NoSQL Decision Guide

Requirement	SQL	NoSQL
Transactions (ACID)	Yes	No
Strong consistency	Yes	No
High horizontal scalability	No	Yes
Flexible or evolving schema	No	Yes
Complex queries with JOINS	Yes	No

11. Real-World Architecture: Polyglot Persistence

Modern production systems rarely rely on a single database. Instead they use the best database for each job — a pattern called polyglot persistence.

User Data	→ PostgreSQL	(relational, ACID transactions)
Session Cache	→ Redis	(fast key-value lookups)
Logs & Events	→ Cassandra	(high-volume time-series writes)
Content/Profiles	→ MongoDB	(flexible document storage)

Each database is chosen deliberately for the characteristics of the data it stores and the queries that will be run against it.

12. Common Mistakes

- Using NoSQL for transactional systems — risks data loss and inconsistency
- Using Cassandra for relational queries — its data model does not support JOINS
- Ignoring indexing in MongoDB — unindexed queries become full collection scans
- Avoiding SQL completely out of trend-following — many problems are relational
- Overengineering early-stage systems — start simple, scale when pain is real

Interview Questions & Answers

Q1: What is ACID and why does it matter?

- ACID stands for Atomicity, Consistency, Isolation, and Durability
- Atomicity: all operations in a transaction succeed or none do — no partial updates
- Consistency: the database always moves between valid states, never violating constraints
- Isolation: concurrent transactions do not interfere with each other
- Durability: committed data survives crashes — it is persisted permanently

Q2: When would you choose SQL over NoSQL?

- When data is relational and relationships between entities need to be enforced
- When ACID transactions are required — financial systems, order management, inventory
- When complex queries with JOINS across multiple tables are needed
- When data integrity and consistency are more important than raw throughput or schema flexibility

Q3: When would you choose NoSQL over SQL?

- When massive horizontal scalability is required and data does not fit on one machine
- When the schema changes frequently and a rigid table structure would slow development
- When the data is document-like, hierarchical, or semi-structured
- When write throughput is the primary concern and eventual consistency is acceptable

Q4: What is the CAP theorem?

- CAP states that a distributed system can guarantee at most two of three properties: Consistency, Availability, and Partition tolerance
- Consistency: all nodes see the same data at the same time
- Availability: every request receives a response
- Partition tolerance: the system continues operating even if network partitions split nodes
- In practice, network partitions are unavoidable, so the real choice is between CP (consistent) and AP (available) systems

Q5: What is database sharding?

- Sharding partitions data horizontally across multiple database instances, each called a shard
- Each shard holds a subset of the total data — for example, users A-M on shard 1, N-Z on shard 2
- Enables horizontal scaling of the database layer — each shard handles only a fraction of the total query volume
- Challenges include shard key selection, cross-shard queries, and shard rebalancing as data grows

Q6: What is a database index and how does it improve performance?

- An index is a separate data structure (typically a B-tree) that maps column values to the rows that contain them
- Without an index, a query must scan every row in the table (full table scan)
- With an index, the database jumps directly to matching rows — dramatically faster for large tables
- Trade-off: indexes consume disk space and slow down write operations because they must be updated on every insert/update/delete

Q7: What is eventual consistency?

- A consistency model where updates propagate to all nodes over time, but reads may return stale data immediately after a write
- Used by distributed NoSQL systems like Cassandra and DynamoDB to achieve high availability
- The system guarantees that if no new updates are made, all nodes will eventually converge to the same value
- Acceptable for social media likes, view counts, and recommendations — not acceptable for financial balances

Q8: What is the N+1 query problem?

- A performance anti-pattern where fetching a list of N records results in N+1 database queries
- Example: fetch 100 users, then for each user fetch their profile separately — 101 queries total
- Solution: use JOINS or eager loading to fetch all related data in a single query
- ORMs like Sequelize and TypeORM can silently introduce N+1 problems if not configured carefully

Q9: What is database replication?

- Replication maintains one or more copies of the database on separate servers
- Primary-replica: the primary handles writes, replicas handle reads — distributes read load
- Replicas lag behind the primary by a small delay — reads from replicas may return slightly stale data
- Also provides redundancy: if the primary fails, a replica can be promoted to primary

Q10: What is polyglot persistence?

- Using different database technologies within a single application, each optimised for its specific use case
- Example: PostgreSQL for user data, Redis for sessions and caching, Cassandra for logs, MongoDB for content
- Embraces the idea that no single database is the best choice for every type of data
- Introduces operational complexity — teams must operate and maintain multiple database systems

CHAPTER 6

Caching in Web Systems

Strategies, tools, and patterns for 10-100x faster systems

1. Why Do We Need Caching?

Databases are relatively slow. Every query involves disk I/O to read from storage, query planning and execution overhead, and in distributed systems, a network round-trip to the database server. Under high load, the database becomes the primary bottleneck — every user request is queued behind every other request hitting the same slow storage layer.

Without caching — every request hits the database:

```
User → Server → Database → Server → User
```

At low traffic this is acceptable. At scale, the database cannot keep up. Response times degrade, queries pile up, and eventually the system fails under load.

With caching — frequently accessed data is served from memory:

```
User → Server → Cache → Response (fast)
                        → (on miss) → DB → Cache → Response
```

Impact: A well-designed cache can make a system 10-100x faster and reduce database load by 80-90%, enabling the same hardware to serve dramatically more users.

2. What Is a Cache?

A cache is a temporary storage layer that holds copies of frequently accessed data in fast memory (RAM). Because RAM access is orders of magnitude faster than disk access, serving data from cache instead of the database produces dramatically lower latency.

The mental model is simple: think of your database as a large warehouse far away, and your cache as a small, well-stocked shop right next to your customers. Most people buy the same popular items — so keeping those items in the nearby shop means most requests never need to travel to the warehouse at all.

- Database — persistent, accurate, slow (disk-based)
- Cache — temporary, fast, stored in RAM
- The cache holds a subset of the most-requested data
- When the cache does not have the data, it falls back to the database (a cache miss)

3. Popular Caching Tools

3.1 Redis

Redis is the industry-standard caching tool. It is an in-memory data store that supports rich data types beyond simple key-value pairs. Because all data lives in RAM, Redis can serve millions of operations per second with sub-millisecond latency.

- Supports strings, lists, sets, sorted sets, hashes, and more
- TTL (Time To Live) — keys can automatically expire after a set duration
- Pub/Sub messaging for real-time event broadcasting
- Persistence options — can optionally write data to disk for durability
- Clustering support — can be sharded across multiple nodes

Best suited for:

- Session storage and authentication tokens
- API response caching
- Real-time leaderboards and counters
- Rate limiting

3.2 Memcached

Memcached is a simpler, high-performance in-memory key-value store. It has fewer features than Redis but is extremely fast and is still used in systems where simplicity and raw throughput are the priority.

- Pure key-value — no complex data types
- Multi-threaded — can utilise multiple CPU cores more efficiently than Redis
- No persistence — purely ephemeral
- No built-in replication or clustering in the open-source version

Best suited for:

- Simple caching of strings and serialised objects
- Systems that prioritise maximum throughput and simplicity

4. Types of Caching

Caching can occur at multiple layers of the system stack. Understanding where each type sits helps you apply the right tool to the right problem.

4.1 Browser Cache

The browser stores copies of static assets — images, CSS stylesheets, and JavaScript files — locally on the user's device. On subsequent visits, the browser serves these assets from its local cache without making any network request to the server.

- Controlled via HTTP response headers: Cache-Control, ETag, Expires
- Assets are stored on the user's disk, not in RAM

- Eliminates entire network round-trips for repeated static asset loads
- Drastically reduces server bandwidth and cost for media-heavy sites

The browser respects cache control directives from the server. A Cache-Control: max-age=86400 header tells the browser to reuse the cached asset for up to 24 hours before checking for updates.

4.2 CDN Cache

A Content Delivery Network (CDN) caches static and sometimes dynamic content at geographically distributed edge nodes. When a user in Mumbai requests content hosted on a server in New York, the CDN serves it from a nearby edge node instead, dramatically reducing latency.

```
User → CDN Edge Node → (on miss) → Origin Server → CDN → User
```

- Reduces latency for global users by serving from the nearest edge
- Absorbs massive traffic spikes — the origin server is protected
- Popular CDNs: Cloudflare, AWS CloudFront, Akamai, Fastly
- Can cache not just static assets but also API responses with proper configuration

4.3 Server-Side Cache (Application Cache)

Server-side caching happens on the backend, typically using Redis or Memcached. The application server checks the cache before querying the database. If the data is found, it is returned immediately. If not, the server fetches from the database and populates the cache for future requests.

```
Server → Redis Cache → (on miss) → Database → populate cache → return
```

- In-memory cache (Redis) — store database query results, computed values
- Application cache — store results of expensive business logic computations
- Fragment cache — store partial page renders or API response fragments
- Reduces database query load significantly for read-heavy endpoints

5. Cache Hit vs Cache Miss

5.1 Cache Hit

A cache hit occurs when the requested data is found in the cache. The system returns the data immediately without touching the database.

```
Request → Cache → Data found → Return immediately (fast)
```

Cache hits are the goal. A high cache hit rate — typically measured as a percentage — means most requests are being served from memory. Systems with 90%+ hit rates experience dramatically lower database load and latency.

5.2 Cache Miss

A cache miss occurs when the requested data is not in the cache — either because it was never stored there, or because it expired and was evicted. The system must fall back to the database.

```
Request → Cache → Not found → Database → Store in cache → Return
```

Cache misses are more expensive than direct database queries because they involve two operations: the failed cache lookup, then the database query, then writing back to the cache. However, all subsequent requests for that data will be cache hits.

Optimising for cache hit rate is one of the most impactful performance improvements in a web system. Techniques include warming the cache on startup, using appropriate TTLs, and understanding access patterns.

6. Caching Strategies

Choosing the right caching strategy depends on your read/write patterns and your tolerance for stale data. Each strategy makes a different trade-off between consistency, performance, and complexity.

6.1 Cache-Aside (Lazy Loading) — Most Common

The application manages the cache explicitly. On a read, it checks the cache first. On a miss, it fetches from the database and populates the cache. Writes go directly to the database; the cache entry is either invalidated or updated separately.

```
READ: Check cache → miss → fetch DB → write to cache → return  
WRITE: Write to DB → invalidate or update cache entry
```

- Simple to implement and reason about
- Only caches data that is actually requested — no wasted memory
- Risk of stale data between write and cache update
- Initial requests after a cold start or expiry are slower (cold cache problem)

6.2 Write-Through

Every write operation updates both the cache and the database synchronously. Reads always find the data in cache because it was written there at the same time as the database.

```
WRITE: Write to cache + write to DB (simultaneously)
READ:  Cache always has latest data → return immediately
```

- Strong consistency — cache is always up to date
- Higher write latency — must wait for both cache and DB to confirm
- Cache may hold data that is never read — wastes memory
- Good for systems where reads immediately follow writes

6.3 Write-Back (Write-Behind)

Writes go to the cache immediately and are acknowledged to the client. The cache asynchronously flushes changes to the database in the background, typically in batches.

```
WRITE: Write to cache → acknowledge to client → async flush to DB
```

- Extremely fast write performance — no waiting for DB confirmation
- Risk of data loss if cache fails before flushing to DB
- Complex to implement correctly — requires durable write queues
- Good for high-throughput write systems where occasional loss is acceptable (e.g. analytics counters)

6.4 Read-Through

The cache sits transparently between the application and the database. On a cache miss, the cache itself is responsible for fetching data from the database and populating itself. The application always reads from the cache only.

```
READ: Application → Cache → (on miss) Cache fetches DB → returns data
```

- Simplifies application code — the app only talks to the cache
- Cache and database synchronisation is handled by the cache layer
- Requires a cache implementation that supports this pattern (e.g. some Redis libraries)

7. TTL — Time To Live

TTL is a duration assigned to a cache entry that determines how long it remains valid before being automatically evicted. When a cached item expires, the next request for that data will result in a cache miss, triggering a fresh fetch from the database.

```
Cache user profile for 5 minutes → TTL = 300 seconds  
Cache product listing for 1 hour → TTL = 3600 seconds  
Cache static config for 24 hours → TTL = 86400 seconds
```

Setting the right TTL requires balancing freshness against performance. A TTL that is too short means the cache is frequently invalidated and provides little benefit. A TTL that is too long risks serving stale data to users after the underlying data has changed.

- Short TTL — fresher data, more DB load, less cache benefit
- Long TTL — more cache benefit, risk of serving outdated data
- No TTL (no expiry) — maximum cache benefit, requires explicit invalidation on every change

8. Cache Invalidation

Famous quote: There are only two hard things in computer science — cache invalidation and naming things. — Phil Karlton

Cache invalidation is the process of removing or updating cached data when the underlying source data changes. It is widely considered one of the hardest problems in distributed systems because invalidating too aggressively defeats the purpose of caching, while invalidating too conservatively leads to stale data.

8.1 Invalidation Strategies

TTL-based expiry:

- Cache entries automatically expire after a set duration
- Simple and requires no coordination between systems
- Accepts that data may be stale for up to the TTL period

Manual invalidation:

- When data changes, the application explicitly deletes or updates the cache entry
- Immediate consistency — the stale entry is removed at write time
- Requires careful implementation — easy to forget to invalidate in all code paths

Event-based invalidation:

- A change event is published (e.g. via a message queue or database change stream)
- A cache invalidation service listens to events and purges the relevant cache entries

- Decoupled and scalable — no tight coupling between write logic and cache management
- More complex infrastructure — requires reliable event delivery

9. Common Caching Problems

9.1 Stale Data

Stale data occurs when the cache holds an outdated copy of data that has since changed in the database. Users receive old information until the cache entry expires or is explicitly invalidated. This is the fundamental tension in caching: performance vs freshness.

- Mitigation: Use appropriate TTLs and implement cache invalidation on writes
- Accept-tolerance: For non-critical data (e.g. view counts), some staleness is acceptable

9.2 Cache Stampede (Thundering Herd)

A cache stampede occurs when a popular cache entry expires and many concurrent requests all find the cache empty simultaneously. They all fall through to the database at the same time, causing a sudden spike in database load — potentially overwhelming it at the worst moment.

- Solution: Mutex locking — only one request fetches from DB; others wait for cache to repopulate
- Solution: Staggered TTLs — add random jitter to expiry times to prevent mass simultaneous expiration
- Solution: Cache warming — proactively populate the cache before expiry using a background job

9.3 Cache Penetration

Cache penetration happens when requests are made for data that does not exist in either the cache or the database. Because the cache has nothing to store, every request bypasses the cache and hits the database — making the cache useless for that data and potentially allowing malicious actors to use non-existent keys as a denial-of-service vector.

- Solution: Cache null values — store a null or empty marker in the cache for non-existent keys
- Solution: Bloom filters — a probabilistic data structure that can quickly determine if a key definitely does not exist

10. When to Use Caching

10.1 Ideal Use Cases

- Read-heavy systems where the same data is requested frequently
- Expensive database queries or aggregations that change infrequently
- API responses for popular, shared data (e.g. trending content, product listings)
- Computed results from CPU-intensive operations
- Static and semi-static content: configuration, feature flags, reference data

10.2 When to Avoid Caching

- Highly dynamic data that changes on every request — the cache provides no benefit
- Sensitive financial or transactional data where any staleness is unacceptable
- Write-heavy systems where data changes more often than it is read
- Data that must be strongly consistent across all users at all times

11. Real-World Example: User Profile API

Consider an API endpoint that returns a user's profile. Profiles are read on every page load but updated rarely. This is an ideal caching candidate.

```
Step 1: Request arrives for user profile (user_id: 12345)
Step 2: Check Redis for key 'user:12345'
Step 3: Cache HIT → return profile immediately (< 1ms)
        Cache MISS → query PostgreSQL for user data
Step 4: Store result in Redis with TTL of 300 seconds
Step 5: Return profile to client
```

On profile update:

```
Step 1: Write new data to PostgreSQL
Step 2: Delete 'user:12345' from Redis (invalidate)
Step 3: Next request repopulates cache with fresh data
```

With a high cache hit rate, the database receives only a fraction of the actual request volume. If 95% of profile requests are cache hits, the database handles just 5% of traffic — a 20x reduction in database load from a single caching layer.

12. Final Takeaways

Mental Model: A cache is a shortcut that avoids expensive operations. The more frequently data is read relative to how often it changes, the more valuable caching it becomes.

- Caching reduces latency and database load — it is one of the highest-impact optimisations available
- Redis is the most widely used caching tool in production systems
- Cache-aside is the most common and practical strategy for most applications
- Cache invalidation is hard — design your invalidation strategy before you design your caching strategy
- Monitor cache hit rate — a low hit rate means your caching strategy needs rethinking
- Caching is most valuable for read-heavy, infrequently-changing data

Interview Questions & Answers

Q1: What is caching and why is it used?

- Caching stores copies of frequently accessed data in fast memory (RAM) to avoid repeating expensive operations
- Primary benefits: reduced response latency, lower database load, and better scalability
- A well-designed cache can serve 90%+ of read requests without touching the database

Q2: What is the difference between a cache hit and a cache miss?

- Cache hit: the requested data is found in the cache and returned immediately — fast and cheap
- Cache miss: the data is not in the cache, so the system falls back to the database, fetches the data, stores it in cache, then returns it
- Cache hit rate is the key metric — a healthy caching layer typically achieves 85–95% hit rate

Q3: What is cache invalidation and why is it hard?

- Cache invalidation is the process of removing or updating cached data when the source data changes
- It is hard because you must decide when the cache is stale, how to propagate invalidation across distributed caches, and how to avoid invalidating too aggressively (defeats caching) or too conservatively (serves stale data)

- Phil Karlton famously said there are only two hard things in computer science — cache invalidation and naming things

Q4: Explain the Cache-Aside pattern.

- On a read: check cache first; if miss, fetch from DB, store in cache, return result
- On a write: update DB directly; either invalidate the cache entry or update it
- The application controls cache population explicitly
- Most widely used pattern — simple, predictable, and only caches data that is actually requested

Q5: What is the Write-Through caching strategy?

- Every write updates both the cache and the database simultaneously
- Reads always find the latest data in cache because it is written there at the same time as the DB
- Strong consistency — cache is never stale after a write
- Downside: higher write latency (must wait for both cache and DB to confirm) and cache may hold data that is never read

Q6: What is a cache stampede and how do you prevent it?

- A cache stampede occurs when a popular cache entry expires and many concurrent requests simultaneously find the cache empty
- All requests fall through to the database at once, causing a sudden traffic spike that can overwhelm it
- Solutions: mutex locking (only one request fetches from DB, others wait), staggered TTLs (random jitter on expiry), or background cache refresh before expiry

Q7: What is cache penetration?

- Cache penetration happens when requests are made for data that does not exist in the cache or the database
- The cache has nothing to store, so every request bypasses it and hits the database
- Malicious actors can exploit this by querying non-existent keys to overload the database
- Solutions: cache null responses (store an empty marker for missing keys) or use a Bloom filter to quickly reject non-existent keys before the cache lookup

Q8: What is the difference between Redis and Memcached?

- Redis supports rich data types (strings, lists, sets, sorted sets, hashes) and has built-in persistence; Memcached is pure key-value only
- Redis supports atomic operations, pub/sub messaging, and TTL per key; Memcached has none of these
- Memcached is multi-threaded and can use multiple CPU cores more efficiently
- Redis is almost always the right choice for new projects; Memcached survives in legacy systems where its simplicity is valued

Q9: What data should not be cached?

- Highly dynamic data that changes on every request — the cache provides no benefit
- Sensitive financial or transactional data where even a moment of staleness is unacceptable
- User-specific personalised data in a shared cache — risk of serving one user's data to another
- Write-heavy data that changes more often than it is read — the overhead of maintaining the cache exceeds the benefit

Q10: How does caching relate to database scaling?

- Caching is often the first and most impactful step before adding database read replicas or sharding
- If 90% of read traffic can be served from Redis, a single database instance can handle 10x more users
- Caching defers the need for complex database scaling strategies — always implement caching before reaching for sharding
- Read replicas and caching are complementary: replicas scale database reads, caching eliminates database reads entirely for hot data

CHAPTER 7

Content Delivery Networks (CDN)

Architecture, usage patterns, and best practices for global delivery

1. What Is a CDN?

A Content Delivery Network is a globally distributed network of servers — called edge nodes or points of presence (PoPs) — that cache and serve content from locations geographically close to the end user. Instead of every request travelling to a single origin server, users receive responses from the nearest edge node, dramatically reducing the physical distance data must travel.

Without a CDN, a user in London requesting assets from a server in Mumbai experiences a round-trip of roughly 150–200ms just in network latency, before any processing begins. With a CDN edge node in Frankfurt, that same user may experience under 10ms of network latency to retrieve the same asset.

Core Insight: The speed of light is a physical constraint. CDNs solve latency not by making servers faster, but by placing them closer to users.

2. How a CDN Works

When a user requests a resource, the CDN routes the request to the nearest edge node based on the user's geographic location and current network conditions. The edge node checks whether it has a cached copy of the requested resource.

On a cache hit:

```
User → CDN Edge Node → Cached asset returned immediately
```

On a cache miss:

```
User → CDN Edge Node → Origin Server → asset fetched and cached → returned to user
```

After the first cache miss, the edge node stores the asset locally. All subsequent requests from users near that edge node are served directly from cache, with zero load placed on the origin server. This pattern means CDNs become more effective over time as their caches warm up.

CDNs also use intelligent DNS routing — when your domain resolves, the DNS system returns the IP address of the closest healthy edge node rather than the origin server. This is completely transparent to the user and requires no changes to the application code.

3. Two Ways of Using a CDN

There is an important distinction between using a CDN as a consumer of third-party resources, and deploying a CDN as part of your own infrastructure. Many systems do the former without realising they are missing out on the full benefits of the latter.

3.1 CDN as a Consumer (Third-Party Libraries)

Many applications load popular external libraries — such as JavaScript frameworks, UI component libraries, or icon sets — directly from public CDN URLs rather than bundling them. This is the most basic form of CDN usage.

Examples of third-party CDN usage:

- Cloudflare (cdnjs.cloudflare.com) — flag icons, moment.js, jQuery, Bootstrap
- jsDelivr (cdn.jsdelivr.net) — OpenLayers, mapping libraries, npm packages
- unpkg (unpkg.com) — any npm package served directly from the registry

This approach has real benefits: these files are already cached globally by the CDN provider, so users often have them pre-cached from visiting other sites. Your server also does not need to transfer these large library files, reducing your bandwidth costs and bundle size.

However, this only helps external dependencies. Your own application code — JavaScript bundles, CSS, images, and fonts — still travels from your origin server to every user on every request. For global users, this means high latency regardless of how many third-party CDNs you reference.

3.2 CDN as Infrastructure (Serving Your Own Assets)

A dedicated CDN setup means placing your own application's static assets behind a CDN layer. Your JS bundles, CSS files, images, and fonts are uploaded to or pulled by the CDN, then served from edge nodes worldwide. This is where the largest performance and scalability gains come from.

Current flow without dedicated CDN:

```
User (anywhere) → Application Server (single location) → Static Assets
```

Improved flow with dedicated CDN:

```
User → CDN Edge Node → Static Assets (served from nearby edge)
      → Load Balancer → App Server (only for API calls)
```

In this model, the application server is completely relieved of serving static content. Its CPU, memory, and bandwidth are freed entirely for processing API requests and business logic — the things only it can do.

4. Impact of Not Using a Dedicated CDN

Running without a CDN for your own assets is a common architectural gap that becomes increasingly costly as your user base grows and becomes more geographically distributed.

4.1 Increased Latency for Global Users

Every static asset request must travel to your origin server and back. For users in different continents, this adds hundreds of milliseconds to every page load. Modern users expect pages to load in under 2–3 seconds — high latency from geographic distance can make that impossible to achieve regardless of how optimised your application code is.

4.2 Higher Backend Load

Static files — which never change between requests — are served repeatedly from the same server, consuming CPU time, network bandwidth, and connection slots. This is wasted capacity that could be serving real API traffic. Under high traffic, serving static assets becomes a significant drag on server resources.

4.3 Poor Scalability Under Traffic Spikes

When traffic spikes — whether from a marketing campaign, a viral moment, or a product launch — all requests hit your backend directly. There is no offloading layer to absorb the burst. This forces either expensive over-provisioning of backend capacity, or accepting degraded performance during peak periods.

4.4 Missed Caching Benefits

Without edge caching, every user receives assets fresh from the server on each session, even if the assets have not changed in months. With a CDN, assets are cached at the edge and served instantly, with cache-control headers ensuring users always receive current versions when updates are deployed.

5. What Should Be Served via CDN

Ideal candidates for CDN delivery:

- JavaScript bundles — your application's compiled JS files
- CSS stylesheets — compiled and minified style files
- Images — product photos, hero images, icons, illustrations

- Fonts — web fonts (.woff2, .ttf) which are large and rarely change
- Videos and audio — media files benefit enormously from CDN edge delivery
- Downloadable files — PDFs, exports, static documents

What should NOT go through CDN (stay on the app server):

- API responses containing user-specific or dynamic data
- Authentication endpoints — login, token refresh, session management
- Anything that changes per-request or per-user

6. Recommended CDN Providers

Provider	Strengths	Best For
Amazon CloudFront	Deep AWS integration, Lambda@Edge	AWS-native architectures
Cloudflare	Security, DDoS protection, Workers	General-purpose, security-focused
Akamai	Largest edge network globally	Enterprise, media, high availability
Fastly	Real-time purging, edge compute	E-commerce, news, fast invalidation

7. Best Practices for CDN Integration

7.1 Cache-Control Headers

Cache-Control headers tell both the browser and the CDN how long to cache a resource. Setting these correctly is essential to getting CDN benefits while ensuring users always see up-to-date content.

```
Cache-Control: max-age=31536000, immutable
```

The max-age value is in seconds. 31,536,000 is one full year — appropriate for versioned assets that will never change at that URL. The immutable directive tells browsers not to revalidate the asset even when the user refreshes the page.

7.2 Asset Versioning

If you cache assets aggressively with long TTLs, you need a mechanism to force cache busting when you deploy new code. Asset versioning appends a content hash or version string to the filename. Because the URL changes, the CDN and browser treat it as a new asset and fetch the latest version.

```
app.a3f9b2.js    (content hash in filename)
styles.v2.css    (version number in filename)
```

Modern build tools like Webpack, Vite, and esbuild generate content-hashed filenames automatically. This lets you set aggressive caching (one year) while guaranteeing users always receive new assets immediately after a deployment.

7.3 Separate Static and Dynamic Traffic

Route traffic explicitly so the CDN handles static files and the load balancer handles API traffic. This is typically done by hosting static assets on a dedicated subdomain (e.g. `static.yourdomain.com` or `assets.yourdomain.com`) that points to the CDN, while `api.yourdomain.com` routes to your backend servers.

7.4 Compression and Optimisation

Enable Gzip or Brotli compression on all text-based assets (JS, CSS, HTML, JSON). Most CDN providers support this automatically at the edge. Brotli provides 15–25% better compression ratios than Gzip and is supported by all modern browsers. For images, use modern formats: WebP instead of JPEG/PNG, and AVIF for the highest compression ratios.

7.5 Cache Invalidation Strategy

CDN caches can be purged explicitly when content changes. Most providers offer an API for cache invalidation — you can trigger a purge as part of your deployment pipeline to ensure new content is served immediately. Combined with content-hashed filenames, you have full control over when and what gets updated at the edge.

8. Before vs After: CDN Integration

Aspect	Without CDN	With Dedicated CDN
Static delivery	Origin app server	CDN edge nodes worldwide

Aspect	Without CDN	With Dedicated CDN
Latency (global users)	High — cross-continent round-trips	Low — served from nearby edge
Backend server load	High — serves all traffic	Reduced — APIs only
Traffic spike handling	Backend absorbs everything	CDN absorbs static spike
Scalability	Limited by server capacity	Effectively unlimited for static
Deployment updates	Immediate (no cache)	Requires versioning + purge strategy

9. Final Takeaways

Using third-party CDNs improves dependency delivery, but without a dedicated CDN layer for your own assets, the application misses out on global performance, reduced backend load, and true scalability benefits.

- A CDN solves latency by placing content physically closer to users — no amount of code optimisation can beat the speed of light
- Using public CDNs for third-party libraries is useful but only covers external dependencies, not your own application assets
- A dedicated CDN dramatically reduces backend load, freeing your servers to handle business logic and API requests
- Asset versioning combined with long cache TTLs is the gold standard — aggressive caching with zero stale content risk
- For any application with a global user base, a dedicated CDN is not optional — it is foundational infrastructure

Interview Questions & Answers

Q1: What is a CDN and how does it work?

- A Content Delivery Network is a globally distributed network of edge servers that cache and serve content from locations physically close to users
- On a cache hit, the edge node returns the content without contacting the origin server
- On a cache miss, the edge fetches from the origin, caches the response, and serves future requests locally

- This reduces latency by minimising the distance data must travel

Q2: What is the difference between a CDN and a load balancer?

- A CDN reduces latency by serving cached static content from geographically distributed edge nodes
- A load balancer distributes dynamic traffic across multiple backend servers in one location
- They solve different problems and work best in combination: CDN for static assets, load balancer for API calls
- CDNs operate at the global level; load balancers operate within a data centre or region

Q3: What types of content should be served via CDN?

- Static assets that do not change per-user: JavaScript bundles, CSS, images, fonts, icons
- Video and audio files — large media benefits enormously from edge delivery and bandwidth offloading
- Downloadable files: PDFs, exports, software packages
- Some CDNs can cache API responses with appropriate Cache-Control headers for semi-static data

Q4: How do Cache-Control headers work with CDNs?

- Cache-Control headers tell the CDN and browser how long to cache a resource
- `max-age=31536000` means cache for one year; `s-maxage` applies specifically to CDN caches
- `no-cache` means always revalidate with the origin before serving from cache
- `immutable` tells browsers the resource will never change at this URL — skip revalidation even on refresh

Q5: What is asset versioning and why is it important?

- Asset versioning appends a content hash or version string to the filename (e.g. `app.a3f9b2.js`)
- When the file changes, the filename changes, so the CDN and browser treat it as a brand-new asset
- This allows aggressive long-term caching (one year) with zero stale content risk
- Modern bundlers (Vite, Webpack, esbuild) generate content-hashed filenames automatically

Q6: What is cache invalidation in a CDN?

- Removing cached content from CDN edge nodes before its TTL expires
- Typically done via an API call to the CDN provider — you specify which URLs or cache tags to purge
- Can be triggered as part of a deployment pipeline to immediately serve new content after a release
- Fastly and Cloudflare support near-instant global purges; other providers may take minutes

Q7: What is edge computing in relation to CDNs?

- Edge computing extends CDN capabilities from serving static files to running code at the edge node
- Edge functions (Cloudflare Workers, AWS Lambda@Edge, Vercel Edge Functions) execute JavaScript at the CDN node closest to the user
- Use cases: personalisation, A/B testing, auth token validation, and API response transformation without round-tripping to the origin
- Reduces latency for logic that previously required a full round-trip to the application server

Q8: What is the origin server in a CDN setup?

- The origin server is your actual application server — the authoritative source of content
- The CDN pulls content from the origin on a cache miss and stores it at the edge
- The origin only needs to handle cache misses and API requests — its load is dramatically reduced
- Origin health and response time matter — if the origin is slow, the first request to any edge location will be slow

Q9: How does a CDN help during a traffic spike?

- Cached static assets are served entirely from the edge — origin servers receive no additional load
- The CDN's global edge network has massive capacity that dwarfs any single origin server
- Even if the origin is temporarily overwhelmed or down, edge nodes continue serving cached content
- This makes CDNs a critical layer of protection against traffic spikes, DDoS attacks, and viral events

Q10: What are the trade-offs of using a CDN?

- Benefits: lower latency globally, reduced origin server load, improved availability, DDoS protection

- Costs: added latency for cache misses on first request; CDN fees (bandwidth-based pricing)
- Complexity: requires cache invalidation strategy, asset versioning, and monitoring of edge cache hit rates
- Vendor lock-in: CDN-specific features (Workers, Rules) can be difficult to migrate away from

CHAPTER 8

Horizontal vs Vertical Scaling

How modern systems grow to handle millions of users

1. What Is Scaling?

Scaling is the process of increasing a system's capacity to handle more users, more requests, or more data — without degrading performance or reliability. As a product grows, the infrastructure it was built on at launch will eventually reach its limits. Understanding how to scale correctly, and when to apply each approach, is one of the most important skills in system design.

There are two fundamentally different approaches to scaling: vertical scaling, which makes existing machines more powerful, and horizontal scaling, which adds more machines to the pool. Each has a distinct set of trade-offs, and most real-world systems end up using both at different stages of their growth.

2. Vertical Scaling (Scale Up)

Vertical scaling means increasing the capacity of a single server by upgrading its hardware — more CPU cores, more RAM, a faster disk, or a higher-bandwidth network interface. The server remains one machine; it simply becomes a more powerful one.

```
Before: 4 vCPU, 8 GB RAM, 100 GB SSD
After:  32 vCPU, 128 GB RAM, 2 TB NVMe SSD
```

This approach feels intuitive because it maps directly to our everyday experience — when a computer feels slow, we upgrade it. In the early stages of a product, vertical scaling is often the right call. There is no architectural change required, no distributed system to reason about, and no code changes needed. You simply provision a larger instance and redeploy.

2.1 When Vertical Scaling Makes Sense

- Early-stage products where simplicity and speed of iteration matter most
- Databases that do not yet support easy horizontal distribution
- Batch processing workloads that benefit from a single large machine with shared memory
- Systems where distributed complexity would outweigh the scaling benefit

2.2 Advantages

- Simple to implement — no code changes, no architecture redesign
- No distributed system complexity — one machine means no network partitions, no consistency challenges
- Easier to reason about — all data and state live in one place
- Works well for relational databases that are difficult to distribute

2.3 Disadvantages

- Hard hardware ceiling — even the largest cloud instances have a maximum size
- Increasingly expensive — the cost of doubling capacity more than doubles at the top end
- Single point of failure — if the server crashes, the entire system goes down
- Downtime during upgrades — resizing a server typically requires a restart
- Does not improve fault tolerance — one failure still takes down everything

Key Limit: In 2024, the largest AWS EC2 instance offers 192 vCPUs and 1.5 TB of RAM. That sounds enormous, but a system serving tens of millions of users will exceed it. Vertical scaling always has a ceiling.

3. Horizontal Scaling (Scale Out)

Horizontal scaling means adding more servers to your system rather than making one server more powerful. Instead of one machine handling all traffic, a fleet of identical machines shares the load behind a load balancer. When traffic grows, you add more servers. When traffic drops, you remove them.

Traffic growth:

```
Week 1:  1 server → handles  10,000 req/sec
Week 4:  4 servers → handles  40,000 req/sec
Month 6: 20 servers → handles 200,000 req/sec
```

This is how every major internet platform — Google, Amazon, Netflix, Facebook — serves billions of requests per day. No single machine could do it alone, regardless of how powerful it is. The architecture is designed from the start to distribute work across many commodity servers.

3.1 How It Works in Practice

Horizontal scaling requires a load balancer sitting in front of the server pool. The load balancer distributes incoming requests across all available servers, monitors their health, and routes traffic away from any server that fails. To the outside world, the system appears as a single endpoint, even though dozens or hundreds of servers may be running behind it.

```
User → Load Balancer → Server A
                        → Server B
                        → Server C (add more as needed)
```

3.2 Advantages

- Virtually unlimited scaling — add servers as long as demand exists
- High availability and fault tolerance — one server failing does not affect users
- Cost-efficient growth — add small increments of capacity as needed, not one expensive upgrade
- Works naturally with cloud auto-scaling — add servers during peak hours, remove them overnight
- Enables zero-downtime deployments — roll updates across servers one at a time

3.3 Disadvantages

- Requires a load balancer — adds infrastructure and configuration complexity
- Applications must be stateless — session data cannot live on individual servers
- Distributed system challenges — network latency, partial failures, and consistency issues
- More complex to debug and monitor — requests may be handled by any server in the pool
- Data synchronisation overhead — all servers must access the same shared data layer

4. Key Differences

Dimension	Vertical Scaling	Horizontal Scaling
Approach	Upgrade a single server	Add more servers to the pool
Scale limit	Hard hardware ceiling	Virtually unlimited
Cost	Expensive at the top end	Incremental, pay-as-you-grow
Fault tolerance	Low — single point of failure	High — failures are isolated
Complexity	Simple — no architecture change	Complex — distributed system
Downtime	Often required during resize	Zero-downtime with rolling deploys
Best stage	Early product, low traffic	Growth stage, high traffic

5. Why Modern Systems Prefer Horizontal Scaling

5.1 Cloud-Native Infrastructure

Modern cloud platforms — AWS, Google Cloud, Azure — are designed around the assumption that you will run many small, interchangeable servers rather than one large one. Auto-scaling groups can add new servers in under a minute in response to traffic spikes, and remove them just as quickly when traffic drops. This elasticity is impossible with vertical scaling, where resizing a server takes several minutes and requires downtime.

5.2 Fault Tolerance by Default

In a horizontally scaled system, individual server failures are routine, expected events — not emergencies. The load balancer detects the failure via health checks and stops routing traffic to that server within seconds. Users are seamlessly redirected to healthy servers and typically do not experience any disruption. Netflix famously runs Chaos Monkey — a tool that deliberately kills random servers in production — to prove that their horizontal architecture is resilient enough to handle failures without human intervention.

5.3 Enables Microservices Architecture

Horizontal scaling is the foundation that makes microservices practical. Each service runs as multiple instances behind its own load balancer. Services that receive more traffic get more instances; services under light load run with fewer. This per-service

elasticity is only possible because each service scales out independently — something that cannot be done in a vertically scaled monolith.

5.4 Cost Efficiency at Scale

At large scale, many smaller commodity servers are significantly cheaper than one equivalent large server. Cloud providers charge a premium for their largest instance types. A fleet of smaller instances also provides better fault isolation — losing one small server out of twenty has a much smaller blast radius than losing one giant server.

6. Challenges of Horizontal Scaling (and How to Solve Them)

6.1 Session and State Management

The most common challenge when moving from one server to many is session state. If user session data is stored in a single server's memory, the next request from that user — which may be handled by a different server — will find no session and log the user out. This is why stateless application design is so critical to horizontal scaling.

The solution is to externalise all state. Session data, authentication tokens, shopping carts, and any other per-user state are stored in a shared data layer — typically Redis — that every server in the pool can access. Any server can then handle any request from any user, because all the state it needs is in the shared store, not in local memory.

```
BAD:  Server A stores session in local RAM → Server B cannot find it  
GOOD: All servers read/write sessions from Redis → any server handles any user
```

6.2 Data Consistency Across Servers

When multiple servers write to the same database simultaneously, consistency becomes a concern. Two servers might read the same record, both decide to update it, and write conflicting values. This class of problem — known as a race condition — does not exist on a single server but emerges in distributed systems.

- Use database-level transactions and row-level locking for critical writes
- Use optimistic concurrency control — include a version field and reject stale updates
- Design operations to be idempotent where possible — the same operation applied twice produces the same result

6.3 Database Bottleneck

Horizontal scaling of application servers is relatively straightforward, but the database can quickly become the new bottleneck. You have scaled from 1 to 20 app servers, all querying the same single database. The database now receives 20x the query volume.

The standard solutions are read replicas and sharding. Read replicas are copies of the database that handle read queries, while the primary handles all writes. Sharding partitions data across multiple database instances so that each instance handles a fraction of the total data and query volume. These are both forms of horizontal scaling applied to the database layer itself.

- Read replicas — distribute read load across multiple database copies
- Connection pooling — reuse database connections rather than opening a new one per request
- Caching (Redis) — reduce database hits by serving repeated reads from memory
- Sharding — partition data across multiple database instances for write scalability

6.4 Debugging and Observability

When a bug or performance issue occurs in a fleet of 20 servers, identifying which server processed the problematic request is non-trivial. Effective horizontal scaling requires centralised logging (all servers ship logs to a single aggregation system like Elasticsearch), distributed tracing (every request carries a trace ID that follows it across services), and centralised metrics dashboards (Prometheus, Grafana) that aggregate data from all servers.

7. The Hybrid Approach — Using Both

Real-world systems almost universally use a combination of both strategies. The typical progression as a product grows looks like this:

11. Launch on a single, appropriately-sized server (vertical scaling for simplicity)
12. As traffic grows, upgrade the server to a larger instance (more vertical scaling)
13. Refactor the application to be stateless — externalise sessions and state to Redis
14. Introduce a load balancer and add a second application server (beginning of horizontal scaling)
15. Scale the server pool horizontally as traffic demands, configure auto-scaling rules
16. Scale the database layer separately — add read replicas, then consider sharding if needed

Notice that even after horizontal scaling begins, the individual servers in the pool may still be vertically scaled to a reasonable size. A fleet of mid-range servers is often more cost-effective than a large fleet of minimal servers, because there is a fixed overhead per server (OS, agent processes, network connections). The right balance depends on the specific workload.

8. Auto-Scaling in Cloud Environments

One of the most powerful benefits of horizontal scaling in cloud environments is auto-scaling — the ability to automatically add or remove servers based on real-time metrics. An auto-scaling group monitors CPU utilisation, request queue length, or custom application metrics, and scales the server count up or down to match actual demand.

```
Rule: if average CPU > 70% for 5 minutes → add 2 servers
Rule: if average CPU < 30% for 10 minutes → remove 1 server
```

- Handles traffic spikes automatically without human intervention
- Reduces cost during low-traffic periods — servers are removed when not needed
- Provides a safety net against viral traffic events or DDoS attempts
- Requires stateless application design — new servers must be able to handle any request immediately

9. Final Takeaways

Mental Model: Vertical scaling is a bigger machine. Horizontal scaling is more machines. One has a ceiling; the other does not.

- Vertical scaling is simple and appropriate early — but it has a hard limit and creates a single point of failure
- Horizontal scaling is the foundation of every large-scale internet system — it trades simplicity for virtually unlimited growth
- Stateless application design is the prerequisite for horizontal scaling — externalise all state to shared stores
- Load balancers are the enabling technology for horizontal scaling — without them, distributing traffic across a server pool is impossible
- The database is usually the next bottleneck after app servers scale — plan for read replicas and caching early
- Most production systems use both — vertical for databases and early-stage simplicity, horizontal for app servers at scale

Interview Questions & Answers

Q1: What is the difference between horizontal and vertical scaling?

- Vertical scaling upgrades a single server with more CPU, RAM, or disk — simple but has a hard hardware ceiling
- Horizontal scaling adds more servers to share the load — complex but virtually unlimited
- Vertical scaling creates a single point of failure; horizontal scaling is inherently fault-tolerant
- Modern cloud systems prefer horizontal scaling for its flexibility and resilience

Q2: What does stateless application design mean?

- A stateless application stores no user-specific data in server memory between requests
- Every request contains all the information needed to process it — the server does not rely on previous interactions
- Session data, user preferences, and tokens are stored in an external shared store (Redis, database)
- Stateless design is the prerequisite for horizontal scaling — any server in the pool can handle any request

Q3: What is auto-scaling?

- Auto-scaling automatically adjusts the number of server instances based on real-time demand
- Scale-out rules add instances when metrics like CPU utilisation or request queue length exceed thresholds
- Scale-in rules remove instances during low-traffic periods to reduce cost
- Requires stateless applications — new instances must be immediately ready to handle any request

Q4: Why is horizontal scaling preferred in cloud-native systems?

- Cloud platforms are designed around it — adding a new instance takes under a minute
- Fault tolerance is built in — individual instance failures have minimal blast radius
- Enables zero-downtime deployments through rolling updates
- Cost efficiency — scale to meet demand and scale back down, paying only for what is used

Q5: What is the database bottleneck problem in horizontal scaling?

- Horizontally scaled app servers can generate far more database queries than a single database can handle
- After scaling app servers, the database often becomes the next bottleneck
- Solutions in order of complexity: caching (Redis), read replicas, connection pooling, sharding
- Plan for database scaling before it becomes urgent — retrofitting sharding onto a live system is painful

Q6: What is a single point of failure?

- A component whose failure causes the entire system to go down
- A single server without redundancy is the most common SPOF
- Other SPOFs: a single load balancer, a single database with no replicas, a single network switch
- Eliminating SPOFs through redundancy and replication is fundamental to high-availability system design

Q7: What is elastic scaling?

- Elastic scaling automatically adds and removes capacity in real time to match fluctuating demand
- During a traffic spike, new servers are provisioned and added to the load balancer pool automatically
- When traffic drops, idle servers are terminated to reduce cost
- AWS Auto Scaling Groups, Google Managed Instance Groups, and Kubernetes Horizontal Pod Autoscaler all implement elastic scaling

Q8: How does Kubernetes enable horizontal scaling?

- Kubernetes runs application instances as Pods and manages their lifecycle on a cluster of nodes
- The Horizontal Pod Autoscaler (HPA) monitors CPU and memory metrics and adjusts Pod replica count automatically
- The Cluster Autoscaler adds or removes cluster nodes based on pending Pod demand
- Kubernetes also handles rolling deployments, health checks, and self-healing — restarting failed Pods automatically

Q9: What are the main challenges of horizontal scaling?

- Stateful applications must be refactored to externalise state before they can be horizontally scaled

- Distributed debugging — a request may be handled by any server, making error tracing harder
- Consistency — shared data accessed by multiple servers simultaneously requires careful coordination
- Infrastructure complexity — load balancers, service discovery, and orchestration all add operational overhead

Q10: What is the difference between scale-up and scale-out in databases?

- Scale-up: move to a larger database instance with more CPU and RAM — simple but expensive and limited
- Read replicas: scale-out for reads by adding replica databases that serve read queries
- Sharding: scale-out for both reads and writes by partitioning data across multiple database instances
- Most systems exhaust scale-up first, then add read replicas, and only introduce sharding when replicas are insufficient

CHAPTER 9

Asynchronous Processing & Message Queues

BullMQ, SQS, Kafka — when to use what and why

1. The Core Problem with Synchronous Processing

In a synchronous system, every operation the server performs happens in sequence before the response is returned to the user. For fast operations — reading from a cache, writing a single row to a database — this is fine. But many real-world tasks are inherently slow, or simply do not need to complete before the user gets a response.

Operations that should not block the user:

- Sending a welcome email after signup — the user does not need to wait for the email to be delivered
- Processing a payment — confirmation can be sent once the request is queued, not after the bank responds
- Generating a large report — can take minutes; the user should receive a notification when it is ready
- Resizing and encoding a video — processing may take longer than any reasonable HTTP timeout

- Pushing notifications to thousands of devices — not all need to succeed before responding

Synchronous approach — everything blocks:

```
User → Server → send email → resize image → update DB → Response
```

In this model, the user waits for every step to complete. If the email service is slow, the user waits. If the image processor is busy, the user waits. The server also holds open a connection, consuming resources, for the entire duration. Under high load this becomes catastrophic — threads pile up waiting for slow tasks, memory is exhausted, and the server stops responding to new requests entirely.

Key Insight: If a task does not need to be completed before the user gets a response, it should not be done synchronously. Offload it to a queue.

2. The Solution: Asynchronous Processing

Asynchronous processing decouples the act of receiving a request from the act of processing it. The server immediately acknowledges the request, pushes the work onto a queue, and returns a response to the user. A separate worker process picks up the task from the queue and processes it independently — without the user waiting.

```
User → Server → push job to Queue → respond immediately (fast)
                        ↓
                        Worker → process job → done
```

The user gets a fast response. The heavy work happens in the background without affecting server availability. If the worker is slow or temporarily unavailable, jobs simply accumulate in the queue and are processed as capacity allows — the server is never blocked.

Real example — order placement:

17. User submits order
18. Server saves order to database
19. Server pushes 'send_confirmation_email' job to queue
20. Server responds to user: 'Order confirmed' (instantly)
21. Email worker picks up job and sends the email (background)

From the user's perspective, the order was confirmed immediately. The email arrives moments later. In a synchronous system, the user would have waited for all five steps before seeing any response.

3. Key Components of a Queue System

3.1 Producer

The producer is the part of your application that creates and enqueues jobs. It knows that a task needs to be done but delegates the actual work to someone else. In the order example above, the API server is the producer — it pushes the email job to the queue and moves on.

3.2 Queue / Message Broker

The queue is the buffer that holds pending jobs between the producer and the consumer. It guarantees that jobs are not lost — if no worker is available, the job waits safely in the queue. The queue also provides ordering guarantees (first-in, first-out in most implementations), retry logic, and durability.

3.3 Worker / Consumer

Workers are separate processes that listen to the queue, pick up jobs one at a time (or in batches), and execute them. Workers are independently scalable — if the queue grows, you spin up more workers to drain it faster. A worker can run on the same server as the producer or on entirely separate infrastructure.

Scaling pattern: If jobs accumulate faster than they are processed, scale horizontally by adding more workers. The queue absorbs bursts — workers drain at their own pace.

4. Queue Technologies

Not all queue systems are the same. They differ significantly in architecture, scalability, operational complexity, and the problems they are designed to solve. Choosing the wrong tool leads to either over-engineering simple problems or under-engineering critical infrastructure.

4.1 BullMQ

BullMQ is a Redis-backed job queue built for Node.js. It is the most approachable option for teams building Node.js applications who need background job processing without the overhead of managing a dedicated messaging infrastructure. Since it runs on top of Redis — which most Node.js applications already use for caching or sessions — there is often no new infrastructure to introduce at all.

Core capabilities:

- Job retries with configurable backoff strategies (fixed, exponential)
- Delayed jobs — schedule a job to run after a specified delay
- Recurring jobs — cron-style scheduling for periodic tasks
- Job prioritisation — high-priority jobs are processed before low-priority ones
- Concurrency control — limit how many jobs a worker processes simultaneously
- Job progress tracking — workers can report percentage completion

Best suited for:

- Email sending, push notifications, and SMS delivery
- Image and media processing pipelines
- Report generation triggered by user actions
- Scheduled maintenance tasks within a Node.js application

Limitations:

- Tied to Node.js ecosystem — not suitable for polyglot architectures
- Redis is a single point of failure unless Redis Cluster is configured
- Not designed for the data volumes or throughput of a streaming system like Kafka

4.2 Amazon SQS (Simple Queue Service)

SQS is a fully managed message queuing service from AWS. It requires no infrastructure setup, no servers to manage, and no capacity planning. AWS handles all availability, scaling, and durability concerns. SQS is the right choice when you want a reliable, scalable message queue without the operational burden of running one yourself — particularly in AWS-native architectures.

Core capabilities:

- Standard queues — at-least-once delivery with best-effort ordering, near-unlimited throughput
- FIFO queues — exactly-once processing with strict ordering, up to 3,000 messages per second
- Visibility timeout — a consumed message is hidden from other consumers while being processed
- Dead Letter Queue (DLQ) — failed messages are moved to a separate queue for inspection

- Long polling — reduces unnecessary API calls by waiting for messages to arrive
- Message retention — messages can be retained for up to 14 days

Best suited for:

- Decoupling microservices — service A pushes a message, service B processes it independently
- Order processing pipelines across multiple services
- Background jobs at cloud scale without managing Redis or a broker cluster
- Any architecture already running on AWS that wants native integration

Limitations:

- Maximum message size of 256 KB — large payloads must be stored in S3 and referenced
- Not designed for real-time streaming or event replay
- Vendor lock-in — tightly coupled to the AWS ecosystem

4.3 Apache Kafka

Kafka is a distributed event streaming platform built for high-throughput, fault-tolerant, and durable message passing at massive scale. Unlike a traditional queue where messages are consumed and deleted, Kafka retains messages in an ordered log for a configurable retention period. Consumers can replay messages from any point in the log, making Kafka suitable for event sourcing, audit trails, and systems where multiple consumers need to independently process the same event stream.

Kafka is partitioned and distributed across a cluster of brokers. Each topic is divided into partitions, which are distributed across brokers for parallelism and fault tolerance. A topic with 20 partitions can be consumed by up to 20 consumers in parallel within the same consumer group, providing linear throughput scaling.

Core capabilities:

- Extremely high throughput — millions of messages per second across a cluster
- Durable event log — messages are persisted to disk and replicated across brokers
- Message replay — consumers can re-read historical messages from any offset
- Consumer groups — multiple independent consumers can each read the full event stream
- Stream processing — integrates with Kafka Streams and Apache Flink for real-time data transformation
- Compacted topics — retain only the latest value per key, useful for maintaining state

Best suited for:

- Real-time analytics pipelines — clicks, page views, user behaviour events
- Event-driven microservices where services react to domain events
- Change data capture (CDC) — propagating database changes to downstream systems
- Activity feeds, audit logs, and systems requiring full event history

Limitations:

- Operationally complex — requires a Kafka cluster, ZooKeeper or KRaft, and careful tuning
- Steep learning curve — partitioning, consumer group rebalancing, and offset management
- Overkill for simple background job queues — BullMQ or SQS is far simpler
- Latency is higher than Redis-backed queues for small message volumes

5. BullMQ vs SQS vs Kafka — Full Comparison

Feature	BullMQ	SQS	Kafka
Type	Job queue	Managed queue	Event streaming
Scale	Small–medium	Large	Very large
Setup complexity	Easy	Very easy (managed)	Complex (cluster)
Infrastructure	Redis required	Fully managed (AWS)	Kafka cluster required
Primary use case	Background jobs	Service decoupling	Data streaming
Message ordering	Yes (per queue)	FIFO (optional)	Per partition
Message replay	No	No	Yes — full history
Durability	Redis persistence	Highly durable	Extremely durable
Throughput	Moderate	Very high	Extremely high
Ecosystem	Node.js only	AWS ecosystem	Polyglot, all languages

6. When to Use Each — Decision Guide

6.1 Use BullMQ when:

- You are building a Node.js backend and need background job processing
- Your jobs are application-level tasks: sending emails, resizing images, generating PDFs
- You already use Redis and want minimal new infrastructure
- Job scheduling, retries, and delays are your primary requirements
- The system is small to medium scale with no cross-language consumers

Think: App-level background jobs. If the job could be triggered by a cron or a button click and lives entirely within your Node.js app, BullMQ is almost always the right answer.

6.2 Use SQS when:

- You need reliable, at-least-once message delivery between microservices
- You want fully managed infrastructure with no servers to operate
- You are building on AWS and want native integration with Lambda, ECS, or SNS
- You need to decouple two or more services so they can scale and fail independently
- Your team's priority is reliability and simplicity over custom control

Think: Service communication pipe. When Order Service needs to tell Payment Service to process a charge — without knowing if Payment Service is up — SQS is the right buffer.

6.3 Use Kafka when:

- You need to process millions of events per second from multiple sources
- Multiple independent consumers must each process the same event stream
- You need message replay — consumers must be able to re-read historical events
- You are building a real-time analytics pipeline, recommendation engine, or activity feed
- Your architecture is event-driven and services react to domain events

Think: Event stream backbone. When user clicks and actions need to flow in real time to analytics, recommendations, and fraud detection simultaneously, Kafka is the central nervous system.

7. Critical Concepts Every Engineer Must Know

7.1 Idempotency

Idempotency is the single most important concept in asynchronous processing. A job is idempotent if running it multiple times produces the same result as running it once. This matters because queues guarantee at-least-once delivery — in failure scenarios, the same message may be delivered and processed more than once.

Without idempotency, a user could receive two welcome emails, be charged twice for a single order, or have a database record created twice. With idempotency, processing the same job twice is harmless — the second execution detects that the work was already done and exits cleanly.

```
// Non-idempotent (dangerous):
chargeUser(userId, amount) // Could charge twice if retried

// Idempotent (safe):
chargeUser(userId, amount, idempotencyKey) // Key prevents duplicate charges
```

Design every worker with idempotency in mind. Use idempotency keys, check-then-act patterns, or database unique constraints to ensure that duplicate job delivery never causes duplicate effects.

7.2 Retry Mechanism

Workers fail. Network connections drop. External APIs return errors. A robust queue system retries failed jobs automatically with a backoff strategy — waiting longer between each successive retry attempt to avoid overwhelming a struggling downstream service.

- Immediate retry — attempt again right away (only for transient errors)
- Fixed backoff — wait a fixed interval between each retry (e.g. 5 seconds)
- Exponential backoff — double the wait time after each failure (e.g. 1s, 2s, 4s, 8s)
- Exponential backoff with jitter — add randomness to prevent thundering herd on retry

7.3 Dead Letter Queue (DLQ)

A Dead Letter Queue is a secondary queue that receives messages that have exhausted all retry attempts and still failed. Rather than losing the message entirely, the system routes it to the DLQ for manual inspection, debugging, and potential reprocessing once the underlying issue is resolved.

- Every production queue system should have a corresponding DLQ configured
- Monitor DLQ depth — a growing DLQ is an early warning of a systemic issue

- DLQ messages contain the full original message plus metadata about the failure reason

7.4 Message Durability

Durability guarantees that a message placed on the queue will not be lost — even if the broker crashes, restarts, or loses power. BullMQ achieves durability through Redis persistence (AOF or RDB snapshots). SQS stores messages redundantly across multiple AWS availability zones. Kafka replicates each partition across multiple brokers with a configurable replication factor.

- Never use an in-memory-only queue for production workloads
- Always configure replication for distributed queue systems
- Understand your queue's durability guarantees before relying on it for critical operations

7.5 Visibility Timeout and Message Locking

When a consumer picks up a message from a queue, the message is temporarily hidden from other consumers — it is not deleted yet. This window is called the visibility timeout. If the consumer successfully processes the message, it deletes it from the queue. If the consumer crashes mid-processing, the visibility timeout expires and the message reappears for another consumer to pick up.

- Set visibility timeout to at least 3x the expected job processing time
- Consumer should extend the timeout if a long-running job needs more time
- This mechanism provides at-least-once delivery guarantees

8. Real-World Architecture — Using Multiple Queue Systems Together

Large production systems often use BullMQ, SQS, and Kafka simultaneously — each handling a different layer of asynchronous work. This is not over-engineering; it is choosing the right tool for each distinct job.

```
User action → API Server
├─ BullMQ → send welcome email (app-level job)
├─ SQS    → notify Payment Service (service communication)
└─ Kafka  → publish 'user.signed_up' event (analytics pipeline)
```

In this architecture, BullMQ handles simple, application-internal jobs that need scheduling and retries. SQS decouples the communication between two microservices with guaranteed delivery. Kafka broadcasts the same event to multiple independent

consumers — the analytics system, the recommendation engine, and the fraud detection service — each processing it in their own way at their own pace.

9. Final Takeaways

Mental Model: BullMQ is a local worker. SQS is a service communication pipe. Kafka is an event stream backbone. They solve different problems at different scales.

- Asynchronous processing is essential for any operation that does not need to complete before the user receives a response
- Queues decouple producers from consumers — they can scale, fail, and deploy independently
- BullMQ is the fastest path to background jobs in Node.js; start here unless you have a reason not to
- SQS is the right choice for reliable service-to-service communication in cloud architectures
- Kafka belongs in systems that need real-time event streaming, message replay, or very high throughput
- Idempotency is non-negotiable — always design workers to be safe when executed more than once
- Always configure a Dead Letter Queue — failed messages must never silently disappear

Interview Questions & Answers

Q1: What is asynchronous processing and when should you use it?

- Asynchronous processing offloads work to a background queue so the user receives an immediate response without waiting
- Use it when an operation does not need to complete before the user gets feedback: emails, report generation, video encoding
- Also use it to decouple services — producer and consumer can scale and fail independently
- Synchronous processing is appropriate only when the user genuinely needs the result immediately to continue

Q2: What is a message queue and how does it work?

- A message queue is a buffer that stores jobs or messages between a producer and one or more consumers

- The producer pushes messages to the queue and continues immediately without waiting for processing
- Workers poll the queue, pick up messages, process them, and acknowledge completion
- The queue guarantees that messages are not lost even if consumers are temporarily unavailable

Q3: What is the difference between a queue and an event stream (Kafka vs SQS)?

- A queue (SQS, BullMQ) delivers each message to one consumer — once consumed and acknowledged, it is gone
- An event stream (Kafka) retains messages in an ordered log — multiple independent consumers can each read the full stream
- Queues are for task distribution; streams are for event broadcasting and replay
- Use SQS to send a payment job to one worker; use Kafka to broadcast a user signup event to analytics, recommendations, and fraud detection simultaneously

Q4: What is the Dead Letter Queue (DLQ) pattern?

- A DLQ is a secondary queue that receives messages that have failed processing after all retry attempts are exhausted
- Instead of losing failed messages, they are preserved for investigation, debugging, and manual reprocessing
- Monitor DLQ depth — a growing DLQ indicates a systemic processing failure that needs investigation
- Every production queue should have a corresponding DLQ configured

Q5: What is idempotency and why is it critical in message queue processing?

- Idempotency means processing the same message multiple times produces the same result as processing it once
- Critical because queues guarantee at-least-once delivery — messages may be delivered more than once in failure scenarios
- Without idempotency: a user could be charged twice, receive duplicate emails, or have duplicate records created
- Implement idempotency with a unique key per job — check if it has already been processed before executing

Q6: When would you use Kafka over SQS?

- When multiple independent consumers must each process the same event stream

- When you need message replay — consumers must re-read historical events from any point
- When throughput is in the millions of messages per second
- When building real-time data pipelines, event sourcing, or change data capture
- SQS is simpler and sufficient for most service-to-service communication; Kafka is for data streaming at scale

Q7: What is the visibility timeout in SQS?

- When a consumer picks up a message, SQS hides it from other consumers for the visibility timeout duration
- If the consumer processes and deletes the message within the timeout, all is well
- If the consumer crashes, the timeout expires and the message reappears for another consumer to pick up
- Set visibility timeout to at least 3x the expected processing time; extend it dynamically for long-running jobs

Q8: What is backpressure in a message queue system?

- Backpressure occurs when consumers cannot process messages as fast as producers publish them — the queue grows
- If left unchecked, the queue grows unboundedly, consuming memory and eventually causing failures
- Solutions: scale out workers (add more consumers), implement rate limiting on producers, or use flow control
- Monitoring queue depth is essential — a consistently growing queue is an early warning of a backpressure problem

Q9: What are the benefits of decoupling services with message queues?

- Independent scaling — producer and consumer can scale separately based on their own resource needs
- Fault isolation — if the consumer is down, the producer continues; jobs queue up and drain when the consumer recovers
- Temporal decoupling — producer and consumer do not need to be available at the same time
- Load smoothing — traffic spikes are absorbed by the queue, giving consumers time to catch up

Q10: What is the fan-out pattern in message queues?

- Fan-out means one message published to a topic is delivered to multiple queues simultaneously

- Implemented with SNS + SQS in AWS: a single SNS topic delivers to multiple SQS queues, each with its own consumer
- In Kafka, fan-out is built in — multiple consumer groups each receive the full event stream independently
- Use case: a new order event triggers the email service, inventory service, and analytics pipeline simultaneously

CHAPTER 10

Advanced Authentication & Authorization

JWT, refresh tokens, RBAC, microservices auth, and interview Q&A

1. Authentication vs Authorization — The Critical Distinction

These two terms are often used interchangeably but describe fundamentally different concepts. Confusing them leads to security vulnerabilities that are common, serious, and entirely avoidable.

Authentication (AuthN) — Who are you?

- The process of verifying the identity of a user or system
- Answers the question: is this person who they claim to be?
- Mechanisms: passwords, biometrics, OTPs, certificates
- Result: a confirmed identity (e.g. user ID 4521, email john@example.com)

Authorization (AuthZ) — What are you allowed to do?

- The process of determining what an authenticated identity is permitted to access
- Answers the question: does this person have permission to do this?
- Mechanisms: roles, permissions, access control lists, policy engines
- Result: access granted or denied to a specific resource or action

Simple analogy: Authentication is showing your ID at the building entrance. Authorization is the list of floors your keycard can access once you are inside.

2. JSON Web Tokens (JWT)

JWT is the most widely used format for representing authentication claims in modern web systems. A JWT is a compact, self-contained token that encodes user information and is cryptographically signed by the server. Because the server's signature can be

verified without a database lookup, JWTs enable stateless authentication — any server can validate any token without shared session storage.

2.1 JWT Structure

A JWT consists of three Base64URL-encoded segments separated by dots:

```
header.payload.signature
```

```
Header:    { alg: 'HS256', typ: 'JWT' }
```

```
Payload:   { userId: 42, role: 'admin', exp: 1719000000 }
```

```
Signature: HMACSHA256(base64(header) + '.' + base64(payload), secret)
```

The header declares the signing algorithm. The payload carries claims — facts about the user and the token itself, including the expiry time (exp). The signature is a cryptographic hash that proves the token was issued by a server holding the secret key, and that the payload has not been tampered with since.

Important: JWT payloads are Base64-encoded, not encrypted. Anyone who holds the token can read the payload. Never store sensitive data (passwords, secrets, PII) in a JWT payload.

2.2 Why JWT + Refresh Tokens?

A naive JWT implementation uses a single long-lived token. This creates a security problem: if the token is stolen, the attacker has access for as long as the token remains valid — potentially days or weeks. Making the token short-lived (15 minutes) solves the theft problem but creates a usability problem: the user is logged out every 15 minutes.

The solution is a two-token system. A short-lived access token is used for API calls. A long-lived refresh token is used only to obtain a new access token when the old one expires. The refresh token is stored securely (server-side in a database, client-side in an HTTP-only cookie) and can be revoked at any time.

```
Access Token: expires in 15 minutes → used for every API request
```

```
Refresh Token: expires in 7 days → used only to get a new access token
```

Flow:

```
Login → receive both tokens
```

```
API request → send access token
```

```
Access token expires → send refresh token → receive new access token
```

```
Logout → delete refresh token from DB → both tokens invalidated
```

2.3 Login API Implementation (Node.js)

The login endpoint verifies credentials, generates both tokens, stores the refresh token server-side, and returns both to the client.

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await findUserByEmail(email);
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });

  const isValid = await bcrypt.compare(password, user.passwordHash);
  if (!isValid) return res.status(401).json({ error: 'Invalid credentials' });

  const accessToken = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.ACCESS_SECRET,
    { expiresIn: '15m' }
  );
  const refreshToken = jwt.sign(
    { userId: user.id },
    process.env.REFRESH_SECRET,
    { expiresIn: '7d' }
  );

  await saveRefreshToken(user.id, refreshToken); // persist in DB
  res.cookie('refreshToken', refreshToken, { httpOnly: true, secure: true });
  res.json({ accessToken });
});
```

2.4 Authentication Middleware

Every protected route runs the authentication middleware, which extracts the access token from the Authorization header, verifies its signature and expiry, and attaches the decoded user object to the request.

```
function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;
  const token = authHeader?.startsWith('Bearer ')
    ? authHeader.split(' ')[1] : null;

  if (!token) return res.status(401).json({ error: 'No token provided' });

  jwt.verify(token, process.env.ACCESS_SECRET, (err, decoded) => {
    if (err) return res.status(403).json({ error: 'Invalid or expired token' });
  });
}
```

```
req.user = decoded; // { userId, role, iat, exp }
next();
});
}
```

2.5 Token Refresh Endpoint

The refresh endpoint validates the incoming refresh token against the database (confirming it has not been revoked), then issues a new access token. Token rotation — issuing a new refresh token and invalidating the old one on each refresh — provides an additional layer of security by limiting the window of exposure if a refresh token is stolen.

```
app.post('/refresh', async (req, res) => {
  const refreshToken = req.cookies.refreshToken;
  if (!refreshToken) return res.sendStatus(401);

  const isValid = await isRefreshTokenInDB(refreshToken);
  if (!isValid) return res.sendStatus(403); // revoked or not found

  jwt.verify(refreshToken, process.env.REFRESH_SECRET, async (err, decoded) => {
    if (err) return res.sendStatus(403);

    // Token rotation: revoke old, issue new
    await deleteRefreshToken(refreshToken);
    const newRefreshToken = jwt.sign(
      { userId: decoded.userId }, process.env.REFRESH_SECRET, { expiresIn: '7d'
    });
    await saveRefreshToken(decoded.userId, newRefreshToken);

    const newAccessToken = jwt.sign(
      { userId: decoded.userId }, process.env.ACCESS_SECRET, { expiresIn: '15m'
    });
    res.cookie('refreshToken', newRefreshToken, { httpOnly: true, secure: true
  });
  res.json({ accessToken: newAccessToken });
});
```

2.6 JWT Security Best Practices

- Always store secrets in environment variables — never hardcode them
- Use HTTP-only, Secure cookies for refresh tokens — JavaScript cannot read them, reducing XSS risk

- Use short access token expiry — 15 minutes is a common production value
- Rotate refresh tokens on every use — detect replay attacks
- Use asymmetric keys (RS256) in microservices — private key signs, public key verifies
- Log and alert on refresh token reuse — may indicate token theft

3. Role-Based Access Control (RBAC)

Authentication tells you who a user is. Authorization tells you what they can do. RBAC is the most widely used authorization model — it assigns permissions to roles, then assigns roles to users. A user inherits all permissions of their assigned role.

3.1 The Role Hierarchy Model

In a typical application, roles form a hierarchy where higher roles inherit the permissions of lower ones. This reduces duplication and makes permission management easier to reason about.

```
Admin    → create, read, update, delete, manage users
Manager  → create, read, update
User     → read only
Guest    → limited read (public content only)
```

3.2 RBAC Middleware Implementation

The authorization middleware runs after authentication. By the time it executes, `req.user` is already populated with the authenticated user's role from the JWT payload. The middleware simply checks whether the user's role is in the list of permitted roles for that route.

```
function authorize(allowedRoles = []) {
  return (req, res, next) => {
    if (!req.user) return res.status(401).json({ error: 'Not authenticated' });

    if (!allowedRoles.includes(req.user.role)) {
      return res.status(403).json({
        error: 'Forbidden: insufficient permissions'
      });
    }
    next();
  };
}

// Usage on routes:
app.delete('/users/:id', authenticate, authorize(['admin']), deleteUser);
```

```
app.put('/reports/:id', authenticate, authorize(['admin','manager']),
updateReport);
app.get('/dashboard', authenticate, authorize(['admin','manager','user']),
getDashboard);
```

3.3 Permission-Based Access Control

Pure RBAC works well for simple role hierarchies, but breaks down when you need fine-grained control — for example, a user who can read all reports but only update reports they created. Permission-based access control assigns individual permissions directly, or attaches them to roles more granularly.

```
// Role definition with explicit permissions:
const ROLES = {
  admin: ['user:create','user:delete','report:read','report:write'],
  manager: ['report:read','report:write','user:read'],
  user: ['report:read'],
};

function hasPermission(role, permission) {
  return ROLES[role]?.includes(permission) ?? false;
}
```

This model gives you the ability to grant and revoke individual capabilities without changing the role structure. It is more complex to maintain than simple role checks but is the right choice when your access control requirements grow beyond a few roles.

4. Authentication in Microservices

In a monolith, authentication is straightforward — one codebase, one session store, one place to check credentials. In microservices, each service is independent, potentially written in different languages, deployed separately, and receives requests from multiple sources. Authentication must be designed as a system-wide concern, not a per-service implementation detail.

4.1 The Wrong Approach — Auth in Every Service

A common mistake is to implement authentication independently in each microservice. Every service validates credentials, maintains its own session logic, and connects to its own user store. This leads to duplicated code, inconsistent security policies, and a maintenance burden that grows with every new service added.

```
BAD: Client → Order Service (has its own auth logic)
      → Payment Service (has its own auth logic)
      → User Service (has its own auth logic)
```

4.2 The Correct Approach — Centralised Auth with API Gateway

The correct pattern centralises all authentication in two places: a dedicated Auth Service that issues tokens, and an API Gateway that validates them. Every request from a client passes through the API Gateway first. The gateway validates the JWT, extracts user identity and role information from the token, and forwards the verified request to the appropriate microservice. Services trust the gateway and never re-validate tokens.

```
GOOD:
Client
  ↓
API Gateway ← validates JWT, extracts user identity
  ↓
Microservices ← receive pre-verified user context in headers
```

4.3 Full Microservices Auth Flow

22. User submits credentials to the Auth Service via the API Gateway
23. Auth Service validates credentials and issues a signed JWT (access token + refresh token)
24. Client stores the access token and sends it in the Authorization header on every request
25. API Gateway intercepts the request, verifies the JWT signature using the public key
26. Gateway extracts user identity and role, injects them as trusted headers (X-User-Id, X-User-Role)
27. Downstream service receives the request with user context — it trusts the gateway and does not re-validate
28. Service applies its own authorization rules using the injected user context

4.4 Responsibilities in Microservices Auth

Auth Service:

- Login, logout, and registration logic
- Token generation (access + refresh) and token rotation
- Refresh token persistence and revocation

API Gateway:

- Token validation on every inbound request
- Route protection — reject requests with invalid or missing tokens before they reach services
- Inject verified user identity into downstream request headers
- Rate limiting and IP-based access control

Microservices:

- Trust the user context injected by the gateway — do not re-authenticate
- Apply service-level authorization: does this user have permission to perform this operation?
- Never expose internal endpoints directly — only accessible via the gateway

4.5 Service-to-Service Authentication

When microservices communicate with each other internally — Order Service calling Payment Service, for example — they need their own authentication mechanism separate from user JWTs. Two common approaches are:

Shared service tokens:

- Each service has a long-lived token used for internal API calls
- Simple to implement, but token rotation is a manual process

Mutual TLS (mTLS):

- Each service has its own certificate — both sides verify each other's identity
- The gold standard for zero-trust internal network security
- Supported natively by service meshes like Istio

4.6 Challenges in Distributed Auth

Token revocation:

Because JWTs are stateless, there is no way to invalidate a token before it expires — unless you maintain a blacklist. A common pattern is to keep a small, in-memory blacklist in Redis with the JTI (JWT ID) of revoked tokens, checked on each request by the gateway.

Role synchronisation:

If a user's role changes (e.g. promoted from User to Manager), the change does not take effect until their current JWT expires and a new one is issued. To force immediate role updates, either use very short token expiry, or maintain a token invalidation list.

Public key distribution:

In microservices using asymmetric signing (RS256), each service needs access to the Auth Service's public key to verify tokens independently. A JWKS (JSON Web Key Set) endpoint published by the Auth Service is the standard solution — services fetch and cache the public keys automatically.

5. OAuth 2.0 — Delegated Authorization

OAuth 2.0 is a protocol that allows users to grant a third-party application access to their account on another service, without sharing their password. When you see 'Sign in with Google' or 'Continue with GitHub', that is OAuth in action. The user authenticates with the identity provider (Google, GitHub), which issues a token to your application on the user's behalf.

5.1 OAuth vs JWT — Common Confusion

OAuth and JWT are frequently confused because they are often used together. They are completely different things:

- OAuth 2.0 is an authorization protocol — it defines the flow and rules for granting access
- JWT is a token format — it defines how to encode and sign claims in a compact string
- OAuth flows often issue JWTs as their access tokens, but OAuth can also issue opaque tokens
- JWT can be used entirely outside of OAuth — your own login system can issue JWTs without any OAuth flow

5.2 OAuth Authorization Code Flow

1. User clicks 'Sign in with Google'
2. App redirects to Google's auth server with `client_id` and `redirect_uri`
3. User authenticates with Google and grants permissions
4. Google redirects back to app with an authorization code
5. App exchanges code for access token (server-to-server, secret stays hidden)
6. App uses access token to call Google APIs on user's behalf

The authorization code flow is the most secure OAuth pattern because the access token is never exposed in the browser URL. The code exchange happens server-to-server, keeping your client secret safe.

6. Interview Q&A

Q1: What is the difference between JWT and session-based authentication?

Session-based authentication stores session state on the server — a session ID is stored in a database or memory store, and the browser holds a cookie with that ID. On each request, the server looks up the session. JWT authentication is stateless — the token contains all user information, signed by the server. There is no server-side lookup required.

- JWT: stateless, scales horizontally without shared session store, tokens cannot be invalidated before expiry
- Sessions: stateful, easier to invalidate (just delete the session), requires shared store in distributed systems
- Rule of thumb: prefer JWT for APIs and microservices; sessions work well for traditional server-rendered apps

Q2: How do you invalidate a JWT before it expires?

JWT invalidation is one of the most common interview follow-up questions because it exposes a real limitation of stateless tokens. Options include:

- Short expiry — 15-minute access tokens limit the damage window to 15 minutes
- Token blacklist in Redis — store the JTI of revoked tokens; gateway checks the blacklist on each request
- Refresh token rotation — when a refresh token is used, the old one is revoked; access tokens remain short-lived
- Version field on user record — include a token version in the JWT; increment on logout; reject tokens with stale versions

Q3: What is OAuth 2.0 and how does it differ from authentication?

OAuth 2.0 is a delegation protocol, not an authentication protocol. It answers 'what can this app access on your behalf?' not 'who are you?' OpenID Connect (OIDC) is a thin identity layer built on top of OAuth 2.0 that adds authentication. When you use 'Sign in with Google', you are using OIDC for authentication, which runs on top of OAuth 2.0 for the delegation mechanics.

Q4: How does authentication work in a microservices architecture?

A dedicated Auth Service issues signed JWTs. The API Gateway validates every incoming token before routing the request. Downstream services trust the gateway and receive user identity via injected request headers. Services never re-authenticate independently. Refresh token rotation is handled exclusively by the Auth Service.

Q5: How do you secure an API?

- JWT validation on every request via middleware
- HTTPS everywhere — never transmit tokens over HTTP

- Rate limiting — prevent brute force and credential stuffing attacks
- Input validation and sanitisation — prevent injection attacks
- RBAC — enforce least-privilege access on every endpoint
- Rotate secrets regularly — API keys, JWT signing secrets, database credentials
- Audit logging — record every authentication event and sensitive operation

Q6: What are common authentication security mistakes?

- Storing passwords in plain text — always use bcrypt, scrypt, or Argon2
- No token expiry — an access token that never expires is equivalent to a permanent password
- Storing tokens in localStorage — vulnerable to XSS; use HTTP-only cookies for refresh tokens
- Not using HTTPS — tokens transmitted over HTTP can be intercepted
- Weak JWT secrets — a short or guessable secret can be brute-forced offline
- No refresh token rotation — a stolen refresh token remains valid indefinitely
- Overly broad roles — violates least-privilege; users have access they do not need

Q7: What is the role of the API Gateway in authentication?

The API Gateway acts as the security perimeter for all inbound traffic. It validates JWT signatures, checks token expiry, enforces rate limits, and rejects unauthorised requests before they ever reach a microservice. This centralises security enforcement — no service needs its own token validation logic. The gateway also injects verified user identity into downstream requests as trusted headers, so services can apply authorisation without re-doing authentication.

7. Final Mental Model and Takeaways

Login	→ Authentication (who are you?)
JWT	→ Portable, verifiable identity token
Refresh token	→ Long-lived credential for renewing access tokens
RBAC	→ Authorization (what can you do?)
API Gateway	→ Central security enforcement layer
OAuth 2.0	→ Delegated access to external provider resources
mTLS	→ Service-to-service identity in zero-trust networks

Core principle: Security is not a feature you add at the end — it is an architectural decision made at the start. Build authentication and authorization into the system design from day one.

- Use JWT + refresh token rotation for all modern API authentication — short-lived access tokens, server-stored refresh tokens
- Never store sensitive data in JWT payloads — the payload is readable by anyone who holds the token
- Use HTTP-only Secure cookies for refresh tokens — they are inaccessible to JavaScript, reducing XSS exposure
- Centralise authentication in an Auth Service and API Gateway in microservices — never replicate auth logic across services
- RBAC is the foundation of authorization — start with roles, add permission granularity only when the complexity justifies it
- Design for token revocation from the beginning — a system that cannot log users out is not production-ready

CHAPTER 11

CAP Theorem

Consistency, Availability, and Partition Tolerance in distributed systems

1. What Is the CAP Theorem?

The CAP Theorem, formulated by computer scientist Eric Brewer in 2000 and formally proved by Gilbert and Lynch in 2002, states that any distributed data system can guarantee at most two of the following three properties simultaneously:

- Consistency (C) — every read receives the most recent write or an error
- Availability (A) — every request receives a response, though it may not be the most recent data
- Partition Tolerance (P) — the system continues operating even when network partitions cause some nodes to be unable to communicate with others

The fundamental rule: when a network partition occurs, you must choose between Consistency and Availability. You cannot have both.

Understanding CAP is essential not just for passing interviews, but for making informed decisions when choosing databases and designing distributed systems. The theorem explains why no single database can be the perfect solution for every problem, and why trade-offs are unavoidable at scale.

2. Understanding Each Property

2.1 Consistency (C)

In the CAP context, consistency means that all nodes in a distributed system return the same data at the same time. After a write completes, any subsequent read — from any node in the system — must return that updated value. There is no version of events where two users querying different nodes see different values for the same piece of data.

This is the guarantee a bank absolutely requires. If you transfer £1,000 from Account A to Account B, every subsequent query to any node in the system must reflect that transfer — no node should report the old balance. In a banking context, serving stale data is not just a UX problem; it is a financial and legal liability.

What consistency means in practice:

- Write to node A — all nodes immediately reflect the new value before any read is served
- If a node cannot guarantee it has the latest data, it must refuse to respond rather than return stale data
- Consistency in CAP is about cross-node agreement, not the same as ACID consistency (which is about valid database state)

2.2 Availability (A)

Availability means that every request to the system receives a response — the system never returns an error or timeout. It does not guarantee that the response contains the most current data, only that a response is always given. An available system keeps serving users even when some of its nodes are unhealthy or out of sync.

Social media platforms prioritise availability. If the server storing the definitive like count for a post is temporarily partitioned, the system still shows you a count — it might be a few seconds out of date, but you are never shown an error page. The temporary inconsistency is an acceptable trade-off for continuous service.

What availability means in practice:

- Every non-failing node returns a response for every request — no request is rejected due to a partition
- The response may contain stale data from before the partition occurred
- Availability does not mean 100% uptime — it describes the system's behaviour during a partition, not its overall reliability

2.3 Partition Tolerance (P)

A network partition is a situation where some nodes in the distributed system cannot communicate with others due to a network failure — a broken cable, a misconfigured router, a cloud availability zone going offline, or simply a network packet being dropped. Partition tolerance means the system continues to operate correctly even when such a partition occurs.

Critical insight: In any real distributed system, network partitions are not edge cases — they are inevitable. Network hardware fails, packets are lost, and data centres have outages. Partition tolerance is not optional; it is a prerequisite for any system that runs on more than one machine.

Because partition tolerance is non-negotiable in distributed systems, the real-world choice CAP presents is not which two of three to pick — it is what your system does when a partition happens: does it sacrifice consistency or availability?

3. The Three System Types

3.1 CP Systems — Consistency + Partition Tolerance

A CP system prioritises correctness over availability. When a network partition occurs, the system stops accepting writes (and sometimes reads) on the affected nodes rather than risk returning or accepting inconsistent data. The user may receive an error or a timeout, but they will never receive incorrect data.

Network partition occurs:

Node B cannot reach Node A

CP system: Node B refuses requests → returns error

Result: user sees error, but data is never stale or wrong

Real-world use cases:

- Banking and financial systems — incorrect balance data is worse than a brief outage
- Distributed locking and coordination — a lock must be either held or not; ambiguity causes race conditions
- Inventory management — overselling is worse than temporarily refusing orders

Database examples: PostgreSQL, HBase, Zookeeper, etcd

3.2 AP Systems — Availability + Partition Tolerance

An AP system prioritises uptime over correctness. When a network partition occurs, all nodes continue accepting requests and serving responses, even if some nodes have stale data. Once the partition heals, nodes synchronise and eventually converge to a consistent state — this is called eventual consistency.

Network partition occurs:

Node B cannot reach Node A

AP system: Node B continues serving requests with its last known data

Result: user gets a response, but it may reflect stale values

Real-world use cases:

- Social media feeds — a slightly delayed like count or follower count is acceptable
- Product catalogues — showing a slightly stale price for a few seconds is harmless
- DNS — DNS servers return cached records during partitions rather than refusing to resolve
- Shopping carts — availability is more important than perfect real-time sync

Database examples: Apache Cassandra, DynamoDB, CouchDB, Riak

3.3 CA Systems — Consistency + Availability (Not Realistic in Practice)

A CA system would offer both consistency and availability but cannot tolerate network partitions. This is theoretically clean but practically useless for distributed systems, because — as established — partitions are unavoidable. A CA system is essentially a single-node system: a standalone SQL database running on one machine has no partitions to worry about, so it can be both consistent and available. The moment you distribute it across multiple nodes, you must confront partition tolerance.

This is why CA is sometimes described as a non-choice — it applies only to single-node systems, which by definition are not distributed systems.

4. What Actually Happens During a Network Partition

To make CAP concrete, consider a distributed database with two nodes — Node A in London and Node B in Singapore — and the network link between them goes down.

Before partition: Node A $\longleftrightarrow$ network $\longleftrightarrow$ Node B (in sync)

During partition: Node A $\longleftrightarrow$ BROKEN $\longrightarrow$ Node B (isolated)

4.1 If the system chooses Consistency (CP)

29. A user in Singapore writes a new value to Node B
30. Node B cannot replicate the write to Node A because the network is down
31. Node B refuses the write — returns an error to the user
32. No inconsistency is introduced — both nodes have the same (old) data
33. When the partition heals, operations resume normally

The user is frustrated by the error, but the system's data integrity is preserved. For a bank transfer, this is the correct behaviour.

4.2 If the system chooses Availability (AP)

34. A user in Singapore writes a new value to Node B
35. Node B accepts the write and returns success to the user
36. A user in London reads from Node A — gets the old (pre-write) value
37. The two nodes are now inconsistent — they hold different data
38. When the partition heals, nodes reconcile and eventually converge to the same value

The user in London sees stale data briefly. For a social media like count, this is perfectly acceptable. For a bank balance, it is not.

5. Eventual Consistency — The AP System's Promise

Eventual consistency is the consistency model used by AP systems. It guarantees that if no new updates are made to a data item, all nodes will eventually return the same value for it. It does not guarantee when convergence will happen, only that it will.

Eventual consistency is often misunderstood as meaning 'no consistency guarantees at all'. In reality, it means exactly-once delivery of updates to all nodes over time, with conflict resolution when two nodes have conflicting writes. The system is designed to heal itself after partitions automatically.

Techniques for managing eventual consistency:

- Last-write-wins (LWW) — the write with the most recent timestamp takes precedence during reconciliation
- Vector clocks — track the causal history of writes so conflicts can be resolved intelligently

- CRDTs (Conflict-free Replicated Data Types) — data structures designed to merge automatically without conflicts
- Read repair — when a read detects inconsistency across nodes, it triggers a background synchronisation

Real-world example: Amazon's shopping cart uses eventual consistency. If you add an item on your phone while offline, the cart reconciles when you reconnect. The system prefers accepting both writes and merging them over losing one.

6. Beyond CAP — The PACELC Theorem

CAP Theorem only describes system behaviour during a network partition. But what about normal operation, when there is no partition? The PACELC theorem (Daniel Abadi, 2010) extends CAP by addressing this gap:

```
If Partition (P): choose between Availability (A) vs Consistency (C)
Else (E):         choose between Latency (L) vs Consistency (C)
```

PACELC recognises that even without failures, distributed systems face a trade-off between latency and consistency. Achieving strong consistency during normal operation requires coordinating writes across multiple nodes — which takes time and increases latency. Eventual consistency is faster because writes are acknowledged without waiting for all nodes to agree.

Database classification under PACELC:

- Cassandra — PA/EL: available during partitions, low latency during normal ops (eventual consistency always)
- PostgreSQL — PC/EC: consistent during partitions (returns errors), consistent during normal ops (higher latency)
- DynamoDB — PA/EL by default, configurable to PC/EC for strongly consistent reads

7. CAP in Database Selection

Database	C	A	P	CAP Type
PostgreSQL	Yes	No*	Yes	CP
MySQL	Yes	No*	Yes	CP

Database	C	A	P	CAP Type
Apache Cassandra	No*	Yes	Yes	AP
DynamoDB	No*	Yes	Yes	AP (default)
MongoDB	Yes	No*	Yes	CP (default)
Redis Cluster	No*	Yes	Yes	AP
Zookeeper	Yes	No*	Yes	CP
CouchDB	No*	Yes	Yes	AP

* 'No' means the property is sacrificed during a network partition. During normal operation, these databases may appear to offer all three properties. The asterisk marks the property that is compromised when a partition forces a choice.

8. Common Misunderstandings

8.1 CAP Consistency vs ACID Consistency

These are two completely different concepts that share a name. CAP Consistency means all nodes in a distributed system return the same data at the same time — it is about cross-node agreement. ACID Consistency means the database transitions from one valid state to another — it is about enforcing database rules and constraints. A database can have ACID consistency without CAP consistency if different nodes hold different versions of the data.

8.2 The Choice Is Not Made at Design Time for All Time

Many databases allow you to tune the consistency level per operation. Cassandra lets you specify a consistency level on each query — QUORUM requires a majority of nodes to agree before responding, providing stronger consistency at the cost of latency. ONE returns from the first node that responds, maximising speed but accepting potential staleness. This means you can use an AP database for most reads but request strong consistency only for the operations that need it.

8.3 CAP Only Applies During Partitions

A common misconception is that you are always operating in a degraded mode under CAP. In practice, network partitions are rare. During normal operation with no partition, a well-designed system can provide both consistency and availability. CAP only forces the

trade-off when a partition actually occurs. The design decision is: what should the system do when that rare partition happens?

9. Decision Framework — Choosing CP or AP

Choose CP (prioritise Consistency) when:

- Incorrect data causes financial loss, legal liability, or safety issues
- The domain requires strict ordering of operations (distributed locking, leader election)
- Users would rather see an error than incorrect information
- Examples: banking, payments, inventory management, healthcare records

Choose AP (prioritise Availability) when:

- The system must always respond, even with slightly stale data
- Brief inconsistency is acceptable and self-correcting over time
- Scale and throughput requirements are extremely high
- Users prefer an approximate answer over no answer
- Examples: social media, product catalogues, DNS, shopping carts, analytics

Practical rule: ask yourself what is worse for your users — receiving an error message, or receiving data that is a few seconds out of date. The answer tells you whether to choose CP or AP.

Interview Questions & Answers

Q1: What is the CAP Theorem and what does it guarantee?

- CAP states that a distributed system can guarantee at most two of: Consistency, Availability, and Partition Tolerance
- Consistency: all nodes return the same data at the same time
- Availability: every request receives a response (possibly with stale data)
- Partition Tolerance: the system operates despite network failures between nodes
- Since partitions are unavoidable in distributed systems, the real choice is always between Consistency and Availability during a partition

Q2: Why is Partition Tolerance non-negotiable?

- Network partitions are a physical reality — hardware fails, cables break, and data centres have outages
- Any system running on more than one machine will eventually experience a partition
- A system that stops working entirely during any network failure is not useful in production
- Therefore all distributed systems must tolerate partitions, reducing the real choice to C vs A

Q3: What is the difference between CP and AP systems?

- CP systems sacrifice availability during partitions — they return errors rather than risk inconsistent data
- AP systems sacrifice consistency during partitions — they return responses (possibly stale) rather than errors
- CP example: a bank database that refuses writes during a partition to prevent two accounts from showing the same balance
- AP example: a social media platform that continues showing follower counts even if they are a few seconds stale

Q4: What is eventual consistency?

- A consistency model used by AP systems where all nodes will converge to the same value over time
- It does not guarantee when convergence happens, only that it will happen if updates stop
- Acceptable for data where brief staleness is tolerable: like counts, view counts, product recommendations
- Not acceptable for financial data, inventory counts, or any scenario where stale data has real-world consequences

Q5: How does Cassandra handle the CAP trade-off?

- Cassandra is an AP system by default — it prioritises availability and partition tolerance
- During a partition, Cassandra continues accepting reads and writes on isolated nodes
- After the partition heals, nodes reconcile using a last-write-wins strategy based on timestamps
- However, Cassandra allows per-query consistency tuning — QUORUM level provides stronger consistency at the cost of latency

Q6: What is the difference between CAP Consistency and ACID Consistency?

- CAP Consistency: all nodes in a distributed system return the same data at the same time — about cross-node agreement
- ACID Consistency: the database always transitions between valid states, enforcing constraints and rules — about data integrity on a single node
- A database can have ACID consistency (never violating constraints) while lacking CAP consistency (different nodes having different values)
- They are completely different properties that happen to share a name

Q7: What is the PACELC theorem?

- An extension of CAP that addresses system behaviour during normal operation (no partition)
- PACELC: if Partition, choose between Availability vs Consistency; Else, choose between Latency vs Consistency
- Even without failures, strong consistency requires coordinating across nodes — this takes time and increases latency
- PACELC explains why eventually consistent databases are faster during normal operation, not just more available during failures

Q8: Give a real-world example of a system choosing AP over CP.

- Amazon's shopping cart is a classic example — it uses eventual consistency
- If you add an item while offline or during a partition, the cart accepts the change locally
- When connectivity is restored, the cart reconciles changes across nodes — it merges rather than rejects
- Amazon explicitly chose to never lose a cart item (availability) over having perfect real-time consistency

Q9: Can a database switch between CP and AP dynamically?

- Yes — many databases allow per-operation consistency level configuration
- Cassandra's consistency levels: ONE (fastest, AP behaviour), QUORUM (majority agreement, near-CP), ALL (all nodes must agree, full CP)
- DynamoDB has strongly consistent reads as an option alongside eventually consistent reads
- This allows using an AP database for most operations while requesting strong consistency only where critical

Q10: What is a split-brain scenario?

- Split-brain occurs when a network partition causes two nodes to both believe they are the authoritative primary
- Both nodes accept writes independently, leading to conflicting data that is difficult to reconcile

- CP systems prevent split-brain by requiring quorum — a node only accepts writes if a majority of nodes agree
- AP systems allow split-brain and resolve it later through conflict resolution strategies like last-write-wins or CRDTs

Final Takeaways

Mental Model: Consistency is correct data. Availability is always responding. Partition tolerance is surviving network failures. In a distributed system, you can never guarantee all three simultaneously — design your system knowing which one you are willing to sacrifice.

- CAP applies only to distributed systems — a single-node database faces no partition trade-off
- Partition tolerance is mandatory — the real decision is always Consistency vs Availability during a partition
- CP systems (PostgreSQL, Zookeeper) prioritise correctness — use for financial, medical, and coordination workloads
- AP systems (Cassandra, DynamoDB) prioritise uptime — use for high-scale, read-heavy, or analytics workloads
- Eventual consistency is not 'no consistency' — it is a promise that the system will self-correct after a partition heals
- PACELC extends CAP to normal operations — consistency costs latency even without failures
- Many modern databases allow per-query consistency tuning — you are not locked into one mode permanently

CHAPTER 12

Indexing & Query Optimization

Making queries go from seconds to milliseconds

1. What Is an Index?

An index is a separate data structure maintained by the database that allows it to locate rows matching a query condition without scanning every row in the table. Think of it like the index at the back of a textbook — instead of reading every page to find a topic, you look it up in the index and jump directly to the right page.

Without an index, the database performs a full table scan — it reads every single row sequentially, checking whether each one satisfies the query condition. For a table with 1,000 rows this is fast. For a table with 10 million rows, it can take several seconds per query. At scale, unindexed queries on large tables are one of the most common causes of database performance degradation.

Analogy: A database table without indexes is a book with no index — you must read every page to find what you need. An indexed table is a book with a detailed index — you go straight to the right page.

2. Full Table Scan vs Index Lookup

2.1 Without an Index — Full Table Scan

Consider a users table with 5 million rows. When you run the following query without an index on the email column, the database must check every row:

```
SELECT * FROM users WHERE email = 'john@example.com';
```

Database execution (no index):

```
Row 1:      email = 'alice@example.com'  - no match, continue
Row 2:      email = 'bob@example.com'    - no match, continue
...
Row 3,847,291: email = 'john@example.com' - match found!
...
Row 5,000,000: checked (scan complete)
```

- Time complexity: $O(n)$ — scales linearly with table size
- For 5 million rows: potentially millions of disk reads
- Response time: seconds to tens of seconds at large scale

2.2 With an Index — Direct Lookup

After creating an index on the email column, the same query is dramatically faster:

```
CREATE INDEX idx_users_email ON users(email);
```

```
SELECT * FROM users WHERE email = 'john@example.com';
```

Database execution (with B-Tree index):

```
Binary search on sorted index structure
→ Level 1: compare with midpoint
→ Level 2: narrow to correct subtree
```

```
→ Level 3: locate exact entry → fetch row  
Total steps: ~23 for 5 million rows (log2 of 5,000,000 ≈ 22.25)
```

- Time complexity: $O(\log n)$ — scales logarithmically with table size
- For 5 million rows: approximately 23 comparisons instead of 5 million
- Response time: sub-millisecond

3. How B-Tree Indexes Work Internally

The overwhelming majority of database indexes — in PostgreSQL, MySQL, SQL Server, and SQLite — use a data structure called a B-Tree (Balanced Tree). Understanding how it works explains both why indexes are so fast and why they have specific limitations.

A B-Tree is a self-balancing tree where all leaf nodes are at the same depth. Each internal node contains sorted keys and pointers to child nodes. The leaf nodes contain the actual indexed values along with pointers to the corresponding table rows (row IDs or primary key values). Because the tree is always balanced, the path from the root to any leaf is the same length — guaranteeing $O(\log n)$ lookup time regardless of which value is searched.

How a B-Tree lookup works:

39. Start at the root node — contains a sorted list of key ranges
40. Compare the search value against the keys to determine which child subtree to follow
41. Recurse down through internal nodes, each comparison halving the remaining search space
42. Reach the leaf node — contains the indexed value and a pointer to the actual table row
43. Follow the pointer to fetch the full row from the table (heap fetch)

B-Trees also support range queries efficiently — because the leaf nodes are sorted and linked, the database can find the start of a range (e.g. `WHERE created_at > '2024-01-01'`) and then scan forward through the linked leaf nodes without returning to the root.

B+ Tree vs B-Tree: Most databases actually use B+ Trees, a variant where all data pointers live in the leaf nodes (not internal nodes) and leaf nodes are linked together. This makes range scans even faster because the database can scan leaves sequentially after finding the start point.

4. Types of Indexes

4.1 Single-Column Index

The most common type — an index on a single column. Speeds up queries that filter, sort, or join on that column.

```
CREATE INDEX idx_users_email ON users(email);

-- Uses the index:
SELECT * FROM users WHERE email = 'john@example.com';
SELECT * FROM users ORDER BY email;
```

4.2 Composite (Multi-Column) Index

An index on two or more columns. The order of columns in the definition is critical — it determines which query patterns benefit from the index.

```
CREATE INDEX idx_users_name ON users(last_name, first_name);

-- Uses the index (leading column present):
WHERE last_name = 'Smith'
WHERE last_name = 'Smith' AND first_name = 'John'

-- Does NOT use the index (leading column absent):
WHERE first_name = 'John'
```

This is called the left-prefix rule: a composite index on (A, B, C) can be used for queries on A, A+B, or A+B+C — but not for queries on B alone or C alone. Design composite indexes with the most selective or most frequently queried column first.

4.3 Unique Index

Enforces uniqueness on a column or combination of columns, and simultaneously speeds up lookups. Creating a PRIMARY KEY or UNIQUE constraint automatically creates a unique index.

```
CREATE UNIQUE INDEX idx_users_email_unique ON users(email);
-- Now any attempt to insert a duplicate email raises an error
```

4.4 Full-Text Index

Optimised for text search queries — finding documents that contain specific words or phrases. Standard B-Tree indexes cannot efficiently support LIKE '%keyword%' patterns, but full-text indexes are built specifically for this use case.

```
-- PostgreSQL full-text search
CREATE INDEX idx_articles_fts ON articles USING gin(to_tsvector('english',
body));

SELECT * FROM articles
WHERE to_tsvector('english', body) @@ to_tsquery('database & indexing');
```

4.5 Hash Index

Uses a hash function to map column values to index entries. Extremely fast for exact equality lookups ($O(1)$ average), but completely useless for range queries or sorting because hash values have no meaningful order.

```
CREATE INDEX idx_sessions_token ON sessions USING hash(token);
```

- Good for: exact equality lookups on high-cardinality columns (e.g. session tokens, UUIDs)
- Bad for: range queries (WHERE id > 100), sorting (ORDER BY), LIKE patterns
- PostgreSQL supports hash indexes; MySQL's InnoDB uses B-Trees for most cases

4.6 Partial Index

An index on a subset of rows, filtered by a WHERE condition. Smaller and faster than a full index, ideal when queries consistently target a specific subset of data.

```
-- Index only active users - much smaller than indexing all users
CREATE INDEX idx_active_users ON users(email) WHERE status = 'active';

-- This query uses the partial index:
SELECT * FROM users WHERE email = 'john@example.com' AND status = 'active';
```

4.7 Covering Index

An index that includes all columns required by a query — eliminating the need to fetch the actual table row after finding the index entry. When a query can be satisfied entirely from the index, it is called an index-only scan and is significantly faster.

```
-- Query needs id, name, email
CREATE INDEX idx_users_covering ON users(email) INCLUDE (id, name);

SELECT id, name FROM users WHERE email = 'john@example.com';
-- No heap fetch needed - all data is in the index itself
```

5. When Indexes Are NOT Used — Common Pitfalls

Creating an index does not guarantee the query planner will use it. Several common query patterns prevent index usage, silently causing full table scans even when an index exists.

5.1 Leading Wildcard in LIKE

```
-- Index on 'name' is NOT used:
WHERE name LIKE '%john%'    -- leading wildcard prevents index use
WHERE name LIKE '%john'     -- same problem

-- Index IS used:
WHERE name LIKE 'john%'     -- trailing wildcard only — index usable
```

The reason: a B-Tree index is sorted by the full value from left to right. A leading wildcard means the database does not know which part of the sorted index to start from — it must check every entry. Use full-text indexes for substring search requirements.

5.2 Functions Applied to Indexed Columns

```
-- Index on 'email' is NOT used:
WHERE LOWER(email) = 'john@example.com'
WHERE YEAR(created_at) = 2024
WHERE email || '@domain.com' = 'john@domain.com'

-- Solution — use a functional/expression index:
CREATE INDEX idx_email_lower ON users(LOWER(email));
WHERE LOWER(email) = 'john@example.com' -- now uses the expression index
```

5.3 Implicit Type Conversion

```
-- Column 'user_id' is INTEGER, but queried as string:
WHERE user_id = '12345' -- database converts all rows → full scan

-- Correct:
WHERE user_id = 12345
```

5.4 OR Conditions Across Different Columns

```
-- May cause full scan if both columns are not individually indexed:
```

```
WHERE first_name = 'John' OR last_name = 'Smith'

-- Solution: separate indexes on both columns, or rewrite as UNION:
SELECT * FROM users WHERE first_name = 'John'
UNION
SELECT * FROM users WHERE last_name = 'Smith'
```

5.5 Low-Cardinality Columns

The query planner may choose to skip an index if the indexed column has very few distinct values (low cardinality). A column like `'is_active'` with only two values (true/false) — where half the table might match — makes an index pointless because a full scan is not much worse and avoids the index overhead.

6. Query Optimization Techniques

6.1 Use EXPLAIN to Read Query Plans

EXPLAIN shows the query execution plan — how the database intends to execute your query. It reveals whether indexes are being used, how many rows are estimated to be examined, and where the bottlenecks are.

```
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'john@example.com';

-- Good output (index used):
Index Scan using idx_users_email on users
  Index Cond: (email = 'john@example.com')
  Rows: 1   Cost: 0.43..8.45

-- Bad output (full scan):
Seq Scan on users
  Filter: (email = 'john@example.com')
  Rows: 5000000   Cost: 0.00..102450.00
```

EXPLAIN ANALYZE actually executes the query and shows real row counts alongside estimates. Always run EXPLAIN on slow queries before adding indexes — the plan often reveals the cause immediately.

6.2 Avoid SELECT *

Selecting all columns forces the database to fetch and transmit every column for every matched row, even if your application only uses two of them. This wastes I/O, memory, and network bandwidth.

```
-- Bad:  fetches 40 columns, transfers megabytes
SELECT * FROM orders WHERE user_id = 123;

-- Good: fetches only what is needed
SELECT id, total_price, status, created_at FROM orders WHERE user_id = 123;
```

Specifying exact columns also enables covering index optimisations — if the index includes all requested columns, the database can return results without fetching the full row.

6.3 Use LIMIT for Large Result Sets

Without a LIMIT clause, a query that matches thousands of rows fetches and transfers all of them even if the application only displays the first 20. Always paginate large result sets.

```
SELECT id, name, email FROM users ORDER BY created_at DESC LIMIT 20 OFFSET 0;
```

For very deep pagination (OFFSET 100000), OFFSET-based pagination becomes slow because the database must scan and discard all preceding rows. Use cursor-based pagination instead — store the last seen ID and filter by it:

```
-- Cursor-based pagination (fast even at deep pages):
WHERE created_at < :last_seen_cursor ORDER BY created_at DESC LIMIT 20
```

6.4 Optimise JOINS

- Always index the columns used in JOIN conditions — unindexed join columns cause nested loop scans
- Join order matters — start with the smallest result set and join to larger tables
- Avoid joining on computed expressions or functions — they prevent index use

```
-- Ensure these columns are indexed:
SELECT u.name, o.total FROM users u JOIN orders o ON u.id = o.user_id;
-- Index needed: orders(user_id)
```

6.5 Use Connection Pooling

Opening a new database connection for every request is expensive — it involves a TCP handshake, authentication, and memory allocation on the database server. Connection pooling maintains a pool of open connections that are reused across requests, dramatically reducing connection overhead at scale.

- PgBouncer — the standard connection pooler for PostgreSQL
- Set pool size to approximately 2-4x the number of database CPU cores
- High connection counts without pooling are a common cause of database performance degradation under load

6.6 Avoid N+1 Queries

The N+1 problem occurs when fetching a list of N records and then issuing a separate query for each one. The result is N+1 database round-trips instead of 1 or 2.

```
-- Bad: 1 query for users + N queries for their orders = N+1 total
const users = await User.findAll();
for (const user of users) {
  user.orders = await Order.findAll({ where: { userId: user.id } });
}

-- Good: 2 queries total using eager loading or a JOIN
const users = await User.findAll({ include: [Order] });
```

7. Indexing in NoSQL Databases

7.1 MongoDB

MongoDB supports rich indexing similar to relational databases. Without indexes, every query triggers a collection scan — equivalent to a full table scan. Unlike SQL databases, MongoDB does not automatically create indexes on fields other than `_id`.

```
// Create a single field index:
db.users.createIndex({ email: 1 }); // 1 = ascending

// Compound index (note: left-prefix rule applies):
db.orders.createIndex({ userId: 1, createdAt: -1 });

// Text index for full-text search:
db.articles.createIndex({ body: 'text' });

// Explain a query:
db.users.find({ email: 'john@example.com' }).explain('executionStats');
```

MongoDB's explain output shows `winningPlan` — if it shows `COLLSCAN` (collection scan), you need an index. If it shows `IXSCAN`, the index is being used correctly.

7.2 Apache Cassandra

Cassandra's indexing model is fundamentally different from relational databases and MongoDB. Because Cassandra is designed for distributed, high-throughput writes, secondary indexes (indexes on non-primary-key columns) are deliberately limited and often discouraged in production.

The correct approach in Cassandra is to design your tables around your query patterns — a concept called query-driven data modelling. Instead of creating one table and indexing different columns, you create multiple tables each optimised for a specific query pattern.

```
-- Instead of one table with secondary indexes:
CREATE TABLE orders (id UUID PRIMARY KEY, user_id UUID, status TEXT, ...);
CREATE INDEX ON orders(user_id);    -- avoid this in Cassandra

-- Create separate tables for each access pattern:
CREATE TABLE orders_by_user (user_id UUID, order_id UUID, ..., PRIMARY KEY
(user_id, order_id));
CREATE TABLE orders_by_status (status TEXT, order_id UUID, ..., PRIMARY KEY
(status, order_id));
```

- Secondary indexes in Cassandra perform poorly at scale because they require cross-node coordination
- Materialized views are an alternative but have significant write-time overhead
- Prefer denormalisation and multiple tables over secondary indexes

8. Trade-offs of Indexing

Aspect	Benefit	Cost
Read performance	Dramatically faster SELECT queries	None — reads always benefit
Write performance	No benefit	Slower INSERT / UPDATE / DELETE — index must be updated
Storage	No benefit	Each index consumes additional disk space
Maintenance	No benefit	Indexes must be rebuilt after large bulk loads
Query planning	Planner can choose optimal path	Too many indexes slow down query planning itself

9. When to Add and When to Avoid Indexes

Add an index when:

- The column appears frequently in WHERE clauses, JOIN conditions, or ORDER BY clauses
- The table is large (thousands of rows or more) and the query is on a hot path
- The column has high cardinality — many distinct values (email, user_id, order_id)
- You can verify with EXPLAIN that the query is performing a full table scan

Avoid or remove indexes when:

- The table is small — the overhead of maintaining the index exceeds the lookup benefit
- The column has very low cardinality (e.g. boolean flags, status with 2-3 values)
- The column is written to far more often than it is read
- The index is not being used — confirmed by EXPLAIN and query monitoring
- You have too many indexes already — each index adds write overhead and query planner complexity

Rule of thumb: Index the columns your queries actually use. Remove indexes your queries never use. Measure with EXPLAIN before and after — never guess.

Interview Questions & Answers

Q1: What is a database index and how does it improve query performance?

- An index is a separate data structure (typically a B-Tree) that maps column values to the rows containing them
- Without an index, a query must scan every row — $O(n)$ complexity
- With an index, the database performs a binary search on the sorted structure — $O(\log n)$ complexity
- For a table of 5 million rows, an index reduces comparisons from 5,000,000 to approximately 23

Q2: How does a B-Tree index work?

- A B-Tree is a self-balancing tree where all leaf nodes are at the same depth

- Internal nodes store sorted keys and pointers to child subtrees
- Leaf nodes store the indexed values and pointers to the actual table rows
- Lookup traverses from root to leaf, halving the search space at each level — guaranteed $O(\log n)$
- Leaf nodes are linked together, enabling efficient range scans without returning to the root

Q3: What is the left-prefix rule for composite indexes?

- A composite index on (A, B, C) can be used for queries on A, A+B, or A+B+C
- It cannot be used for queries on B alone, C alone, or B+C (without A)
- The database needs the leftmost column to know where to start in the sorted index structure
- Design composite indexes with the most frequently filtered or most selective column first

Q4: Why does LIKE '%keyword%' not use an index?

- A B-Tree index is sorted from left to right by the full column value
- A leading wildcard means the database does not know which part of the sorted structure to start searching from
- It must check every index entry — equivalent to a full scan
- Solution: trailing wildcard only (LIKE 'keyword%') uses the index; use full-text indexes for substring search

Q5: What is the N+1 query problem and how do you fix it?

- N+1 occurs when fetching N records results in N additional queries — 1 query for the list and 1 per item
- Example: fetch 100 users then query each user's orders separately — 101 database round-trips
- Fix with eager loading: fetch users and their orders in 2 queries using a JOIN or ORM include
- ORMs (Sequelize, TypeORM, ActiveRecord) can silently introduce N+1 problems — always check the queries they generate

Q6: What is EXPLAIN and how do you use it?

- EXPLAIN shows the query execution plan — how the database will execute your query
- EXPLAIN ANALYZE actually runs the query and shows real row counts alongside estimates
- Key things to look for: Seq Scan (bad — full table scan) vs Index Scan (good)
- Cost estimates help compare alternative query formulations

- Always run EXPLAIN before adding an index — it may reveal a different root cause

Q7: What is a covering index?

- A covering index contains all columns required by a query, eliminating the need to fetch the actual table row
- When a query can be satisfied entirely from the index, the database performs an index-only scan
- Index-only scans are significantly faster than index scans because they avoid random I/O to the heap
- Created using INCLUDE in PostgreSQL: `CREATE INDEX idx ON users(email) INCLUDE (id, name)`

Q8: What are the trade-offs of adding too many indexes?

- Every index increases write latency — INSERT, UPDATE, and DELETE must update all relevant indexes
- Indexes consume disk space — large tables with many indexes can have indexes larger than the table itself
- Too many indexes slow down query planning — the planner must evaluate more options
- Unused indexes are pure overhead — they consume resources without providing any read benefit
- Rule: add indexes deliberately based on measured query performance, not speculatively

Q9: How is indexing different in Cassandra compared to SQL databases?

- Cassandra is designed around query-driven data modelling — you design tables for specific queries rather than adding indexes
- Secondary indexes in Cassandra require cross-node coordination and perform poorly at scale
- The correct pattern is to create separate tables for each access pattern (`orders_by_user`, `orders_by_date`)
- This is a fundamental design difference — normalisation and flexible indexing are SQL concepts; Cassandra favours denormalisation

Q10: What is a partial index and when would you use one?

- A partial index covers only a subset of rows, defined by a WHERE condition in the index definition

- Smaller and faster than a full index — only the filtered rows are stored in the index
- Ideal when queries consistently target a specific subset: active users, pending orders, unread notifications
- Example: `CREATE INDEX idx_pending ON orders(user_id) WHERE status = 'pending'` — only indexes pending orders

Final Takeaways

Mental Model: An index is a shortcut to data. Without it, the database reads everything. With it, the database jumps directly. The cost of that shortcut is slower writes and extra storage — index only when the read benefit outweighs those costs.

- Indexing is the single highest-impact optimisation for read-heavy database workloads — a well-placed index can reduce query time from seconds to milliseconds
- B-Tree indexes support equality, range, and sort queries; Hash indexes support equality only; Full-Text indexes support keyword search
- The left-prefix rule governs composite indexes — always put the most selective or most queried column first
- Functions and leading wildcards on indexed columns prevent index use — use expression indexes or full-text indexes as alternatives
- EXPLAIN is your primary diagnostic tool — read query plans before and after every index change
- In NoSQL: MongoDB supports flexible indexing similar to SQL; Cassandra requires query-driven table design instead of secondary indexes
- Never add indexes speculatively — measure first, index what is actually slow, and remove indexes that are never used

CHAPTER 13

GraphQL — Modern API Design

Schemas, resolvers, federation, and when to use it over REST

1. What Is GraphQL?

GraphQL is a query language for APIs and a runtime for executing those queries, developed by Facebook in 2012 and open-sourced in 2015. Unlike REST, where the server defines fixed endpoints that return fixed data shapes, GraphQL lets the client specify exactly what data it needs — nothing more, nothing less. The server exposes a

single endpoint and a strongly-typed schema describing all available data and operations.

Facebook built GraphQL to solve a real problem they were facing: their mobile apps were over-fetching enormous amounts of data from REST APIs designed for the web, causing slow load times on slower mobile networks. They needed a way for clients to request precisely the fields they needed, and for the server to fulfil those requests efficiently. GraphQL was that solution.

Core idea: In REST, the server decides what data each endpoint returns. In GraphQL, the client decides what data it receives. This single shift solves over-fetching, under-fetching, and the need for multiple endpoints.

2. GraphQL vs REST — The Key Differences

2.1 The Over-fetching Problem in REST

In REST, endpoints return a fixed data shape regardless of what the client actually needs. A GET /users/:id endpoint might return 30 fields — name, email, phone, address, preferences, billing info — when the mobile app only needs name and avatar. Those extra 28 fields waste bandwidth and slow down the response, especially on mobile connections.

```
REST — GET /users/42
{
  id: 42, name: 'John Smith', email: 'john@example.com',
  phone: '+44 7700 900123', address: { ... },
  preferences: { ... }, billing: { ... },    // not needed!
  createdAt: '2023-01-15', updatedAt: '2024-03-01' // not needed!
}
```

GraphQL — client requests exactly what it needs:

```
query { user(id: 42) { name avatar } }
```

Response:

```
{ data: { user: { name: 'John Smith', avatar: 'https://...' } } }
```

2.2 The Under-fetching Problem in REST

Under-fetching occurs when one API call does not return enough data and the client must make multiple sequential requests to assemble the full picture it needs. Displaying

a user's profile with their recent orders and each order's items might require 3 separate REST calls, each waiting for the previous one to complete.

REST – 3 sequential round-trips:

```
GET /users/42          → user data
GET /users/42/orders   → order list
GET /orders/7/items    → items for order 7
```

GraphQL – 1 request, exactly the data needed:

```
query {
  user(id: 42) {
    name
    orders(last: 5) {
      id total status
      items { name quantity price }
    }
  }
}
```

2.3 Side-by-Side Comparison

Aspect	REST	GraphQL
Endpoints	Multiple (one per resource)	Single /graphql endpoint
Data fetching	Fixed response shape	Client specifies exact fields
Over-fetching	Common problem	Eliminated by design
Under-fetching	Requires multiple calls	Resolved in one query
Versioning	v1, v2, v3 URLs	Schema evolution, no versioning
Type system	Optional (OpenAPI)	Built-in, strongly typed
Real-time	Requires polling or SSE	Native subscriptions
Caching	Simple (HTTP caching)	Complex (per-field)
Learning curve	Low	Moderate to high
Tooling	Mature	Excellent (GraphiQL, Apollo)

3. Core Concepts

3.1 Schema — The Contract

The schema is the heart of every GraphQL API. It is a strongly-typed contract between the client and the server, written in the GraphQL Schema Definition Language (SDL). The schema defines every type of data available, every field on each type, and every operation the API supports. Clients can only request what the schema explicitly declares.

```
type User {  
  id:      ID!  
  name:    String!  
  email:   String!  
  role:    Role!  
  orders:  [Order!]!  
  createdAt: String!  
}  
  
type Order {  
  id:      ID!  
  total:   Float!  
  status:  OrderStatus!  
  items:   [OrderItem!]!  
}  
  
enum OrderStatus { PENDING PROCESSING SHIPPED DELIVERED CANCELLED }  
enum Role          { ADMIN  USER  MANAGER }
```

The exclamation mark (!) denotes a non-nullable field — the server guarantees it will never return null for that field. Without it, the field may be null. This type information is available to clients at development time, enabling powerful IDE auto-completion and compile-time validation of queries.

3.2 Queries — Reading Data

A query is a read operation in GraphQL. The client specifies the exact fields it wants and the server returns precisely that shape. Multiple resources can be fetched in a single query regardless of how many different data sources they come from on the server.

```
# Fetch a user with their last 3 orders and each order's items  
query GetUserDashboard($userId: ID!) {  
  user(id: $userId) {  
    name  
    email  
    orders(last: 3) {  
      id  
      total  
      status  
    }  
  }  
}
```

```
    items {
      name
      quantity
      price
    }
  }
}
```

3.3 Mutations — Writing Data

Mutations are the GraphQL equivalent of POST, PUT, PATCH, and DELETE in REST. They modify server-side data and return the updated result. Like queries, mutations are strongly typed and the client specifies exactly which fields to return after the mutation completes.

```
mutation CreateOrder($input: CreateOrderInput!) {
  createOrder(input: $input) {
    id
    status
    total
    createdAt
  }
}

# Variables sent alongside:
{ input: { userId: '42', items: [{ productId: '7', qty: 2 }] } }
```

3.4 Subscriptions — Real-Time Data

Subscriptions allow clients to receive real-time updates when data changes on the server. Unlike queries and mutations which are single request-response cycles, subscriptions maintain a persistent connection (typically over WebSockets) and the server pushes updates to the client whenever a specified event occurs.

```
subscription OnOrderStatusChanged($orderId: ID!) {
  orderStatusChanged(orderId: $orderId) {
    id
    status
    updatedAt
  }
}

# Server pushes a message every time the order status changes
# Client receives: { data: { orderStatusChanged: { id: '99', status: 'SHIPPED' } } }
```

Subscriptions are ideal for live dashboards, chat applications, order tracking, stock price feeds, and any feature where users need to see changes without polling.

3.5 Fragments — Reusable Field Sets

Fragments allow you to define a reusable set of fields that can be included in multiple queries. This avoids repetition and keeps queries maintainable as the schema grows.

```
fragment UserBasics on User {  
  id name email avatar  
}  
  
query { user(id: 42) { ...UserBasics orders { id total } } }  
query { me { ...UserBasics role createdAt } }
```

4. Resolvers — How GraphQL Fetches Data

A resolver is a function on the server that knows how to fetch the data for a specific field in the schema. Every field in a GraphQL schema has a corresponding resolver. When a query arrives, the GraphQL runtime calls each resolver for each requested field, assembles the results into the shape the client requested, and returns the final response.

```
// Schema definition:  
// type Query { user(id: ID!): User }  
// type User { id: ID!, name: String!, orders: [Order!]! }  
  
const resolvers = {  
  Query: {  
    // Root resolver — fetches the user by ID  
    user: async (parent, { id }, context) => {  
      return await context.db.users.findById(id);  
    },  
  },  
  User: {  
    // Field resolver — fetches orders for the resolved user  
    orders: async (user, args, context) => {  
      return await context.db.orders.findByUserId(user.id);  
    },  
  },  
  Mutation: {  
    createOrder: async (parent, { input }, context) => {  
      return await context.db.orders.create(input);  
    },  
  },  
}
```

```
};
```

Resolvers receive four arguments: parent (the resolved value of the parent field), args (the arguments passed to this field in the query), context (shared data like database connections and the authenticated user), and info (metadata about the query execution). The context object is the standard way to pass authentication information and shared services to all resolvers.

5. The N+1 Problem in GraphQL and DataLoader

The N+1 problem is one of the most significant performance challenges in GraphQL. Because resolvers are called independently for each field on each object, fetching a list of users and their orders will trigger one query for the users and then one separate query per user for their orders — N+1 database queries total.

```
query {  
  users {  
    name  
    orders {  
      id total  
    }  
  }  
}
```

1 query: SELECT * FROM users
N queries: SELECT * FROM orders WHERE user_id = ?
One per user — if 100 users, 101 queries total

5.1 DataLoader — Batching and Caching

DataLoader is a utility library (originally developed by Facebook, now part of the GraphQL ecosystem) that solves the N+1 problem by batching multiple individual load requests into a single batched query. Instead of firing a database query for each user's orders immediately, DataLoader collects all the order requests that arrive within a single event loop tick and issues one batched query for all of them.

```
const DataLoader = require('dataloader');  
  
// Instead of: SELECT * FROM orders WHERE user_id = 1  
//             SELECT * FROM orders WHERE user_id = 2   ... x100  
//  
// DataLoader sends: SELECT * FROM orders WHERE user_id IN (1, 2, 3, ... 100)  
  
const orderLoader = new DataLoader(async (userIds) => {  
  const orders = await db.orders.findByUserIds(userIds);  
  // Group by userId so DataLoader can map results back  
  return userIds.map(id => orders.filter(o => o.userId === id));  
});
```

```
});  
  
// In resolver — looks like a single load, batched automatically:  
User: {  
  orders: (user) => orderLoader.load(user.id)  
}
```

DataLoader also caches results within a single request — if the same user ID is requested multiple times in the same query (through different paths), DataLoader returns the cached result without hitting the database again. This combination of batching and per-request caching makes DataLoader essential for any production GraphQL server.

6. GraphQL in Microservices — Schema Stitching & Federation

In a microservices architecture, different services own different parts of the data model. The User Service owns user data, the Order Service owns order data, and the Product Service owns product data. Exposing all of this through a single GraphQL API requires composing schemas across service boundaries.

6.1 Schema Stitching (Older Approach)

Schema stitching manually combines multiple GraphQL schemas into one unified schema at a gateway layer. A GraphQL gateway fetches the sub-schemas from each service and merges them. When a query spans multiple services, the gateway calls each service's API and assembles the combined result.

```
// Gateway merges schemas from multiple services:  
User Service → { type User { id name email } }  
Order Service → { type Order { id total userId } }  
  
// Client queries the gateway as if it were a single API:  
query { user(id: 1) { name orders { id total } } }
```

6.2 Apollo Federation (Modern Approach)

Apollo Federation is the industry-standard approach for composing GraphQL schemas across microservices. Instead of a central gateway manually stitching schemas, each service publishes its own subgraph schema that declares which types it owns and which fields from other services' types it extends. The Apollo Router composes these into a unified supergraph automatically.

```
// User Service subgraph:
```

```
type User @key(fields: 'id') {
  id: ID!
  name: String!
  email: String!
}

// Order Service subgraph — extends User with orders field:
type User @key(fields: 'id') @extends {
  id: ID! @external
  orders: [Order!]!
}

type Order {
  id: ID!
  total: Float!
  status: String!
}
```

Federation enables true separation of concerns — each team owns their service's schema independently. The Apollo Router handles query planning across subgraphs, figuring out which services to call and in what order to fulfil a client's query.

7. Authentication & Authorization in GraphQL

7.1 Authentication

Authentication in GraphQL works the same way as in REST — validate a JWT or session token from the request header and attach the authenticated user to the context object. Every resolver then has access to the authenticated user via context.

```
const server = new ApolloServer({
  typeDefs,
  resolvers,
  context: ({ req }) => {
    const token = req.headers.authorization?.split('Bearer ')[1];
    const user = token ? verifyJWT(token) : null;
    return { user, db }; // available in all resolvers
  },
});
```

7.2 Authorization — Field-Level Control

GraphQL's field-level resolution model enables fine-grained authorization — different fields on the same type can have different access rules. A User type might expose name and avatar publicly but require admin role to access email and billingInfo.

```
User: {  
  // Anyone can read name  
  name: (user) => user.name,  
  
  // Only the user themselves or an admin can read email  
  email: (user, _, { currentUser }) => {  
    if (currentUser?.id !== user.id && currentUser?.role !== 'ADMIN') {  
      throw new GraphQLError('Forbidden', { extensions: { code: 'FORBIDDEN' } })  
    }  
    return user.email;  
  },  
},
```

For complex authorization logic, the graphql-shield library provides a middleware layer that declaratively defines permission rules per type and field — keeping authorization logic separate from business logic in resolvers.

8. Performance Considerations

8.1 Query Depth and Complexity Limits

Because GraphQL lets clients construct arbitrarily complex queries, a malicious or poorly-written client could send a deeply nested query that triggers thousands of database calls. Protecting against this requires depth and complexity limits.

```
// A malicious deeply-nested query:  
{ user { friends { friends { friends { friends { name } } } } } }  
  
// Protection: reject queries exceeding depth or complexity thresholds  
const server = new ApolloServer({  
  validationRules: [  
    depthLimit(5), // max 5 levels of nesting  
    createComplexityRule({ maximumComplexity: 1000 })  
  ]  
});
```

8.2 Persisted Queries

In production, clients can register known query strings with the server ahead of time and then send only a hash ID rather than the full query text. This reduces request payload size, prevents clients from executing arbitrary queries, and enables server-side whitelisting.

```
// Client sends hash instead of full query:
POST /graphql
{ extensions: { persistedQuery: { sha256Hash: 'abc123...' } } }

// Server looks up hash → executes registered query
```

8.3 HTTP Caching Challenges

REST's HTTP caching model does not apply well to GraphQL because all requests go to the same POST /graphql endpoint. Two requests for completely different data are indistinguishable at the HTTP cache layer.

- Apollo Client caches query results in a normalised in-memory store on the client side
- Persisted queries with GET requests allow CDN-level caching of specific query results
- Automatic Persisted Queries (APQ) enables GET-based caching for queries that have been registered
- Field-level cache hints in the schema tell Apollo Server how long individual fields can be cached

8.4 Rate Limiting

Because GraphQL allows clients to compose complex queries, traditional request-count-based rate limiting is insufficient — one GraphQL request can be far more expensive than another. Rate limiting should be based on query complexity score rather than request count.

- Assign a cost to each field (simple fields: 1 point, list fields: 10 points, nested lists: multiplied)
- Reject requests whose total complexity exceeds the per-client allowance
- Track and rate-limit by IP, API key, or authenticated user ID

9. When to Use GraphQL vs REST

9.1 Choose GraphQL when:

- Multiple client types (web, iOS, Android, smart TV) each need different subsets of the same data

- The data model is complex and highly interconnected — users, orders, products, reviews all relating to each other
- Rapid frontend iteration is a priority — schema changes are additive, no new endpoints needed
- Real-time features are required — subscriptions provide native WebSocket support
- You want a single, self-documenting API that teams across the organisation can explore via GraphQL or GraphQL Playground

9.2 Choose REST when:

- The API is simple and CRUD-oriented with clear resource boundaries
- HTTP caching is critical — REST endpoints benefit from CDN and browser caching out of the box
- The API is public and consumed by third-party developers — REST is more universally understood
- The team is small and the overhead of a GraphQL schema and resolver layer is not justified
- File uploads are central — REST handles multipart form data more naturally than GraphQL

Practical rule: GraphQL shines when clients have diverse data needs and the data model is richly interconnected. REST is simpler, cacheable, and better understood — prefer it for straightforward APIs and public-facing integrations.

Interview Questions & Answers

Q1: What is GraphQL and how does it differ from REST?

- GraphQL is a query language for APIs where clients specify exactly what data they need; REST has fixed endpoints that return fixed data shapes
- REST: multiple endpoints (/users, /orders, /products); GraphQL: single /graphql endpoint
- REST suffers from over-fetching and under-fetching; GraphQL eliminates both by letting the client define the response shape
- GraphQL has a strongly-typed schema that serves as a contract; REST schemas are optional (OpenAPI)

Q2: What is the N+1 problem in GraphQL and how does DataLoader solve it?

- N+1 occurs when resolving a list of N items triggers N additional queries — one per item for a related field
- Example: 100 users returned → 100 separate order queries = 101 database calls
- DataLoader batches all individual load() calls within a single event loop tick into one batched query
- It also caches results within the request — requesting the same ID twice returns the cached result

Q3: What are GraphQL resolvers?

- A resolver is a server-side function responsible for fetching the data for a specific field in the schema
- Every field in a GraphQL schema has a corresponding resolver function
- Resolvers receive four arguments: parent (resolved parent object), args (field arguments), context (shared state like DB and auth), and info (query metadata)
- The GraphQL runtime calls resolvers bottom-up, assembling results into the shape the client requested

Q4: What is Apollo Federation and why is it used?

- Apollo Federation is a specification for composing multiple GraphQL subgraph schemas into a single unified supergraph
- Each microservice owns and publishes its own subgraph schema independently
- Services can extend types owned by other services — Order Service adds an orders field to User without touching the User Service
- The Apollo Router handles cross-service query planning and execution automatically

Q5: How do you handle authentication in GraphQL?

- Validate the JWT or session token from the Authorization header in the server context function
- Attach the authenticated user object to the context — available in all resolvers
- Each resolver checks context.user to determine what the authenticated user is permitted to access
- Field-level authorization allows different fields on the same type to have different access rules

Q6: What is a GraphQL subscription?

- A subscription is a real-time operation where the server pushes data to the client when an event occurs
- Maintained over a persistent WebSocket connection rather than a single request-response
- Ideal for live dashboards, chat, notifications, order tracking, and stock price feeds

- The client subscribes to a named event; the server publishes to that event whenever the underlying data changes

Q7: Why is HTTP caching harder with GraphQL than REST?

- All GraphQL requests go to the same POST /graphql endpoint — the cache cannot distinguish between different queries
- REST endpoints have unique URLs that map to unique cache keys; GraphQL does not
- Solutions: Apollo Client's normalised in-memory cache on the client, persisted queries enabling GET requests, and field-level cache hints
- For public APIs, persisted queries with GET requests allow CDN-level caching of registered queries

Q8: What are fragments in GraphQL?

- Fragments are reusable field sets that can be shared across multiple queries and mutations
- They reduce duplication when the same set of fields is requested in multiple places
- Inline fragments allow conditional field selection based on type (useful with union types and interfaces)
- Named fragments are defined once and spread into any query using the ... syntax

Q9: How do you protect against malicious queries in GraphQL?

- Depth limiting — reject queries that nest beyond a maximum depth (e.g. 5 levels)
- Complexity limits — assign a cost to each field and reject queries exceeding a total complexity budget
- Persisted queries — whitelist known safe queries; reject arbitrary query strings in production
- Rate limiting based on complexity score rather than request count — one expensive query counts more than one cheap query

Q10: When would you NOT use GraphQL?

- Simple CRUD APIs where REST endpoints map cleanly to resources — GraphQL adds unnecessary complexity
- When HTTP caching is critical — REST benefits from CDN and browser caching out of the box
- Public third-party APIs — REST is more universally understood and documented
- When the team is unfamiliar with GraphQL — the learning curve (schema design, resolver patterns, DataLoader) has real productivity costs

Final Takeaways

Mental Model: REST is a menu with fixed dishes. GraphQL is a buffet where you pick exactly what you want. The buffet is more flexible, but it requires more infrastructure to run well.

- GraphQL eliminates over-fetching and under-fetching — clients receive exactly the data they request in a single network call
- The schema is the contract — strongly typed, self-documenting, and the foundation of every GraphQL API
- Resolvers are functions that fetch data field by field — design them to be small, focused, and composable
- DataLoader is not optional in production — without it, the N+1 problem will degrade performance under any real load
- Apollo Federation is the standard for GraphQL in microservices — each service owns its schema, the router composes the supergraph
- GraphQL does not replace REST — it solves specific problems (complex data requirements, multiple client types) and introduces its own challenges (caching, query complexity)
- Always protect production GraphQL APIs with depth limits, complexity limits, and persisted queries

CHAPTER 14

Fault Tolerance & Reliability

Retries, circuit breakers, redundancy, and building systems that survive failure

1. What Is Fault Tolerance and Why It Matters

Fault tolerance is the ability of a system to continue operating correctly when one or more of its components fail. In distributed systems, failures are not edge cases — they are normal, expected events. Hardware fails, networks partition, services crash, third-party APIs time out. A fault-tolerant system is designed with these realities built in from the start.

Reliability is usually expressed as availability — the percentage of time a system is operational. The difference between availability levels translates to vastly different amounts of downtime:

```
99%      availability → 87.6  hours downtime per year  (3.65 days)
99.9%    availability → 8.76  hours downtime per year
```

```
99.99% availability → 52.6 minutes downtime per year
99.999% availability → 5.26 minutes downtime per year (five nines)
```

Core principle: Design for failure, not against it. The question is not whether components will fail — they will. The question is whether the system degrades gracefully or collapses entirely.

2. Failure Modes in Distributed Systems

2.1 Crash Failures

A component stops responding entirely. This is the simplest failure to detect via health checks and handle via redundancy. Load balancers automatically stop routing to crashed instances.

2.2 Timing Failures

A component responds, but too slowly. Without timeout handling, a slow dependency holds connections and threads open, eventually exhausting shared resources. This is how cascading failures begin — a slow database takes down all app servers.

```
Slow DB → App servers hold connections → Pool exhausted
          → New requests timeout → Queue grows → System-wide outage
```

2.3 Omission Failures

A component is running but intermittently fails to send or receive messages — dropped packets, missed queue messages, silent failures. Harder to detect than crashes because the component appears healthy.

2.4 Byzantine Failures

A component is running but producing incorrect results — corrupted data, wrong computations. Defending against Byzantine failures requires consensus protocols (Raft, Paxos) and is primarily relevant in blockchain and safety-critical infrastructure.

3. Retry Mechanisms

When an operation fails, retry it — but only for transient failures that are likely to resolve on their own. Never retry permanent failures like 400 Bad Request or 404 Not Found — they will not succeed regardless of how many times you try.

Retry for:

- Network timeouts, 5xx server errors, 429 rate limit errors, connection refused

Never retry:

- 4xx client errors (except 429), business logic failures, non-idempotent operations without idempotency keys

3.1 Retry Strategies

Immediate retry — retry instantly. Rarely correct; usually hammers a struggling service harder.

Fixed delay — wait a constant interval. Better, but can cause thundering herd when all clients retry simultaneously.

Exponential backoff — double the wait time after each failure. The gold standard.

```
Attempt 1: FAIL → wait 1s
Attempt 2: FAIL → wait 2s
Attempt 3: FAIL → wait 4s
Attempt 4: FAIL → wait 8s → give up
```

Exponential backoff with jitter — add random delay to prevent synchronised retries from all clients at once.

```
wait = min(cap, base * 2^attempt) + random(0, 1000ms)
```

3.2 Retry Implementation (Node.js)

```
async function withRetry(fn, { maxAttempts = 3, baseDelay = 1000 } = {}) {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try { return await fn(); }
    catch (err) {
      const isLast = attempt === maxAttempts;
      const retryable = err.status >= 500 || err.code === 'ETIMEDOUT';
      if (isLast || !retryable) throw err;
      const delay = baseDelay * 2 ** (attempt - 1) + Math.random() * 1000;
      await new Promise(r => setTimeout(r, delay));
    }
  }
}
```

```
}
```

3.3 Idempotency Keys — Safe Retries for Mutations

Retrying write operations (payments, order creation) without idempotency keys risks duplicate effects. An idempotency key is a unique identifier the server uses to detect and deduplicate repeated requests.

```
await fetch('/api/payments', {
  method: 'POST',
  headers: { 'Idempotency-Key': 'order-99-attempt-1' },
  body: JSON.stringify({ amount: 5000 })
}); // safe to retry — server deduplicates by key
```

4. Circuit Breaker Pattern

The circuit breaker prevents a system from repeatedly calling a failing service, giving it time to recover without being overwhelmed. Without a circuit breaker, every request waits for the full timeout before failing — 1,000 req/s with a 30s timeout means 30,000 waiting requests in one minute, exhausting all threads and memory.

4.1 Three States

CLOSED (normal): requests pass through; failures are counted.

OPEN (failure detected): all requests fail immediately with no downstream call; after a cooldown, transitions to HALF-OPEN.

HALF-OPEN (testing recovery): a few test requests are allowed through; success closes the circuit, failure reopens it.

```
CLOSED —(failure rate > threshold)—→ OPEN
OPEN   —(cooldown expires)————→ HALF-OPEN
HALF-OPEN —(success)————→ CLOSED
HALF-OPEN —(failure)————→ OPEN
```

4.2 Implementation (Node.js)

```
class CircuitBreaker {
  constructor(fn, { failureThreshold=5, cooldown=30000 }={}) {
    this.fn = fn; this.failureThreshold = failureThreshold;
    this.cooldown = cooldown; this.state = 'CLOSED';
    this.failureCount = 0; this.nextAttempt = Date.now();
  }
  async call(...args) {
```

```
if (this.state === 'OPEN') {
  if (Date.now() < this.nextAttempt)
    throw new Error('Circuit OPEN - service unavailable');
  this.state = 'HALF-OPEN';
}
try {
  const result = await this.fn(...args);
  this.failureCount = 0; this.state = 'CLOSED';
  return result;
} catch (err) {
  this.failureCount++;
  if (this.failureCount >= this.failureThreshold) {
    this.state = 'OPEN';
    this.nextAttempt = Date.now() + this.cooldown;
  }
  throw err;
}
}
```

4.3 Fallback Strategies

When the circuit is open, return a degraded but useful response rather than an error where possible.

- **Cached response** — return the last known good value from Redis or in-memory cache
- **Default value** — return a safe default (empty list, placeholder, zero count)
- **Alternative service** — route to a backup or simplified implementation

```
async function getRecommendations(userId) {
  try { return await breaker.call(() => recService.get(userId)); }
  catch { return cache.get('popular-items') ?? DEFAULT_RECOMMENDATIONS; }
}
```

5. Timeout Patterns

Every call to an external dependency must have a timeout. Without timeouts, a hanging dependency holds threads and connections open indefinitely — eventually exhausting all available capacity and causing a cascading failure.

```
// HTTP:      axios.get(url, { timeout: 5000 })
// Database:  new Pool({ connectionTimeoutMillis: 3000, query_timeout: 10000 })
// Redis:     createClient({ socket: { connectTimeout: 2000 } })
```

Set timeouts based on measured P99 response times, not guesswork. A query that normally takes 20ms should timeout at 100-200ms — not 30 seconds. Outer timeouts must be longer than inner timeouts to allow retries to complete.

```
Client timeout:    30s
API Gateway:      25s
  Order Service: 20s  (3 retries × 5s + backoff)
  Payment API:   5s   (per attempt)
```

6. Bulkhead Pattern — Isolating Failures

Named after watertight ship compartments — if one floods, others stay sealed. The bulkhead pattern isolates resource pools so one failing dependency cannot exhaust shared resources and bring down unrelated features.

```
Without bulkheads:
  Payment (slow) → consumes all 100 shared threads
  Auth, Orders, Search → starved, failing

With bulkheads — isolated thread pools:
  Payment: 20 threads  Auth: 15 threads  Orders: 15 threads  Search: 10 threads
  Payment hangs → only Payment pool affected; all others continue normally
```

7. Redundancy — Eliminating Single Points of Failure

Redundancy means having more capacity than the minimum required so that when one component fails, others absorb the load without downtime. Every non-redundant component is a single point of failure.

7.1 Active-Active

All instances serve traffic simultaneously. Failover is instant — the load balancer stops routing to failed instances. Requires stateless application design.

```
Server A ┘
Server B ┣ Load Balancer — Users  (Server B fails → A and C continue)
Server C ┙
```

7.2 Active-Passive

One instance serves all traffic; others stand by. Failover requires promoting a standby — takes seconds to minutes. Common for databases with primary-replica setups.

```
Server A (ACTIVE) — Users      Server A fails
```

Server B (STANDBY)

Server B promoted → Users

7.3 Geographic Redundancy

- Distributes systems across multiple regions — protects against data centre-level failures
- Active-active multi-region: both regions serve traffic with latency-based routing
- Active-passive multi-region: one primary region, others are disaster recovery standbys
- Data replication between regions introduces latency and CAP theorem trade-offs

8. Health Checks and Self-Healing

8.1 Liveness vs Readiness

Liveness — is the process alive? Failure: restart the container.

```
app.get('/health/live', (req, res) => res.json({ status: 'ok' }));
```

Readiness — is the app connected to dependencies and ready to serve requests? Failure: remove from load balancer (do not restart).

```
app.get('/health/ready', async (req, res) => {
  try {
    await db.query('SELECT 1'); await redis.ping();
    res.json({ status: 'ready' });
  } catch (err) {
    res.status(503).json({ status: 'not ready', error: err.message });
  }
});
```

8.2 Kubernetes Self-Healing

Kubernetes uses both probes automatically — failing liveness restarts the pod; failing readiness removes it from Service endpoints until it recovers, then re-adds it.

9. Graceful Degradation

When part of a system fails, continue providing reduced but functional service rather than failing completely. Amazon's product pages exemplify this — if the recommendations service fails, the page still loads with product details and checkout. The failure of one non-critical service does not take down the critical purchase flow.

- Feature flags — disable non-critical features during incidents without a deployment
- Cached fallbacks — serve stale data when live data is unavailable
- Read-only mode — allow reads when the write path is degraded
- Queue and process later — accept user actions and process them when the system recovers

10. Chaos Engineering

Chaos engineering deliberately introduces failures in a controlled way to discover weaknesses before they cause real outages. Netflix's Chaos Monkey randomly terminates production instances during business hours, proving their architecture is genuinely resilient rather than resilient only in theory.

Common experiments:

- Terminate random instances — does the load balancer reroute correctly?
- Inject network latency — does the circuit breaker open? Do timeouts trigger?
- Kill a database replica — does the app automatically reconnect to another?
- Terminate an availability zone — does geo-redundancy work as designed?

Interview Questions & Answers

Q1: What is fault tolerance and why is it important?

- Fault tolerance is the ability of a system to continue operating when one or more components fail
- In distributed systems, failures are inevitable — hardware fails, networks partition, services crash
- The goal is graceful degradation — individual component failures should not cause system-wide outages
- Measured as availability: 99.99% means only 52.6 minutes of downtime per year

Q2: Explain the circuit breaker pattern and its three states.

- CLOSED: requests pass through normally; failure rate is tracked in a rolling window
- OPEN: triggered when failure rate exceeds threshold; all requests fail instantly with no downstream call

- HALF-OPEN: after cooldown, test requests are allowed; success closes the circuit, failure reopens it
- Prevents cascading failures by stopping resource exhaustion from repeated calls to known-failing services

Q3: What is the difference between exponential backoff and jitter in retry strategies?

- Exponential backoff doubles wait time after each failure: 1s, 2s, 4s, 8s — gives the service more time to recover
- Without jitter, all clients retry at exactly the same times — the thundering herd can overwhelm a recovering service
- Jitter adds a random component to the delay, spreading retries across time
- Best practice: always combine exponential backoff with jitter in distributed systems

Q4: What is the bulkhead pattern?

- Isolates resource pools so one failing dependency cannot exhaust shared resources and bring down unrelated features
- Thread pool isolation: each downstream service gets its own thread pool
- Connection pool isolation: each consumer gets its own database connection pool with a hard cap
- Without bulkheads, a single slow service can consume all shared threads, starving every other feature

Q5: What is the difference between active-active and active-passive redundancy?

- Active-active: all instances serve real traffic simultaneously; failover is instant — load balancer stops routing to failed instance
- Active-passive: one instance serves traffic, others stand by; failover requires promoting a standby — takes seconds to minutes
- Active-active is more efficient (no idle capacity) but requires truly stateless application design
- Active-passive is simpler and common for databases where primary-replica promotion is well understood

Q6: What is the difference between liveness and readiness health checks?

- Liveness: is the process running and responsive? Failure triggers a container restart

- Readiness: is the app connected to dependencies and ready to serve? Failure removes it from load balancer rotation
- A pod can be live (process running) but not ready (still connecting to DB on startup)
- Never use liveness to check external dependencies — a DB being slow should not cause your pod to restart in a loop

Q7: What is graceful degradation?

- When part of a system fails, the remaining parts continue providing a reduced but functional experience
- Critical paths (checkout, authentication) remain operational even when non-critical features (recommendations, analytics) fail
- Implemented with feature flags, cached fallbacks, static responses, and read-only modes
- Goal: users notice reduced functionality, not a complete outage

Q8: What is chaos engineering?

- Deliberately introducing failures in a controlled way to discover weaknesses before real outages occur
- Netflix Chaos Monkey randomly kills production instances — forces teams to build genuinely resilient systems
- Experiments should start in staging, form a hypothesis, limit blast radius, and graduate to production incrementally
- An untested failover mechanism is not a failover mechanism — chaos engineering proves it works

Q9: How do you prevent cascading failures?

- Circuit breakers — stop calling a known-failing service before it exhausts resources
- Timeouts — never wait indefinitely; a hanging dependency without a timeout will eventually take down its caller
- Bulkheads — isolate thread and connection pools so one slow service cannot starve others
- Rate limiting — cap concurrent calls between services
- Graceful degradation — return fallback responses rather than propagating errors upstream

Q10: What are RTO and RPO?

- RTO (Recovery Time Objective): maximum acceptable downtime after a failure — how quickly must the system be restored?

- RPO (Recovery Point Objective): maximum acceptable data loss measured in time — how much data can be lost?
- RTO = 1 hour means the system must be back online within 1 hour of failure
- RPO = 5 minutes means at most 5 minutes of data can be lost in a failure scenario
- Tighter RTO/RPO requirements drive decisions about redundancy, backup frequency, replication mode, and failover speed — and cost significantly more

Final Takeaways

Mental Model: A fault-tolerant system is like a ship with watertight compartments. One compartment flooding does not sink the ship. Design every component assuming the others will fail — because they will.

- Failures are inevitable — design for failure, not against it; the goal is graceful degradation, not zero failures
- Retries with exponential backoff and jitter handle transient failures; idempotency keys make retries safe for write operations
- Circuit breakers prevent cascading failures by stopping calls to known-failing services before resources are exhausted
- Timeouts are mandatory on every external call — a hanging dependency without a timeout will eventually bring down its caller
- Bulkheads isolate failure domains — one slow service cannot starve another through shared resource exhaustion
- Redundancy eliminates single points of failure — active-active for zero downtime, active-passive for simpler failover
- Liveness checks trigger restarts; readiness checks manage traffic — never confuse them
- Test your resilience with chaos engineering — an untested failover mechanism is not a failover mechanism

FINAL CHAPTER

How to Approach System Design

A Step-by-Step Framework for Any System Design Problem

Bringing together every concept in this guide into one repeatable process

System design is not a test of memorising patterns — it is a test of structured thinking. There is no single correct answer to any system design question. What interviewers and senior engineers evaluate is your ability to reason through trade-offs, ask the right questions, communicate clearly, and build a coherent architecture that matches the

actual requirements. The 8-step framework below gives you a repeatable, structured approach to any system design problem, whether in an interview or on the job.

Golden rule: Never jump straight into architecture. Engineers who start drawing boxes and arrows before clarifying requirements almost always design the wrong system. Slow down at the start to move faster later.

The 8-Step Framework at a Glance

Step	Name	What You Are Doing
1	Clarify Requirements	Understand what you are building — functional and non-functional requirements
2	Estimate Scale	Quantify the problem — users, data volume, read/write ratio, throughput
3	Define APIs	Specify the interface — what does the system expose to clients?
4	Choose Architecture	Monolith or microservices? How are components organised?
5	Design the Database	What data model? SQL or NoSQL? Schema design and access patterns
6	Add Caching	What is read-heavy? What data can be served from memory?
7	Plan Scaling	How does the system grow? Horizontal scaling, replication, sharding
8	Handle Failures	What happens when things break? Retries, circuit breakers, redundancy

Step 1 — Clarify Requirements

This is the most important step and the one most people skip. Before drawing a single component, you must understand what you are actually building. Requirements come in two categories, and you need both.

1.1 Functional Requirements — What the System Does

Functional requirements define the core features and behaviours the system must support. These are the actions users can perform and the outcomes they expect.

Questions to ask:

- What are the core user actions? (create, read, update, delete — what specifically?)
- Who are the users? (end users, admins, external APIs, internal services?)
- What are the critical user journeys? (sign up → post → feed, search → view → purchase)
- Are there any features that are explicitly out of scope for this design?

Example — designing a URL shortener:

- Users can submit a long URL and receive a short URL
- Users can visit the short URL and be redirected to the original URL
- Users can optionally choose a custom short code
- Out of scope: analytics dashboard, A/B testing, link expiry (unless stated)

1.2 Non-Functional Requirements — How the System Performs

Non-functional requirements define quality attributes — how the system should behave rather than what it should do. These drive the vast majority of architecture decisions.

Questions to ask:

- What is the target availability? (99.9%, 99.99%? — this drives redundancy decisions)
- What is the acceptable response latency? (<100ms, <500ms, <2s?)
- Is the system read-heavy or write-heavy? (drives caching and database choices)
- Is consistency critical or is eventual consistency acceptable? (CAP theorem choice)
- Are there compliance requirements? (GDPR, HIPAA, PCI-DSS affect data storage and encryption)
- What are the durability requirements? (can we lose data, even momentarily?)

Interview tip: Spend 5-10 minutes on requirements. State your assumptions out loud. Interviewers want to see that you do not design in a vacuum — you design for a specific problem with specific constraints.

Step 2 — Estimate Scale

Scale estimation forces you to confront the actual size of the problem. A system serving 1,000 users and a system serving 100 million users look completely different

architecturally. Without scale estimates, you cannot make informed decisions about database choice, caching strategy, or the need for horizontal scaling.

You do not need exact numbers — rough order-of-magnitude estimates are sufficient and expected. The goal is to understand whether you are dealing with kilobytes or terabytes, thousands of requests or millions.

2.1 Key Numbers to Estimate

Users:

- How many total registered users?
- How many daily active users (DAU)?
- What is the peak concurrent user count? (often 10-20% of DAU)

Throughput:

```
Reads per second = DAU × reads per user per day / 86,400
Writes per second = DAU × writes per user per day / 86,400
```

Example — Twitter-like system:

```
100M DAU, each user reads 100 tweets/day, writes 1 tweet/day
Read RPS = 100,000,000 × 100 / 86,400 ≈ 115,000 reads/second
Write RPS = 100,000,000 × 1 / 86,400 ≈ 1,150 writes/second
→ Heavily read-dominated (100:1 read-write ratio)
```

Storage:

```
Storage per day = writes/second × record size × 86,400
Storage per year = storage per day × 365
```

Example — tweet storage:

```
1,150 writes/s × 300 bytes × 86,400 = ~29 GB/day
~29 GB × 365 = ~10.5 TB per year
→ Needs distributed storage; single-server disk is not enough
```

2.2 What Estimates Drive

- Read:write ratio above 10:1 → caching is highly effective; invest there
- Write throughput above 10,000/s → single database primary will struggle; plan sharding
- Storage in terabytes → distributed object storage, not a single relational database
- Peak concurrent users in millions → multiple app server instances, auto-scaling mandatory

Step 3 — Define APIs

Before designing internal components, define the system's external interface. The API is the contract between your system and its clients — defining it first ensures your architecture actually serves the user journeys you identified in Step 1.

3.1 REST API Design

For most web systems, define the key REST endpoints. Use resource-oriented naming, appropriate HTTP methods, and include request/response shapes.

```
URL Shortener API:

POST    /urls
  Request: { longUrl: string, customCode?: string }
  Response: { shortUrl: string, code: string, createdAt: string }

GET     /urls/:code
  Response: 301 Redirect to longUrl
            (or 404 if code not found)

GET     /urls/:code/stats          (if analytics in scope)
  Response: { clicks: number, uniqueVisitors: number, topCountries: [] }
```

3.2 Why API Definition Comes Before Architecture

- APIs reveal the read/write patterns — GET-heavy systems lean toward caching; POST-heavy systems lean toward reliable queuing
- APIs expose consistency requirements — redirects must return the correct URL, so that field cannot be stale
- APIs define what the database must support — the query patterns in Step 5 come directly from the API operations
- APIs make the scope concrete — everything inside the system serves these endpoints and nothing else

Step 4 — Choose Architecture

With requirements, scale, and APIs defined, you can now make an informed architectural decision. The core question: should this be a monolith or microservices? And how should components be organised?

4.1 The Decision

Start with a monolith when:

- The team is small or the product is early-stage
- The domain boundaries are not yet clear
- Time-to-market is more important than scalability

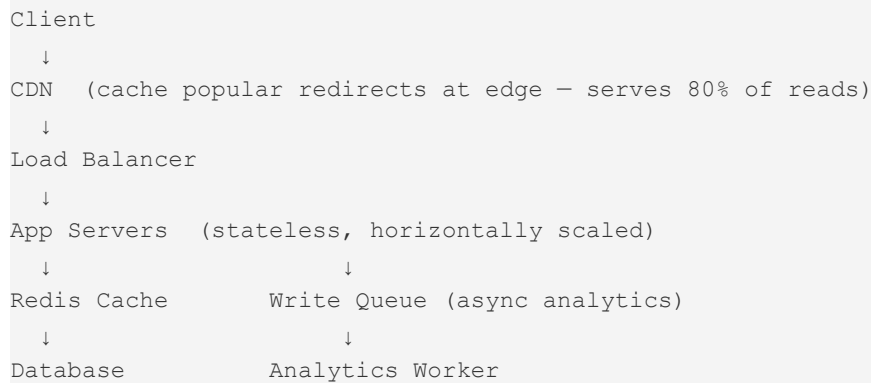
Move to microservices when:

- Different parts of the system have very different scaling requirements
- Multiple teams need to deploy independently
- The system has clear domain boundaries that map to separate services

4.2 High-Level Architecture Sketch

Draw the major components and how they connect. At this stage, a high-level diagram is more valuable than detailed implementation. Focus on the data flow from client to database and back.

URL Shortener — High-Level Architecture:



Step 5 — Design the Database Layer

The database choice is one of the most consequential decisions in system design. It must be driven by the data model, access patterns, consistency requirements, and scale — not by what is trendy.

5.1 Choose SQL or NoSQL

Choose SQL (PostgreSQL, MySQL) when:

- Data is relational and relationships need to be enforced
- ACID transactions are required (payments, inventory, order management)
- Complex queries with JOINS are needed

Choose NoSQL when:

- Schema is flexible or changes frequently
- Horizontal scalability is the primary requirement
- Data access patterns are simple and known in advance (Cassandra's query-driven design)

5.2 Design the Schema Around Access Patterns

Every table and index should be driven by the queries your API will run. Work backwards from the API endpoints defined in Step 3.

URL Shortener – Database Design:

Table: urls

id	BIGINT PRIMARY KEY	(auto-increment or snowflake ID)
code	VARCHAR(10) UNIQUE	(the short code – indexed)
long_url	TEXT NOT NULL	
user_id	BIGINT	(nullable – anonymous allowed)
created_at	TIMESTAMP DEFAULT NOW()	
expires_at	TIMESTAMP	(nullable)

Indexes:

PRIMARY KEY (id)
UNIQUE INDEX idx_code (code) -- critical: every redirect looks up by code

5.3 Plan for Data Growth

- If single-table storage will exceed a few TB: plan read replicas for read scaling
- If write throughput will exceed what one primary can handle: plan sharding strategy
- Identify the shard key early — changing it later is extremely painful
- For time-series or append-only data: consider Cassandra or TimescaleDB from the start

Step 6 — Add Caching

Once the database layer is designed, identify what can be served from cache instead. Caching is the highest-impact optimisation for read-heavy systems — a well-placed cache layer can reduce database load by 80-90% and cut response latency by an order of magnitude.

6.1 What to Cache

- Frequently read, infrequently changing data — user profiles, product listings, configuration
- Expensive computed results — aggregations, recommendation outputs, search results
- The result of any database query that the same user or many users will request repeatedly

URL Shortener caching strategy:

```
Cache key: 'url:abc123'
Cache value: 'https://www.longdestinationurl.com/some/long/path'
TTL:      24 hours (URLs rarely change; a day of staleness is acceptable)

Flow:
GET /urls/abc123
→ Check Redis: cache hit → 301 redirect (sub-millisecond)
→ Cache miss → query DB → store in Redis → 301 redirect
```

6.2 Where to Cache

- Redis — server-side, shared across all app instances, the standard choice
- CDN — for publicly accessible redirects and static assets, serves from the network edge
- Application memory — in-process cache for configuration and reference data that changes very rarely

6.3 Cache Invalidation Strategy

- TTL-based: set an appropriate expiry; accept that data may be stale within the TTL window
- Explicit invalidation: when a URL is updated or deleted, immediately delete the cache key
- Write-through: update the cache at the same time as the database on every write

Step 7 — Plan Scaling

With the core architecture defined, think through how each layer scales as traffic grows. Identify potential bottlenecks before they occur and have a plan for each one.

7.1 Application Layer Scaling

- App servers should be stateless — all session state in Redis, not in server memory
- Add servers horizontally behind the load balancer as traffic increases

- Configure auto-scaling groups to add instances when CPU or request queue exceeds thresholds
- Use blue-green or rolling deployments for zero-downtime releases

7.2 Database Layer Scaling

Traffic level	Solution
Low	Single primary database + Redis cache
Medium	Add read replicas – distribute read queries
High	Connection pooling (PgBouncer) + more replicas
Very high	Shard the database by user_id or region
Extreme	Consider NoSQL (Cassandra, DynamoDB) for specific tables

7.3 Cache Layer Scaling

- Single Redis node handles millions of operations per second for most systems
- Redis Cluster partitions data across multiple nodes for larger datasets
- CDN edge caching for publicly accessible content scales to any global traffic level

7.4 Identifying Bottlenecks Proactively

- The database is almost always the first bottleneck — plan caching and read replicas early
- The load balancer itself can become a bottleneck at very high traffic — use cloud-managed LBs with auto-scaling
- Network bandwidth limits — especially for media-heavy systems — plan CDN offloading from the start
- External API rate limits — plan queuing and batching if you depend on third-party services

Step 8 — Handle Failures

The final step is making the system resilient. Every dependency will eventually fail. Every component will eventually experience a bug or an overload. The architecture must account for these events so that partial failures do not become total outages.

8.1 Identify Single Points of Failure

Walk through your architecture and ask: what happens if this component fails? Any component whose failure takes the entire system offline is a single point of failure and needs redundancy.

Component	SPOF?	Solution
Load balancer	Yes	Active-passive pair or cloud-managed LB
App servers	No	Multiple instances – one fails, others continue
Redis cache	Yes	Redis Sentinel or Cluster for HA
Primary DB	Yes	Synchronous replica for failover
CDN	No	Multiple PoPs – regional failure is isolated

8.2 Apply the Fault Tolerance Toolkit

- Timeouts on every external call — set based on P99 response time, never infinite
- Retries with exponential backoff and jitter for transient failures
- Circuit breakers on all downstream service calls — prevent cascading failures
- Bulkhead isolation — separate thread and connection pools per downstream dependency
- Health checks — liveness for process recovery, readiness for traffic management

8.3 Graceful Degradation Plan

For every non-critical feature, define what happens when it fails. Users should experience reduced functionality, not a broken page.

```
URL Shortener – degradation plan:
Redis down:      Fall back to direct DB lookup (slower but functional)
DB replica down: Route reads to primary temporarily (increased load)
Analytics down:  Queue events – process when service recovers
CDN down:        All traffic falls back to origin (higher load, higher
latency)
```

Putting It All Together — Worked Example: URL Shortener

Here is how the 8-step framework produces a complete design for a URL shortening service similar to bit.ly.

Step 1 — Requirements:

- Functional: create short URLs, redirect to long URLs, custom codes optional
- Non-functional: redirect latency < 50ms, 99.99% availability, 10B URLs stored

Step 2 — Scale:

- 100M DAU, 1B redirects/day = 11,600 redirects/second (read-heavy: 100:1)
- 10B URLs × 500 bytes = 5 TB storage over 5 years

Step 3 — APIs:

- POST /urls → { shortUrl, code }
- GET /urls/:code → 301 Redirect

Step 4 — Architecture:

- Monolith initially, split write (URL creation) and read (redirect) paths
- CDN at the edge to cache popular redirects

Step 5 — Database:

- PostgreSQL: urls table with UNIQUE index on code
- At scale: shard by code hash across multiple DB instances

Step 6 — Caching:

- Redis: cache url:CODE → long_url with 24h TTL
- CDN: cache redirect responses at edge for popular codes

Step 7 — Scaling:

- Stateless app servers behind ALB with auto-scaling
- Read replicas for DB → shard at 10TB+

Step 8 — Failures:

- Redis down → fall back to DB
- DB replica down → route to primary
- App server down → LB health check removes it in seconds

System Design Interview Cheat Sheet

Time allocation for a 45-minute system design interview: Requirements 5-8 min • Scale 3-5 min • APIs 3-5 min • Architecture 5-8 min • Database 5-8 min • Caching 3-5 min • Scaling + Failures 5-8 min • Q&A and deep dives: remainder

Questions that show senior-level thinking:

- What consistency model does this require — strong or eventual? (CAP Theorem)
- Where is the read:write ratio and how does it affect our caching strategy?
- What is the most likely bottleneck at 10x our initial scale?
- Which components are single points of failure and how do we eliminate them?
- What is the blast radius if the cache layer goes down?
- How do we handle the thundering herd when a popular cache entry expires?
- Should this be synchronous or should we queue it and process asynchronously?

Red flags to avoid:

- Jumping straight to architecture without clarifying requirements
- Designing a monolith vs microservices debate without knowing the team size or scale
- Choosing a database without justifying it against the access patterns
- Adding caching everywhere without specifying the invalidation strategy
- Designing scaling without identifying where the actual bottleneck is
- Ignoring failure modes — every system design must address what happens when things break

How Every Chapter in This Guide Connects

Every chapter in this guide is a tool in the system designer's toolkit. Here is how they map to the 8-step framework:

Step	Chapter	Key Concept Applied
1. Requirements	Ch 10 — CAP Theorem	Consistency vs availability choice
2. Scale Estimate	Ch 7 — Scaling	Horizontal vs vertical, auto-scaling
3. Define APIs	Ch 12 — GraphQL	REST vs GraphQL tradeoffs
4. Architecture	Ch 1 — Monolith vs Microservices	When to split services
4. Architecture	Ch 2,3 — Load Balancing	Traffic distribution design
5. Database	Ch 4 — Database Handbook	SQL vs NoSQL, ACID, transactions
5. Database	Ch 11 — Indexing	Query optimization, index design
6. Caching	Ch 5 — Caching	Strategies, TTL, invalidation
6. Caching	Ch 6 — CDN	Edge caching for static assets
7. Scaling	Ch 7 — Scaling	Horizontal scaling patterns
7. Scaling	Ch 8 — Message Queues	Async decoupling for scale
8. Failures	Ch 13 — Fault Tolerance	Retries, circuit breakers, redundancy
Security	Ch 9 — Auth	JWT, RBAC, API gateway security

You have reached the end of the System Design Guide

[13 Chapters](#) • [130+ Interview Q&As](#) • [Real-World Patterns](#) • [Production Code](#)

The best system designers are not the ones who know the most patterns.

*They are the ones who ask the best questions,
reason through trade-offs clearly,
and design systems that match the actual problem.*

Now you have both.